Luis Enrique Vásquez Prado, BSc

Understanding Graph Neural Network Performance in Semi-Supervised Node Classification under Label Scarcity Conditions

MASTER'S THESIS

to achieve the university degree of

Master of Science

Master's degree programme: Software Engineering and Management

submitted to

Graz University of Technology

Supervisor

Denis Helic, Assoc.Prof. Dipl.-Ing. Dr.techn.
Institute of Human-Centered Computing
Hussain Hussain, Dipl.-Ing. Dr.techn.
Know Center Research GmbH

Graz, October 2026

Abstract

Graph Neural Networks (GNNs) are known to outperform graph-agnostic models in semi-supervised node classification under certain conditions. This advantage is commonly associated with the homophily phenomenon, the tendency of nodes from the same class to connect to each other. The goal of this work is to study the mechanisms that determine GNN performance and to investigate whether their advantage over graph-agnostic models can be reliably predicted under label scarcity conditions. For this, we use the Graph Convolutional Network (GCN) and the Multi-Layer Perceptron (MLP) as the canonical graph-aware and graph-agnostic architectures, respectively. We compare the accuracy of the models in synthetic as well as empirical networks, relate this performance with homophily, and experimentally test to which extent homophily estimators computed using a subset of labeled nodes in the graph are able to predict the advantage of graph-aware models before model training. We show how synthetic graphs reveal the main structural mechanism that drives GNN performance, i.e. class-level node distinguishability, the advantage of GNNs can therefore be faithfully predicted in this setting, yet the structurally heterogeneous nature of empirical datasets make the estimation, especially under label scarcity, more challenging. We provide guidelines on which label-, feature- and topology-based metrics best serve as proxies to estimate the potential advantage of graph-aware models, both in synthetic and empirical networks, and show how these can be used in label-scarce settings. Finally, we derive that model selection under label scarcity should leverage graph properties beyond label homophily, especially in empirical datasets, where model performance is driven by multiple factors. Overall, the work offers comprehensive guidelines for understanding and predicting GNN advantage under limited supervision.

Kurzfassung

Es ist bekannt, dass Graph Neural Networks (*Graph-neuronale Netzwerke*, GNNs) unter bestimmten Bedingungen graphunabhängige Modelle bei der semi-überwachten Knotenklassifikation übertreffen. Ein solcher Vorteil wird häufig mit dem Phänomen der Homophilie in Verbindung gebracht, also der Tendenz von Knoten derselben Klasse, miteinander verbunden zu sein. Ziel dieser Arbeit ist es einerseits, jene Mechanismen zu analysieren, welche die Leistung von GNNs bestimmen und andererseits zu untersuchen, ob deren Vorteil gegenüber graphunabhängigen Modellen bei geringer Verfügbarkeit von Labels zuverlässig vorhergesagt werden kann. Zu diesem Zweck verwenden wir das Graph Convolutional Network (GCN) und das Multi-Layer Perceptron (MLP) als kanonische graphbasierte bzw. graphunabhängige Architektur. Wir vergleichen die Klassifikationsgenauigkeit (Accuracy) der Modelle auf synthetischen sowie auf empirischen Netzwerken und setzen diese Leistung in Beziehung zur Homophilie. Zudem prüfen wir experimentell, inwieweit Homophilie-Metriken, die anhand einer Teilmenge gelabelter Knoten berechnet werden, den Vorteil graphbasierter Modelle bereits vor Modelltraining vorhersagen können. Die Ergebnisse zeigen, dass synthetische Graphen den zentralen strukturellen Mechanismus hinter der Leistung von GNNs offenlegen, nämlich die Unterscheidbarkeit von Knoten auf Klassenebene. Der Vorteil von GNNs lässt sich daher in diesem Kontext zuverlässig vorhersagen. Trotzdem bleibt die Leistungsschätzung von GNNs auf empirischen Datensätzen eine Herausforderung, insbesondere bei geringer Verfügbarkeit von Labels. Wir stellen Leitlinien zur Verfügung, welche label-, merkmals- und topologiebasierten Homophilie-Metriken am besten als Prädiktoren des Vorteils graphbasierter Modelle auf synthetischen und empirischen Netzwerken dienen und zeigen, wie diese bei geringer Verfügbarkeit von Labels genutzt werden können. Zuletzt leiten wir ab, dass die Modellauswahl bei geringer Verfügbarkeit von Labels strukturelle Eigenschaften über Label-Homophilie hinaus ausnutzen sollte, insbesondere bei empirischen Datensätzen, bei denen die Modellleistung von mehreren Faktoren bestimmt wird. Insgesamt bietet diese Arbeit umfassende Leitlinien zum Verständnis und zur Vorhersage des Vorteils von GNNs bei begrenzter Überwachung.

Acknowledgements

I'd like to express my deepest gratitude to my main supervisor, Denis Helic, not only for all his invaluable inputs and feedback throughout the entire process of this thesis, but also for his guidance and trust for the next steps of my professional career. Furthermore, I'd like to thank Hussain Hussain, my second supervisor, for all the time and effort dedicated to this endeavor. Thank you for all the insightful conversations and the countless hours invested mentoring me. I look forward to our future collaborations.

Este trabajo no hubiese sido posible sin todo el respaldo de mi extensa red de apoyo. Para mis papás, Mónica y Luis Miguel, por su amor incondicional, por creer en mí y por apoyarme en todos y cada uno de mis proyectos. Cada uno de mis logros ha sido fruto de su esfuerzo y tiene su nombre tanto como el mío. Para mis hermanos, Alejandro y Diana Carolina, que este trabajo les sirva de inspiración para sus futuros proyectos. Para Mariale, por acompañarme cada día, bueno o malo, durante la realización de la tesis y a lo largo de la maestría. Gracias por creer en mí los días que yo no lo hice, por tu paciencia infinita y por tus palabras de aliento. Para mis nuevas amistades en Graz, por hacer el proceso de adaptación a una nueva cultura más sencillo. Finalmente, para mis abuelos, especialmente para Lola, quien me vio emprender este camino, y quien ahora me verá concluirlo desde arriba.

Contents

1. Introduction **19**

2. Background **23**
2.1. Basic concepts and terminology . . . 23
2.1.1. Graph fundamentals . . . 23
2.1.2. Attributed graphs . . . 25
2.2. Machine learning on graphs . . . 26
2.2.1. Tasks . . . 26
2.2.2. Node embeddings . . . 27
2.2.3. Node embedding models . . . 27
2.2.4. Graph Neural Networks . . . 29
2.2.5. Modern GNN architectures . . . 34
2.3. Semi-supervised learning . . . 37
2.3.1. Self-training, Co-training . . . 38
2.3.2. Semi-supervised learning on graphs . . . 39
2.4. Network analysis . . . 40
2.4.1. Characterizing networks . . . 40
2.4.2. Homophily . . . 44
2.4.3. Synthetic graph generation models . . . 51

3. Related work **55**
3.1. When do GNNs outperform graph-agnostic models? . . . 55
3.2. GNNs for heterophily . . . 56
3.3. Estimating GNN performance beyond homophily . . . 57
3.4. The label scarcity regime . . . 59
3.5. Research gap . . . 60

4. Approach **61**
4.1. Task definition . . . 61
4.2. Pipeline description . . . 62
4.2.1. Synthetic pipeline . . . 62
4.2.2. Empirical pipeline . . . 64
4.3. Synthetic graph generation . . . 64
4.4. Empirical datasets . . . 66
4.4.1. Exploratory Data Analysis . . . 68
4.5. Proxies in the label-scarce setting . . . 70

4.6. Training . . . 71
4.6.1. Models . . . 72
4.6.2. Hyperparameters . . . 73
4.7. Gain characterization . . . 73
4.8. Evaluation . . . 74
4.8.1. Ground-truth homophily . . . 74
4.8.2. Node distinguishability . . . 75
4.8.3. Effect of homophily and supervision regime . . . 76
4.8.4. Gain estimation . . . 76

5. Experiments and Results **79**
5.1. Synthetic datasets . . . 79
5.1.1. GCN feature aggregation . . . 79
5.1.2. Training on CSBM-H graphs . . . 82
5.1.3. Gain estimation . . . 90
5.2. Empirical datasets . . . 97
5.2.1. Characterizing label homophily measures . . . 97
5.2.2. Performance degradation analysis . . . 99
5.2.3. Gain estimation . . . 104

6. Discussion **111**

7. Conclusion and Future Work **115**

Bibliography **117**

A. ANOVA tables for the multiclass CSBM-H **127**

B. Accuracy and gain plots of empirical datasets **129**

List of Figures

2.1. Adjacency, Degree and Laplacian matrices of an unweighted, undirected graph. 24

2.2. Given a graph $G = (\mathcal{V}_G, \mathcal{E}_G)$ with $\mathcal{V}_G = \{v_1, v_2, v_3, v_4, v_5, v_6\}$ and $\mathcal{E}_G = \{\{v_1, v_2\}, \{v_1, v_5\}, \{v_1, v_6\}, \{v_2, v_3\}, \{v_2, v_6\}, \{v_3, v_4\}, \{v_3, v_6\}, \{v_4, v_5\}, \{v_4, v_6\}, \{v_5, v_6\}\}$, $H = (\mathcal{V}_H, \mathcal{E}_H)$ is a subgraph of G since $\mathcal{V}_H \subseteq \mathcal{V}_G$ and $\mathcal{E}_H \subseteq \mathcal{E}_G$, and $G[\mathcal{U}]$ is the corresponding induced subgraph, where $\mathcal{U} = \mathcal{V}_H = \{v_1, v_2, v_3, v_6\}$. . 25

2.3. The SSNC task: given sets of labeled and unlabeled samples (represented by colored and gray nodes respectively), predict the classes of the unlabeled nodes using the available labels, all nodes' feature vectors and the graph's structure (edge index). 39

2.4. Degree distributions (on a log-log scale) of the Tolokers and Questions datasets. Both exhibit heavy-tailed degree distributions, with a vast number of low degree nodes and a low number of very high degree nodes ("hub nodes"), with Tolokers having a more concentrated number of intermediate degree nodes. 41

2.5. Synthetically generated networks with different levels of (label) homophily, where colors correspond to class membership. 44

2.6. A network generated through the CSBM-H model (node colors indicate class membership, where c_0 is the blue class) and its nodes' 2-dimensional feature space, with parameters $n_0 = 100$, $n_1 = 50$, $d_0 = 2$, $d_1 = 5$, $\boldsymbol{\mu}_0 = (0, 0)$, $\boldsymbol{\mu}_1 = (3, 3)$, $\sigma_0 = 1$, $\sigma_1 = 2$ and $h = 0.7$. 53

4.1. Synthetic pipeline, which uses CSBM-H graphs as inputs and evaluates homophily on a predefined supervision regime. 63

4.2. Empirical pipeline, which uses real-world graphs as inputs and evaluates homophily on different label rates per dataset. 65

4.3. Class distributions of the empirical datasets. 68

4.4. Network structure and edge distribution bubble plots for Cora and Roman-Empire. Bubble sizes represent the number of edges between pairs of classes. Roman-Empire's graph visualization is based on a sub-sampling of 5,000 nodes. 70

5.1. A representative instance of the 2-class CSBM-H model, rewired to achieve different levels of homophily, and feature distributions of the original features $\mathbf{X}$, the 1-hop aggregated features $\mathbf{X}^{(1)}$, and the 2-hop aggregated features $\mathbf{X}^{(2)}$. 81

5.2. MLP and GCN test set accuracy along all homophily values, with values averaged over runs and ±1 standard deviation bands. Coloring indicates different label rates for both plots. 82
5.3. Gain characterization under different homophily and label rate levels. . . 83
5.4. Absolute gain as a function of h and lr for 2-class CSBM-H graphs. . . . 84
5.5. Gain plots for multiclass CSBM-H graphs with $C \in \{3, 4, 5\}$. 90
5.6. Label-based estimators on the homophilic regime ($h \geq 0.5$). The top row shows the relationship between Gain_{abs} and h_{label} for each metric computed on the entire graph G, and the bottom row shows the relationship between Gain_{abs} and $\hat{h}_{\text{label}}$ for each metric computed on $G[\mathcal{V}_{\text{train}}]$. Each point represents a CSBM-H graph and color represents the value of the CSBM-H parameter h. 92
5.7. Label-based estimators on the heterophilic regime ($h < 0.5$). The top row shows the relationship between Gain_{abs} and h_{label} for each metric computed on the entire graph G, and the bottom row shows the relationship between Gain_{abs} and $\hat{h}_{\text{label}}$ for each metric computed on $G[\mathcal{V}_{\text{train}}]$. Each point represents a CSBM-H graph and color represents the value of the CSBM-H parameter h. 93
5.8. Label-based estimators error distribution ($e_i = \frac{|h_i - \hat{h}_i|}{h_{\max} - h_{\min}}$) for each metric with the corresponding smooth KDE line. The y-axis represents the number of observations in the corresponding error range. 94
5.9. Feature-based estimators on the homophilic and heterophilic regimes. Each point represents a CSBM-H graph and color represents the value of the CSBM-H parameter h. 95
5.10. Structure-based estimators on the homophilic and heterophilic regimes. Each point represents a CSBM-H graph and color represents the value of the CSBM-H parameter h. 96
5.11. Pairwise relationships between homophily measures. The upper triangle shows the pairwise Pearson correlation coefficients, with Bonferroni correction ($\alpha' \approx 0.0024$), where $*$ denotes $p < \alpha'$. The lower triangle shows the pairwise scatterplots and linear regression lines with 95% confidence intervals. The diagonal shows the marginal distribution of each measure. . 99
5.12. Distribution of test set accuracies for GCN and MLP over 50 runs on each dataset at $lr = 0.20$. Datasets are grouped into heterophilic (left panel), neutral (middle panel) and homophilic (right panel) according to h_{unb}. . . 100
5.13. Accuracy and gain plot for a representative homophilic dataset, Cora ($h_{\text{unb}} = 0.92$), trained at different levels of label scarcity. Label rates are approximate due to sampling. 101
5.14. Accuracy and gain plot for a representative heterophilic dataset, Roman-Empire ($h_{\text{unb}} = -0.4913$), trained at different levels of label scarcity. Label rates are approximate due to sampling. 102

5.15. Accuracy and gain plot for a representative neutral datasets, Chameleon ($h_{\text{unb}} = 0.0742$) and Actor ($h_{\text{unb}} = 0.0053$), trained at different levels of label scarcity. Label rates are approximate due to sampling. 103
5.16. $\text{Gain}_{\text{abs}}@lr$ plots for different supervision regimes with lines of best linear fit and 95% confidence intervals. Observations represent every dataset's mean gain after training at the given label rate, with ±1 standard deviation bars. Markers indicate the homophily regime of each dataset. 107

B.1. Homophilic datasets. 129
B.2. Neutral datasets. 130
B.3. Heterophilic datasets. 131

List of Tables

2.1. Families of models for node embedding. 28
2.2. GNN-MLP correspondence. 33

4.1. Parameters for CSBM-H graph generation. 66
4.2. Empirical benchmark datasets summary. 67
4.3. Empirical dataset characterization. 69
4.4. Homophily metrics used in the predictive task. 71
4.5. Training hyperparameters used for both models. 73

5.1. SNR on the original and aggregated features for different homophily regimes. 80
5.2. Two-Way Repeated Measures ANOVA test results. 86
5.3. One-Way Repeated Measures ANOVA results. 88
5.4. Two-way repeated-measures ANOVA results for $C \in \{3, 4, 5\}$. 91
5.5. Spearman correlation ρ_s between label-based homophily metrics and Gain_{abs}. We highlight the strongest correlation in each case. 93
5.6. Normalized root mean squared error (NRMSE) between the estimated and ground-truth label-based homophily metrics. We highlight the lowest expected error. 94
5.7. Spearman correlation ρ_s between feature- and structure-based metrics and Gain_{abs}. We highlight the strongest correlation in each case. 96
5.8. Homophily metrics at full label visibility, ranked by h_{unb} and separated according to their homophily regime (homophilic, neutral, heterophilic). . 97
5.9. Spearman correlation between label-based estimators and the absolute gain obtained when training models at different label rates. 105
5.10. Normalized root mean squared error (NRMSE @lr) between the estimated and ground-truth label-based homophily metrics at different label rates. We highlight the lowest error at each label rate. 108
5.11. Spearman correlation between feature- and structure-based estimators with the absolute gain obtained when training models at different label rates. 109

A.1. One-Way Repeated Measures ANOVA results for $C = 3$. 127
A.2. One-Way Repeated Measures ANOVA results for $C = 4$. 127
A.3. One-Way Repeated Measures ANOVA results for $C = 5$. 128

List of Acronyms and Symbols

ANOVA	Analysis of Variance
BP	Belief Propagation
CCNS	Cross-Class Neighborhood Similarity
CPM	Classifier-based Performance Metric
CSBM-H	Contextual Stochastic Block Model for Homophily/Heterophily
CSBM-3H	Contextual Stochastic Block Model with three types of Homophily
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GIN	Graph Isomorphism Network
GNN	Graph Neural Network
HSBM	Heterophilous Stochastic Block Model
ICA	Iterative Classification Algorithm
LCC	Local Clustering Coefficient
LI	Label Informativeness
LP	Label Propagation
ML	Machine Learning
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
ND	Node Distinguishability
NRMSE	Normalized Root Mean Squared Error
NS	Neighborhood Similarity
NSV	Normalized Smoothness Value
NTV	Normalized Total Variation
ReLU	Rectified Linear Unit
RMI	Relative Mutual Information
SBM	Stochastic Block Model
SNR	Signal-to-Noise Ratio
SSNC	Semi-Supervised Node Classification
SSL	Semi-Supervised Learning
WL	Weisfeiler-Leman

1. Introduction

Historically, Graph Neural Network (GNN) performance in semi-supervised node classification has been linked to homophily (Kipf and Welling 2017; Y. Ma et al. 2023; Luan, Hua, M. Xu, et al. 2024; Platonov, Kuznedelev, Diskin, et al. 2024), the tendency of nodes to connect to nodes from the same class. This builds upon the original definition of message passing neural networks like the Graph Convolutional Network (GCN) as low-pass filters, which iteratively average the signals coming from neighboring nodes to update each node's embedding, such that neighboring nodes tend to end up with similar representations and predict similar labels. Therefore, in networks with high homophily, message passing produces embeddings that are similar across nodes of the same class, which results in nodes being easily classifiable, whereas in heterophilic (or anti-homophilic) networks, message passing tends to mix signals coming from neighbors from different classes, therefore making nodes less distinguishable and hurting classification.

However, the claim that homophily alone drives GNN performance has been increasingly challenged in recent years (Zheng, Luan, and Lihui Chen 2024; Y. Ma et al. 2023; Platonov, Kuznedelev, Diskin, et al. 2024). Particularly, it has been shown that, for heterophilic datasets, it doesn't forcefully hold that graph-aware architectures like GCN underperform graph-agnostic models, i.e. models that learn from data without observing its relational structure, like the Multi-Layer Perceptron (MLP). Thus, the metrics traditionally used to quantify homophily are not fully reliable in anticipating whether graph-aware models should be used in a specific dataset. Furthermore, label-based homophily metrics used to predict the advantage of GNNs are typically computed using the entire set of node labels, which leads to an inherent inconsistency: if all labels are available in the first place, then fitting a model for node class prediction is vacuous, assuming a transductive setting. This highlights the need of reliable estimators of GNN advantage when only a subset of the labels are visible.

From this identified gap, we frame the problem as follows: given a strictly label-scarce setting, where only the labels of the nodes in the training set are visible, we want to reliably select the model that will potentially have the strongest accuracy in node classification prior to training, which should be done using this subset of labeled nodes or any other information available in the graph in question. This opens the door for metrics that serve as proxies of GNN performance which exploit not only label information but also the nodes' feature vectors as well as the graph's structural properties, which we assume we can compute even on a graph with incomplete label information. While some metrics have shown to correlate with GNN performance, we test their robustness in different settings and their applicability in the label scarce setting.

Ultimately, the ability to anticipate which model family to select is fundamental from two main perspectives beyond the predictive performance itself. On the one side, graph-

aware models can be unnecessarily complex and computationally expensive due to repeated neighbor aggregation, and are graph-dependent and sensitive to edge noise during inference (Kohn, Hoffmann, and Scherp 2024; Huang et al. 2020). On the other side, and perhaps more importantly, it helps us understand the underlying mechanisms that drive GNN performance in order to design architectures that exploit these mechanisms and are applicable independently of the dataset.

Therefore, the present work is a systematic study of the effect of label rate on the relationship between the performance *gain* of the GCN as the canonical graph-aware layer (Kipf and Welling 2017) w.r.t. the MLP as its graph-agnostic counterpart, its relation with homophily, and of gain estimation under label scarcity. The goal is to answer the following research questions:

- **RQ1** (descriptive task): What is the effect of label rate on the performance gap between graph-aware and graph-agnostic models for semi-supervised node classification? Is there a connection of this effect with homophily?

- **RQ2** (predictive task): How can we reliably predict the advantage (i.e. gain) of graph-aware models over graph-agnostic models under label scarcity?

The first question relates to the influence of the supervision regime on the performance of each model. We want to characterize the advantage of GCN over MLP, i.e. the gain, and determine whether its behavior follows some pattern as we modify the supervision regime and across datasets with different levels of homophily. For this, we perform controlled experiments on synthetic datasets and then validate the effects in empirical datasets. We use the Contextual Stochastic Block Model for Homophily (CSBM-H) synthetic graph generation framework (Luan, Hua, M. Xu, et al. 2024) to isolate the effect of homophily (operationalized through edge homophily) and to control for confounders, and study the structural mechanisms that influence GNN performance. For the empirical evaluation, we use 15 benchmark datasets from the Planetoid (Z. Yang, Cohen, and Salakhutdinov 2016), WebKB (Pei et al. 2020), WikipediaNetwork (Rozemberczki, Allen, and Sarkar 2021), WikiCS (Mernyei and Cangea 2022), Actor (Pei et al. 2020) and HeterophilousGraphDataset (Platonov, Kuznedelev, Diskin, et al. 2024) families.

The second question relates to the extent to which the existing label-based (using the nodes' ground truth label assignments), feature-based (using the nodes' attributes) and structure-based (using the graph's structural properties) measures to quantify homophily (h_{label}, h_{feature}, $h_{\text{structure}}$) serve as predictors of the advantage of GNNs. In the case of the label-based metrics, we investigate whether they are able to predict the advantage of GNNs even when their score is computed as an estimation from a subset of visible nodes in the graph, and how the preference for a metric changes when making the estimation with increasing label scarcity. This is done by building label-induced subgraphs from the original datasets, and then using this subgraphs as training set for the models while simultaneously using them to compute an estimator ($\hat{h}_{\text{label}}$) of the original score at full label visibility. The metrics, which come from traditional network analysis, are grouped into the following families:

- Label-based: node homophily, edge homophily, class homophily, adjusted homophily, unbiased homophily, 2-hop homophily, Relative Mutual Information (RMI).
- Feature-based: Generalized Edge Homophily, Feature Assortativity, Local Similarity (cosine, Euclidean).
- Structure-based: Degree Assortativity, Local Clustering Coefficient Assortativity, Neighborhood Similarity.

We evaluate the first research question in the synthetic datasets by comparing the behavior of the gain across different levels of edge homophily and label rates for training and visualizing the gain as a surface defined by the interaction between these two variables. We then statistically quantify the effect of both homophily and label rate and their interaction by means of a Two-Way Repeated Measures ANOVA, plus post-hoc comparisons. In the empirical setting, we evaluate the gain as well as its variability as the proportion of visible labels decreases on each dataset, and observe how a number of factors additional to those identified in the synthetic datasets influence the behavior of graph-aware and graph-agnostic models.

For the second research question, we evaluate the aforementioned metrics based on their rank-based correlations with the gain after model training, their deviation (Normalized Root Mean Square Error) from the full-label reference in the case of the label-based family and, in the empirical dataset pipeline, the preservation of the original homophily regime assignments. This criteria help us determine the robustness of the proxies in the label-scarce setting and derive practical guidelines for model selection under partial label visibility.

All-in-all, through our experiments on synthetic and real-world datasets, we make the following contributions:

1. We provide a deeper understanding on the underlying structural mechanism that determines the potential for GNN performance, i.e. the notion of node distinguishability. We show how this can be easily captured in synthetic datasets, yet in empirical datasets, a number of factors play a role in modulating effective class separability after aggregation, therefore unidimensional descriptors struggle to predict GNN advantage.
2. We give insights on the effect of label scarcity on model performance in semi-supervised node classification both in controlled and real-world environments, and connect this effect with graph homophily both qualitatively and quantitatively.
3. We empirically test the most prominent label-, feature- and structure-based metrics used for gain estimation, and provide guidelines to estimate the potential advantage of graph-aware over graph-agnostic models, which can help with model selection before training.

The rest of this work is organized as follows. Chapter 2 provides the necessary background to set the stage for the empirical analysis, including machine learning on graphs,

graph-aware and graph-agnostic models for semi-supervised node classification, Graph Neural Networks and their mathematical foundation, and a brief description of some popular architectures. Chapter 3 reviews current research in the context of the GNN performance analysis, the homophily and heterophily regimes and label-augmentation methods for performance boosting in the label-scarce setting. Chapter 4 describes the task, the two pipelines, and the experiments conducted to address each of the research questions, including model description, hyperparameters and evaluation methodology. Chapter 5 describes the performed experiments and in parallel discusses the main findings from each pipeline. Chapter 6 compares the results with previous literature, maps them to the research questions, and discusses the limitations of the work. Finally, Chapter 7 summarizes the findings and contributions, and evaluates possible directions for future work.

2. Background

This chapter provides a detailed review of the most important topics required to follow the current work.

2.1. Basic concepts and terminology

2.1.1. Graph fundamentals

A graph is a tuple $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the set of nodes (or vertices), with $N = |\mathcal{V}|$, and $\mathcal{E}$ represents the set of edges (or links), with $E = |\mathcal{E}|$. A graph is also described through its adjacency matrix $\mathbf{A} \in \{0,1\}^{N \times N}$ (assuming G is unweighted), whose entries are defined as follows:

$$A_{ij} = \begin{cases} 1, & e_{ij} = \{v_i, v_j\} \in \mathcal{E}, \\ 0, & \text{otherwise.} \end{cases}$$

$\mathbf{A}$ is symmetric for undirected graphs and it is the basis of several algebraic operations on graphs. In particular, it works as a linear operator for neighbor aggregation, which is the mechanism behind several message-passing layers in GNNs.

For an arbitrary node $v \in \mathcal{V}$, its 1-hop neighborhood, defined as $\mathcal{N}(v) = \{u \in \mathcal{V} \ : \ \{u, v\} \in \mathcal{E}\}$, represents all the nodes immediately connected to v via an edge, such that we can define its degree as $d(v) = |\mathcal{N}(v)|$. Furthermore, the diagonal degree matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$ collects the degrees of all nodes in G:

$$D_{ij} = \begin{cases} d(v_i), & i = j, \\ 0, & \text{otherwise.} \end{cases}$$

We can observe that $D_{ii} = \sum_j A_{ij}$.

Intuitively, one can also define higher order k-hop neighborhoods by considering those nodes that can be reached from v by a path of length at most k[1]: $\mathcal{N}^{(k)}(v) = \{u \in \mathcal{V} \ : \ 1 \leq \text{dist}(u, v) \leq k\}$, where $\text{dist}(u, v)$ denotes the distance between v and u, i.e. the smallest number of edges that must be traversed to get from v to u.

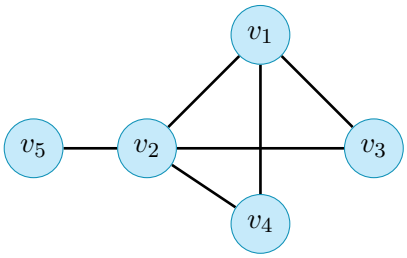
The graph's (unnormalized, combinatorial) Laplacian is a fundamental object in graph theory as it provides a mathematical description of the graph's topology. Given an

[1] Depending on the definition, the k-hop neighborhood might exclude the ego node and/or be restricted to those nodes strictly at distance k.

undirected graph, it is defined as $\mathbf{L} = \mathbf{D} - \mathbf{A}$, such that:

$$L_{ij} = \begin{cases} d(v_i), & i = j, \\ -1, & \{v_i, v_j\} \in \mathcal{E},\ i \neq j, \\ 0, & \text{otherwise.} \end{cases}$$

The eigenvalues and eigenvectors of the Laplacian contain valuable information about a graph's structure. For instance, the multiplicity of the zero eigenvalues $\lambda_1 = 0$ (smallest eigenvalues as $\mathbf{L}$ is Positive-Semidefinite (PSD)) for a graph with c connected components, is c (William L Hamilton 2020). Additionally, the Laplacian's second eigenvalue λ_2, known as the Fiedler value, defines the algebraic connectivity of the graph: if $\lambda_2 = 0$, the graph is disconnected, and if $\lambda_2 > 0$, the graph is connected, with higher values indicating stronger connectivity, meaning the graph is more densely connected and more difficult to partition into communities.



$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} \quad \mathbf{D} = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{L} = \begin{bmatrix} 3 & -1 & -1 & -1 & 0 \\ -1 & 4 & -1 & -1 & -1 \\ -1 & -1 & 2 & 0 & 0 \\ -1 & -1 & 0 & 2 & 0 \\ 0 & -1 & 0 & 0 & 1 \end{bmatrix}$$

Figure 2.1.: Adjacency, Degree and Laplacian matrices of an unweighted, undirected graph.

The graph Laplacian is also used to define the graph convolution operation in the spectral domain: its eigendecomposition is the foundation for constructing filters that operate over the graph signal $\mathbf{x}^{(i)}$, as will be defined in the next section, e.g. by smoothing it when operating as low-pass filters (see Subsection 2.2.4). Figure 2.1 shows a simple graph along with its descriptive matrices.

Given some graph $G = (\mathcal{V}_G, \mathcal{E}_G)$, $H = (\mathcal{V}_H, \mathcal{E}_H)$ is a subgraph of G if $\mathcal{V}_H \subseteq \mathcal{V}_G$ and $\mathcal{E}_H \subseteq \mathcal{E}_G$. Additionally, H is called *induced* if and only if for each edge $e \in \mathcal{E}_G$ whose endpoints are present in $\mathcal{V}_H$, then $e \in \mathcal{E}_H$. Given some set of nodes $\mathcal{U} \subseteq \mathcal{V}_G$, we denote the induced subgraph defined by $\mathcal{U}$ as $G[\mathcal{U}]$, as shown in Figure 2.2.

A graph can be further characterized through the following concepts:

- Directed/undirected: a graph is directed if the existence of an edge $\{v_i, v_j\} \in \mathcal{E}$ doesn't imply the existence of the reverse edge $\{v_j, v_i\}$. Else, it is undirected and thus symmetric, $\mathbf{A} = \mathbf{A}^\top$.

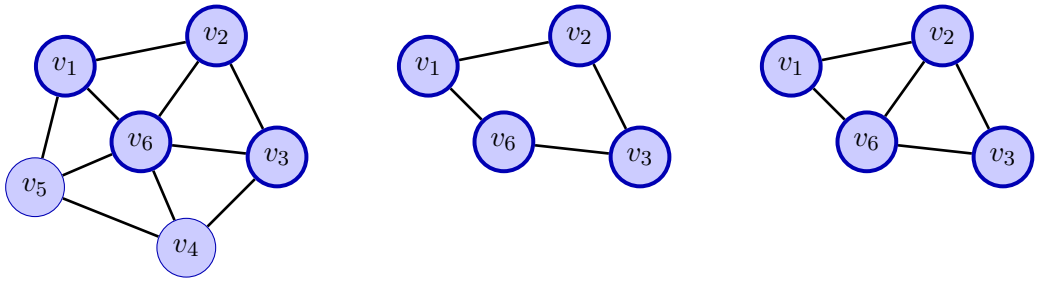


Figure 2.2.: Given a graph $G = (\mathcal{V}_G, \mathcal{E}_G)$ with $\mathcal{V}_G = \{v_1, v_2, v_3, v_4, v_5, v_6\}$ and $\mathcal{E}_G = \{\{v_1, v_2\}, \{v_1, v_5\}, \{v_1, v_6\}, \{v_2, v_3\}, \{v_2, v_6\}, \{v_3, v_4\}, \{v_3, v_6\}, \{v_4, v_5\}, \{v_4, v_6\}, \{v_5, v_6\}\}$, $H = (\mathcal{V}_H, \mathcal{E}_H)$ is a subgraph of G since $\mathcal{V}_H \subseteq \mathcal{V}_G$ and $\mathcal{E}_H \subseteq \mathcal{E}_G$, and $G[\mathcal{U}]$ is the corresponding induced subgraph, where $\mathcal{U} = \mathcal{V}_H = \{v_1, v_2, v_3, v_6\}$.

- Weighted/unweighted: a graph is weighted if its edges can take any value in the real domain, $A_{ij} \in \mathbb{R}$. It is unweighted if $A_{ij} \in \{0, 1\}$.

- Connected/disconnected: a graph is connected if each node can be reached by every other node, i.e. there exists a path between every $v_i, v_j \in \mathcal{V}$, else it is disconnected.

- Weakly/strongly connected: for a directed graph, it is strongly connected if each pair of nodes is connected in both directions. If such paths only exist when disregarding directionality, the graph is weakly connected.

In this work, we assume graphs are undirected, unweighted, and have no self-loops, i.e. $\forall i : A_{ii} = 0$, unless otherwise stated.

2.1.2. Attributed graphs

In attributed graphs, each node v is associated with a feature vector $\mathbf{x}_v \in \mathbb{R}^d$, a numerical representation of its attributes, and all feature vectors are collected as the rows of the feature matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, where $N = |\mathcal{V}|$ and d is the feature dimensionality. In graph signal processing, each column $\mathbf{x}^{(i)} \in \mathbb{R}^N$ of the feature matrix represents the signal vector of feature i:

$$\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \cdots & \mathbf{x}^{(d)} \\ | & | & & | \end{bmatrix}$$

Additionally, each node v is assigned a label $y_v \in \mathcal{Y}$, where $\mathcal{Y}$ denotes the set of labels, and $C = |\mathcal{Y}|$ is the number of classes in the graph. The one-hot encodings of the nodes' ground-truth class assignments are collected in the label encoding matrix $\mathbf{Y} \in \{0, 1\}^{N \times C}$ (Luan, Hua, Q. Lu, et al. 2023).

2.2. Machine learning on graphs

The goal of graph machine learning is to learn a function that maps from graph-structured input data (e.g. nodes, edges, graphs or subgraphs) to predictions in the output space:

- Graph-level tasks: $f : G \to \mathcal{Y}$.
- Node-level tasks: $f : (G, v) \to \mathcal{Y}$, for $v \in \mathcal{V}$.
- Edge-level tasks: $f : (G, u, v) \to \mathcal{Y}$, for $\{u, v\} \in \mathcal{E}$.

In each case, we define $\mathcal{Y} = \mathbb{R}$ for regression and $\mathcal{Y} = \mathcal{C}$, where $\mathcal{C}$ represents the set of classes, for classification.

2.2.1. Tasks

Each task is characterized as follows:

- Graph-level tasks make predictions at the level of entire graphs by generating an embedding that summarizes the information inside the graph and is used predict a value (discrete or continuous) based on this representation (K. Xu et al. 2019). These representations can be used, for instance, to predict some property of molecules, which are naturally represented by graphs, where nodes represent atoms and edges represent the bonds between them (Gilmer et al. 2017), or for network popularity prediction in social platforms (Q. Cao et al. 2019).

- Node-level tasks aim to predict a value for each node in a graph, be it a continuous number, i.e. node regression, or a discrete value, i.e. node classification. This task is widespread and numerous benchmarks have been developed to test the strength of GNNs in all sorts of networks, e.g. social networks (Rozemberczki and Sarkar 2021), citation networks (Z. Yang, Cohen, and Salakhutdinov 2016), and webpage/hyperlink networks (Pei et al. 2020; Mernyei and Cangea 2022; Rozemberczki, Allen, and Sarkar 2021). The current work focuses on semi-supervised node-level classification, in which, given a set of –potentially scarce– labeled nodes $\mathcal{V}_L \subset \mathcal{V}$, the goal is to predict the classes of the unlabeled nodes $\mathcal{V}_U = \mathcal{V} \backslash \mathcal{V}_L$ using any information available in the graph.

- The most commonplace edge-level task is link prediction for recommender systems, where given some bipartite user-item graph, the goal is to predict whether a new edge between a user-type node v_u and an item-type node v_i might exist. Models for collaborative filtering like LightGCN (He et al. 2020), which is a simplified version of the canonical GCN, were specifically designed for this kind of task. Another prominent task in this scope is edge classification, e.g. for anomaly detection in bank or e-commerce transactions (B. Cao et al. n.d.).

- Beyond these tasks, graph generation/reconstruction is concerned with producing new samples of graphs with specific structural properties. Classical probabilistic models rely on heuristics to create nodes and edges and include Erdős–Rényi random graph models (Gilbert 1959), Stochastich Block Models (SBM) (Holland, Laskey, and Leinhardt 1983) and Preferential Attachment models (Barabasi and Albert 1999). In recent years, deep learning generative models have gained particular attention thanks to their flexibility and ability to model complex patterns without relying on predefined rules. Prominent approaches include autoregressive models (You, Ying, Ren, et al. 2018), Variational Autoencoder-based models (Grover, Zweig, and Ermon 2019) and diffusion-based models (Vignac et al. 2023).

2.2.2. Node embeddings

As previously discussed, graph machine learning requires learning a meaningful node representation for any downstream prediction task. This representation is a multidimensional vector $\mathbf{z}_v \in \mathbb{R}^d$, with $d \ll N$, and the task of mapping from node v to such embedding is called graph representation learning. William L. Hamilton, Ying, and Leskovec 2018b define the task by means of an *encoder-decoder* framework, where the goal of the encoder is to capture the information in both the structure of the graph ($\mathbf{A}$) and the initial node features ($\mathbf{X}$) to generate d-dimensional embeddings:

$$f : \mathcal{V} \to \mathbb{R}^d$$

Intuitively, nodes that are similar in the original graph should be similar in the latent space by means of some similarity measure, for instance, the inner product: $\text{sim}(u, v) \approx \mathbf{z}_u^\top \mathbf{z}_v$. The decoder is a function mapping back from the embedding space to such similarity measure. It takes a pair of embeddings $(\mathbf{z}_u, \mathbf{z}_v)$ and outputs a scalar representing the estimation of their similarity in the original graph (William L Hamilton 2020):

$$f : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^+$$

A model is therefore optimized to minimize the difference between the original similarity of the nodes in the graph space and the reconstruction computed by the decoder via e.g. stochastic gradient descent.

2.2.3. Node embedding models

Early approaches to generate node embeddings, commonly referred to as shallow encoders, relied on a lookup table $\mathbf{Z} \in \mathbb{R}^{d \times N}$ containing the embeddings of each node as its columns: $\mathbf{z}_v = \mathbf{Z} \cdot \mathbf{v}$, where $\mathbf{v}$ is a node-wise indicator vector (William L. Hamilton, Ying, and Leskovec 2018b). Random walk-based methods and matrix factorization belong to this category, and operate by mapping nodes individually to a latent representation, such that each node gets an independent parameter vector.

While shallow encoders learn node-specific parameter vectors, deep learning-based encoders rely on shared learnable parameters that are common to all nodes. A node's

representation can either depend solely on its own features $\mathbf{z}_v = f_\theta(\mathbf{x}_v)$, as is the case in Multi-Layer Perceptrons, or incorporate information from its neighborhood via message passing $\mathbf{z}_v = f_\theta(\mathbf{x}_v, \{\mathbf{x}_u : u \in \mathcal{N}(v)\})$. This notion ultimately gave rise to Graph Neural Networks. These architectures offer several improvements: they are able to incorporate node features/attributes during learning, they are generally more scalable to large networks due to their parameter sharing nature, plus, certain architectures can generalize beyond nodes seen during training, i.e. the learned weight matrices can be used to generate embeddings of previously unseen nodes via inductive learning (William L. Hamilton, Ying, and Leskovec 2018a).

Table 2.1 summarizes the taxonomy of the most prominent shallow and neural encoder families.

Table 2.1.: Families of models for node embedding.

Family	Uses Features	Uses Graph Structure	Shared Parameters
Matrix factorization	No	Yes	No
Random walk	No	Yes	No
Neural (graph-agnostic)	Yes	No	Yes
Neural (graph-aware)	Yes	Yes	Yes

Shallow encoders

Encoders based on matrix factorization commonly define the decoder-optimized node similarity measure from the graph's connectivity. As seen in Section 2.1, this information is found in the adjacency matrix $\mathbf{A}$, so the objective is defined as the factorization $\mathbf{z}_u^\top \mathbf{z}_v \approx A_{uv}$, for a pairwise relation, or $\mathbf{Z}^\top \mathbf{Z} \approx \mathbf{A}$ for all pairs. Ahmed et al. 2013 optimize this objective by minimizing squared –reconstruction– error with L_2 regularization. S. Cao, W. Lu, and Q. Xu 2015 (GraRep) further extend the paradigm by not only considering immediate neighbors but by capturing k-step relationships, therefore incorporating global structural information into each node's representation, while Ou et al. 2016 (HOPE) incorporate the ability to model asymmetric relations (i.e. directed edges) between nodes while preserving the power of capturing higher-order dependencies.

Random walk methods establish that a destination node u is similar to some source node v if it frequently appears in random walks[2] starting from v. Specifically, Perozzi, Al-Rfou, and Skiena 2014 (DeepWalk) encode similarity by defining a distribution $p(u \mid v)$, which quantifies the probability of observing node u when generating random walks of length t starting from source node v. Intuitively, a walk must not sample nodes uniformly, but can incorporate information about the previously visited nodes to bias the decision of which node should be visited next. Grover and Leskovec 2016 (node2vec) build upon this idea and introduce two hyperparameters p, q, which control the probability of returning to the previously visited node and of exploring nodes further away from the current

[2]A walk is a more general concept than a path in that it allows for nodes to be visited more than once.

node's immediate neighborhood, respectively, effectively balancing between breadth-first and depth-first strategies that enable learning more informative embeddings.

Neural encoders

Multi-Layer Perceptrons (Rumelhart, Hintont, and Williams 1986) naturally accommodate to the task of generating latent representations of the feature vectors in relational datasets like graphs and they mainly differ from message-passing neural networks in that the generated embeddings do not consider graph structure, i.e. a node's representation does not depend on the features of its neighbors. Therefore, MLPs are considered the graph-agnostic equivalent of GNNs and they are widely used as a baseline for modern message-passing architectures.

Although the canonical MLP is purely feature-driven, variations that do exploit graph structure exist, as is the case in Graph-MLP (Hu et al. 2021), which enforces a connectivity bias through a neighbor-aware loss (Neighborhood Contrastive Loss), overcoming the need of message passing during feature extraction. ES-MLP (Kohn, Hoffmann, and Scherp 2024) builds upon this architecture and defines an edge-splitting layer that separates task-relevant from task irrelevant embeddings before using the same contrastive loss, while MLP+C&S (Huang et al. 2020) uses a graph-agnostic model to make initial predictions and then integrate label structure by enforcing a positive correlation between the errors of connected nodes (correcting step), and then enforces smoothness on the predicted label distributions based on the homophily assumption (smoothing step).

Finally, Graph Neural Networks[3] is the term used to denote the family of neural-based encoders that simultaneously leverage node features and graph topology to learn a shared parametric function mapping from the original features to an embedding that jointly incorporates information from the ego node and from its –potentially multi-hop– neighborhood. These will be presented in detail in the next section.

2.2.4. Graph Neural Networks

The core mechanism behind GNNs is message passing. One GNN layer consists of a composition of functions that operate on the embeddings of each node and its neighbors'. Essentially, for each $v \in \mathcal{V}$, a GNN layer updates its embedding as:

$$\mathbf{h}_v^{(\ell)} = \text{UPDATE}\left(\mathbf{h}_v^{(\ell-1)}, \text{AGGREGATE}_{u\in\mathcal{N}(v)}\left(\left\{\text{MESSAGE}(\mathbf{h}_v^{(\ell-1)}, \mathbf{h}_u^{(\ell-1)})\right\}\right)\right) \quad (2.1)$$

This breaks downs to the following components (Bronstein et al. 2021):

- MESSAGE($\cdot$): a learnable transformation that computes a message from each neighbor u to v based on their corresponding previous layer embeddings $\mathbf{h}_v^{(\ell-1)}$, $\mathbf{h}_u^{(\ell-1)}$.

[3]Strictly speaking, we refer in this work to Message-Passing Neural Networks (MPNNs) which specifically use the message passing mechanism and do not include e.g. formulations defined purely in the spectral domain, which are also GNNs. Nevertheless, since the terms are often used interchangeably in practice, we use GNNs when referring to message passing models.

- AGGREGATE($\cdot$): a permutation-invariant (w.r.t a node's neighbors) operation that summarizes the messages computed in the previous step, for example, sum($\cdot$), mean($\cdot$), max($\cdot$).
- UPDATE($\cdot$): a function that produces the node's updated representation from its own previous embedding and/or its neighbor's aggregated messages, possibly followed by a non-linearity, e.g. ReLU($\cdot$).

Any GNN layer is an instantiation of this general architecture. For the first layer, the input is $\mathbf{h}_v^{(0)} = \mathbf{x}_v$, i.e. the node's original feature vector. By stacking multiple layers, information effectively propagates across increasingly distant neighborhoods such that, for an L-layer GNN, the receptive field of each node is defined by all nodes within L hops, including the ego node. For node classification, the last layer's output is a vector of class logits $\mathbf{z}_v = \mathbf{h}_v^{(L)}$, to which softmax is applied to get a vector of predicted class probabilities: $\hat{\mathbf{y}}_v = \text{softmax}(\mathbf{z}_v)$. Hence, a multi-layer GNN can be visualized as:

$$\mathbf{x}_v \xrightarrow{\text{GNN}^{(1)}} \mathbf{h}_v^{(1)} \xrightarrow{\text{GNN}^{(2)}} \cdots \xrightarrow{\text{GNN}^{(L)}} \mathbf{h}_v^{(L)} \xrightarrow{\text{softmax}} \hat{\mathbf{y}}_v$$

And in matrix notation, by stacking the node feature vectors, intermediate representations, and vectors of class distributions, respectively:

$$\mathbf{X} \xrightarrow{\text{GNN}^{(1)}} \mathbf{H}^{(1)} \xrightarrow{\text{GNN}^{(2)}} \cdots \xrightarrow{\text{GNN}^{(L)}} \mathbf{H}^{(L)} \xrightarrow{\text{softmax}} \hat{\mathbf{Y}}$$

where $\mathbf{X} \in \mathbb{R}^{N \times d_0}$, $\mathbf{H}^{(\ell)} \in \mathbb{R}^{N \times d_\ell}$, $\mathbf{H}^{(L)} = \mathbf{Z} \in \mathbb{R}^{N \times C}$, and $\hat{\mathbf{Y}} \in [0, 1]^{N \times C}$.

Spectral analysis

In graph spectral theory, GNNs can be interpreted as graph filters, whose function is to transform a graph signal in some way, an operation that is simpler in the frequency domain for irregular topologies like graphs. Formally, Sandryhaila and Moura 2013 define a filter $\mathcal{H} \in \mathbb{C}^{N \times N}$ as a function that operates on a graph shift operator $\mathbf{S} \in \mathbb{R}^{N \times N}$ (for instance, the adjacency matrix, the graph Laplacian or some normalized version thereof), commonly realized as a polynomial of the shift operator:

$$\mathcal{H}(\mathbf{S}) = h_0(\mathbf{I}) + h_1(\mathbf{S}) + h_2(\mathbf{S}^2) + \cdots + h_L(\mathbf{S}^L)$$

If $\mathbf{S}$ is a real, symmetric matrix, then its eigendecomposition exists. If we define $\mathbf{S} = \mathbf{L}$, its eigendecomposition is:

$$\mathbf{L} = \mathrm{U}\Lambda\mathrm{U}^\top$$

where the columns of $\mathrm{U} \in \mathbb{R}^{N \times N}$ are the N eigenvectors, which define the *graph Fourier basis*, and $\Lambda \in \mathbb{R}^{N \times N}$ contains the N corresponding eigenvalues, which define the *graph frequencies*:

$$\mathrm{U} = \begin{bmatrix} | & | & & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_n \\ | & | & & | \end{bmatrix} \qquad \Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

In the Fourier domain, the filter is defined as:

$$g(\mathbf{L}) = \mathrm{U} g(\Lambda) \mathrm{U}^\top$$

Therefore, we have $g(\Lambda) = \mathrm{diag}(g(\lambda_1), \ldots, g(\lambda_n))$, i.e. the filter operates element-wise on the elements of $\mathbf{L}$, and defines how much each of the graph's frequencies are scaled by the filter. Low-pass filters commonly take the form of a smoothing operator $g(\lambda) = \frac{1}{1+\alpha\lambda}$, where $\alpha > 0$, defines the smoothing strength, i.e. how much higher frequencies are attenuated. More generally, it can defined by a polynomial parametrized by some vector of –learnable– coefficients $\theta \in \mathbb{R}^K$ (Defferrard, Bresson, and Vandergheynst 2017):

$$g_\theta(\Lambda) = \sum_{k=0}^{K} \theta_k \Lambda^k \tag{2.2}$$

Given a graph signal $\mathbf{x} \in \mathbb{R}^N$ in the node domain, which can be decomposed as a linear combination of the basis' eigenvectors (graph frequencies), the filtered signal $\mathbf{y} \in \mathbb{R}^N$ is obtained by applying the filter to the signal (a.k.a. spectral graph convolution $g_\theta \star \mathbf{x}$):

$$\mathbf{y} = g_\theta(\mathbf{L})\mathbf{x} = \mathrm{U} g_\theta(\Lambda) \mathrm{U}^\top \mathbf{x} \tag{2.3}$$

So, signal filtering can be understood as transforming a signal to the spectral domain, $\tilde{\mathbf{x}} = \mathrm{U}^\top \mathbf{x}$, applying a filter by scaling the graph frequencies independently, $\tilde{\mathbf{y}} = g_\theta(\Lambda)\tilde{\mathbf{x}}$, and applying the inverse transform to the resulting filtered signal in the Fourier domain back to the node domain $\mathbf{y} = \mathrm{U}\tilde{\mathbf{y}}$.

The Graph Convolutional Network (GCN)

The main problem of the aforementioned procedure for graph filtering is that the Laplacian eigendecomposition's complexity is $\mathcal{O}(N^3)$, which is intractable for large graphs. Therefore, Hammond, Vandergheynst, and Gribonval 2009 propose to approximate the filter $g_\theta(\Lambda)$ using a K-order Chebyshev polynomial, recursively defined as $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$, with $T_0(x) = 1$ and $T_1(x) = x$, such that, from Eq. (2.2), we get:

$$g_{\theta'}(\Lambda) \approx \sum_{k=0}^{K} \theta'_k T_k(\tilde{\Lambda})$$

where θ' is a vector of Chebyshev coefficients, and $\tilde{\Lambda}$ is the diagonal eigenvalue matrix of the rescaled Laplacian $\tilde{\mathbf{L}} = \frac{2}{\lambda_n(\mathbf{L})}\mathbf{L} - \mathbf{I}$, where $\lambda_n(\mathbf{L})$ is the largest eigenvalue and is approximated as $\lambda_n \approx 2$, used to map the individual eigenvalues to the $[-1, 1]$ interval defined by the Chebyshev polynomials. $\mathbf{L}$ is the symmetric, degree-normalized Laplacian $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$, used to address degree imbalances, such that $\tilde{\mathbf{L}} = -\mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$. Since T_k is a polynomial in $\tilde{\Lambda}$ and $(\mathrm{U}\Lambda\mathrm{U}^\top)^k = \mathrm{U}\Lambda^k\mathrm{U}^\top$, from Eq. (2.3) we have:

$$g_{\theta'}(\mathbf{L})\mathbf{x} \approx \mathrm{U} \left[\sum_{k=0}^{K} \theta'_k T_k(\tilde{\Lambda}) \right] \mathrm{U}^\top \mathbf{x} \approx \sum_{k=0}^{K} \theta'_k T_k(\tilde{\mathbf{L}})\mathbf{x}$$

which does not require an eigenvector decomposition. By setting $K = 1$, Kipf and Welling 2017 define a first-order approximation of the filter, with the signal convolution in the node domain given as:

$$g_{\theta'}(\mathbf{L})\mathbf{x} \approx \theta'_0 T_0(\tilde{\mathbf{L}})\mathbf{x} + \theta'_1 T_1(\tilde{\mathbf{L}})\mathbf{x}$$

And by evaluating the base terms of the Chebyshev polynomial:

$$g_{\theta'}(\mathbf{L})\mathbf{x} \approx \theta'_0\mathbf{x} + \theta'_1\tilde{\mathbf{L}}\mathbf{x}$$

So, setting $K = 1$ effectively incorporates information from the ego node and its immediate neighbors (1^{st} order function of the Laplacian). By fixing $\theta = \theta'_0 = -\theta'_1$, the convolution simplifies to:

$$g(\mathbf{L})\mathbf{x} \approx \theta\left(\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}\right)\mathbf{x}$$

Finally, to prevent numerical instabilities when stacking multiple layers, the authors establish the approximate equivalence: $\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2} \approx \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ (*renormalization trick*), where $\tilde{\mathbf{A}} = (\mathbf{A} + \mathbf{I})$ and $\tilde{\mathbf{D}} = (\mathbf{D} + \mathbf{I})$ introduce self-loops in the computation. Applied to the entire feature matrix $\mathbf{X} \in \mathbb{R}^{N\times d}$, and using a matrix of learnable coefficients $\mathbf{W} \in \mathbb{R}^{d_{in}\times d_{out}}$ (where commonly $d = d_{in} = d_{out}$) instead of a scalar θ:

$$g(\mathbf{L})\mathbf{X} \approx \left(\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}\right)\mathbf{X}\mathbf{W}$$

So, the general update rule for one GCN layer given an input $\mathbf{H}^{(\ell)}$ with non-linearity σ is:

$$\boxed{\mathbf{H}^{(\ell+1)} = \sigma\left(\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}\mathbf{H}^{(\ell)}\mathbf{W}^{(\ell)}\right)}$$

Equivalently, for a single node v:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\sum_{u\in\mathcal{N}(v)\cup\{v\}} \frac{1}{\sqrt{\tilde{d}_v\tilde{d}_u}}\mathbf{h}_u^{(\ell)}\mathbf{W}^{(\ell)}\right) \tag{2.4}$$

where $\tilde{d}_v = |\mathcal{N}(v) \cup \{v\}|$ for any v. It is therefore clear that the update rule of a node's embedding is a function of its own previous embedding as well as its immediate neighbors' (degree-normalized and with self-loops), i.e. it defines how messages from 1-hop neighbors are linearly transformed and aggregated, followed by a non-linear function, which matches the general definition in Eq. (2.1).

Since the layer-specific $\mathbf{W}^{(\ell)}$ is learned jointly and applied to every node in the graph, it is reminiscent of how CNNs operate by defining convolutional kernels that are shared at all spatial locations in a regular lattice (like pixels in an image), so they can be considered a special case of GNNs. The weight sharing nature of these two architectures make them highly parameter-efficient in comparison to fully connected layers.

Finally, by stacking multiple layers, it is possible to construct increasingly complex architectures, where the parameters of the weight matrices on each layer are updated

via gradient descent. For the semi-supervised node classification setting, the learning objective is given by the cross-entropy loss over the labeled nodes $\mathcal{V}_L \subseteq \mathcal{V}$ given the label encoding matrix $\mathbf{Y} \in \{0,1\}^{N \times C}$ and the matrix of predicted class distributions $\hat{\mathbf{Y}} \in [0,1]^{N \times C}$:

$$\mathcal{L} = -\sum_{v \in \mathcal{V}_L} \sum_{c=1}^{C} Y_{vc} \log \hat{Y}_{vc} \tag{2.5}$$

In practice, the mean cross-entropy loss is typically preferred, which results from normalizing the loss by the size of the labeled set, i.e. $\mathcal{L}_{\text{norm}} = \frac{1}{|\mathcal{V}_L|}\mathcal{L}$.

GNN-MLP correspondence

The use of Multi-Layer Perceptrons (MLPs) for all sorts of predictive tasks is widespread, however, they naturally operate on independent feature-label tuples $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, i.e. the rows of the feature vector $\mathbf{X} \in \mathbb{R}^{N \times d}$ are treated as independent observations and a datapoint's representation does not influence that of other datapoints. In contrast, GNNs break the *i.i.d.* assumption and allow us to model relational data, so predictions are made based not only on the feature vector associated to each datapoint/node, but also on the topology of the graph, encoded in –some variant of– the adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$. Hence, for C-class node classification, the output logits $\mathbf{Z} \in \mathbb{R}^{N \times C}$ of each model are:

$$\begin{aligned} \text{MLP}: &\quad \mathbf{Z} = f_\theta(\mathbf{X}) \\ \text{GNN}: &\quad \mathbf{Z} = f_\theta(\mathbf{X}, \mathbf{A}) \end{aligned}$$

Luan, Hua, M. Xu, et al. 2024 show how graph-aware models are naturally coupled to a graph-agnostic architecture, both for 1- and 2-layer architectures (up to layer dimensions). Specifically, by using the degree-normalized adjacency matrix with self loops $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$, we build the 2-layer GCN, and by disregarding neighbor aggregation, we get a 2-layer MLP, and analogously for a 1-layer MLP[4]. Table 2.2 illustrates this correspondence.

Table 2.2.: GNN-MLP correspondence.

Encoder	1-Layer	2-Layer
Graph-agnostic	$\mathbf{Z} = \mathbf{X}\mathbf{W}$	$\mathbf{Z} = \left(\text{ReLU}\left(\mathbf{X}\mathbf{W}^{(0)}\right)\right)\mathbf{W}^{(1)}$
Graph-aware	$\mathbf{Z} = \hat{\mathbf{A}}\mathbf{X}\mathbf{W}$	$\mathbf{Z} = \hat{\mathbf{A}}\left(\text{ReLU}\left(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}^{(0)}\right)\right)\mathbf{W}^{(1)}$

So, the adjacency matrix defines how nodes communicate, i.e. how to incorporate information from a node's neighbors. It is also noteworthy that this is the key component that differentiates graph-aware from graph-agnostic architectures, so intuitively, how informative a node's neighborhood is of its own label is the central factor determining

[4]The Simple Graph Convolution (SGC, F. Wu et al. 2019) is a similar implementation which performs K aggregation steps and only then applies a single weight matrix and a task-specific nonlinearity.

whether message passing will hurt or improve predictions w.r.t. the corresponding graph-agnostic baselines. Finally, the degree-normalized aggregation operator $\hat{\mathbf{A}}$ is specific to GCN, but several other realizations have been proposed, as will be seen in the following section.

2.2.5. Modern GNN architectures

Bronstein et al. 2021 provide a categorization of GNN layers into 3 classes ("flavors") based on the mechanism used to update a node's representation $\mathbf{h}_v$ given its neighbor's embeddings $\{\mathbf{h}_u : u \in \mathcal{N}(v)\}$:

- *Convolutional*: Features from neighbors are weighted by a fixed coefficient α_{uv}, i.e. this factor is not learned and does not depend on the aggregated features.
- *Attentional*: Messages from neighbors are weighted through a learnable scalar $\alpha_{uv} = a(\mathbf{h}_u, \mathbf{h}_v)$, where a is some attention mechanism, according to their relative importance to the receiving node's representation.
- *Message-Passing*: A message depends jointly on the representations of the neighbors and of the ego node $\alpha_{uv} = f(\mathbf{h}_u, \mathbf{h}_v)$.

So, one can understand the message-passing flavor to be the most general term and the other flavors being as specific realizations thereof. GCN is best categorized as convolutional, as the aggregated features depend on a fixed, graph-dependent normalizing factor. From Eq. (2.4):

$$\alpha_{uv} = \frac{1}{\sqrt{\tilde{d}_v \tilde{d}_u}}$$

In this section we explore some other representative architectures of the convolutional (GraphSAGE, GIN) and attentional (GAT) classes.

Graph Attention Network (GAT)

First introduced by Veličković et al. 2018, it builds upon the concept of localized filters used by the GCN. A GAT layer operates fully on the node domain and uses a *masked self-attention* mechanism that lets nodes selectively attend to neighbors' features by learning weights denoting the importance of each neighbor's embeddings for the node in question. As a result, the aggregation coefficients are not fixed and graph-dependent, but rather learned through a parametric function, so a GAT layer can operate in the inductive setting by using the learned functions to obtain representations of nodes originally not present in the graph.

As mentioned, the attention mechanism is usually realized through a learnable function, e.g. a single layer neural network, such that the mechanism's linear transformations can be learned jointly with the message-passing layers. For a pair of connected nodes $\{v, u\} \in \mathcal{E}$ the unnormalized attention scores result from concatenating their respective –linearly transformed– features and passing them through the linear layer (plus a

LeakyReLU activation):

$$s_{vu} = \text{LeakyReLU}\left(\text{Linear}\left(\mathbf{W}^{(\ell)}\mathbf{h}_v^{(\ell-1)} || \mathbf{W}^{(\ell)}\mathbf{h}_u^{(\ell-1)}\right)\right)$$

Then, to get normalized attention coefficients across the node's neighborhood, the attention scores of each neighbor are fed to a softmax function:

$$\alpha_{vu} = \text{softmax}_u(s_{vu})$$

Importantly, the computation of the attention scores for each pair of nodes is parallelizable, making the process computationally efficient. Furthermore, for training stability and learning richer representations, multiple parallel attention heads are used to generate head-specific attention scores α_{vu}^h and then aggregated into a single message, which is reminiscent of the multi-head attention mechanism used in language transformers (Vaswani et al. 2023).

Therefore, the updating function can be interpreted as a weighted average of the messages from neighboring nodes, using their learned relative importance as weighting factors:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(\ell)} \mathbf{h}_u^{(\ell)} \mathbf{W}^{(\ell)}\right)$$

In the multi-head setting, each head's output is concatenated or averaged to get the final node representation. As the layer's weights and attention mechanism are common to all neighbors, the architecture is highly scalable and independent of the size of the graph, as was the case with GCN layers.

Graph Sample and Aggregate (GraphSAGE)

A GCN layer explicitly uses $\hat{\mathbf{A}}$ both during the forward pass and backpropagation, meaning that a trained model cannot be used to predict labels for nodes not seen during training, as the learning process was optimized for a specific graph topology. To address this limitation, William L. Hamilton, Ying, and Leskovec 2018a designed GraphSAGE specifically for inductive learning: it learns a neighbor aggregation function instead of node-specific embeddings, it is therefore capable of doing inference on previously unseen data. This generalization power is a desired capability in real world graphs, which evolve dynamically, such that re-training and deploying a model repeatedly is not viable.

Concretely, in contrast to the approach of GCN of using a fixed weighted average over neighbor embeddings, GraphSAGE introduces a family of aggregation functions (AGG): –weighted– average functions, RNN-based sequential models (e.g. LSTM$(\cdot)$), or some pooling operator like a dimension-wise maximum, which are used to compute a summary of the embeddings in a node's neighborhood. This aggregated representation is layer-specific, so for each layer the neighborhood-dependent message is:

$$\mathbf{h}_{\mathcal{N}(v)}^{(\ell)} = \text{AGG}_{u \in \mathcal{N}(v)}^{(\ell)}\left(\left\{\mathbf{h}_u^{(\ell-1)}\right\}\right)$$

Importantly, in this step, it is possible to sample a fixed-size subset of the neighbors to compute a message, which enables mini-batch training and is more computationally efficient than using the entire neighborhood across all nodes. Once a message is generated, it gets concatenated with the corresponding node's previous embedding to update its representation through a learnable function with non-linearity:

$$\mathbf{h}_v^{(\ell)} = \sigma\left(\mathbf{W}^{(\ell)}\left[\mathbf{h}_v^{(\ell-1)} \| \mathbf{h}_{\mathcal{N}(v)}^{(\ell)}\right]\right)$$

Therefore, the GCN aggregation function can be interpreted a variant of this architecture that uses a weighted mean (using the corresponding node degrees as weighting factor) as aggregation operator, and aggregates the ego and neighbor embeddings to generate the corresponding message instead of concatenating. Finally, an L_2-normalization is applied to the embedding to ensure that the vectors across different nodes have similar scales and preventing exploding representations:

$$\mathbf{h}_v^{(\ell)} = \frac{\mathbf{h}_v^{(\ell)}}{\left\|\mathbf{h}_v^{(\ell)}\right\|_2}$$

For parameter learning, two alternatives are provided: an unsupervised loss which relies on the assumption that nearby (i.e. co-appearing in random walks of fixed length) nodes should have similar output representations, while dissimilar nodes should have different representations, which is quantified via a logistic loss function aiming to maximize the inner product between nearby nodes (positive samples) while minimizing the inner product of randomly sampled pairs of nodes. Alternatively, a supervised, task-specific loss can be used (e.g. mean squared error for regression).

Graph Isomorphism Network (GIN)

The Graph Isomorphism Network (K. Xu et al. 2019) is motivated from the observation that the aggregation functions used in commonplace architectures like GCN (mean-based aggregation) and GraphSAGE (e.g. element-wise maximum pooling) are not injective, i.e. functions where no two inputs map to the same output. Specifically, in graph representation learning, neighborhood representations are multisets (sets where the multiplicity of each element can be higher than 1), as a result, there are specific neighborhood configurations where these aggregation functions will map different multisets of embeddings to the same output embedding. Therefore, the expressive power of those GNN layers is limited.

K. Xu et al. 2019 showed that GNNs can have at most the expressive power of the 1-dimensional Weisfeiler-Leman (1-WL) test (a.k.a. color refinement algorithm), which checks whether two graphs are non-isomorphic. In each step of the algorithm, an injective mapping of a node v to a new color c_v is performed based on the multiset of its current color and its neighbors'. Since this mapping is realized through a hash function, two different node configurations will invariably be mapped to distinct colorings, so different neighborhoods remain distinguishable at each step (Arvind et al. 2015):

$$c_v^{(i+1)} = \text{Hash}\left(c_v^{(i)}, \{\{c_u^{(i)} : u \in \mathcal{N}(v)\}\}\right) \tag{2.6}$$

where $\{\{\ldots\}\}$ represents a multiset.

A GIN layer is designed to match the expressive power of the 1-WL test by using an aggregation function that is also injective. Thus, the definition of a GIN relies on the fact that an injective function g operating on bounded-size multisets $\mathcal{S}$ can be expressed as a non-linear function Φ applied to the sum of the outputs of another non-linear function f applied element-wise to the members s of $\mathcal{S}$:

$$g(S) = \Phi\left(\sum_{s \in \mathcal{S}} f(s)\right)$$

In principle, given the Universal Approximation Theorem (Hornik, Stinchcombe, and White 1990), both Φ and f can be realized through a 1-layer MLP. In fact, a GIN layer is operationalized this way, where both functions are instantiated through a feed-forward neural network, explicitly separating the ego node representation ("coloring") and its neighborhood representations, as in Eq.(2.6), and modeling the composition $\Phi \circ f$ through a single MLP:

$$\mathbf{h}_v^{(\ell)} = \mathrm{MLP}^{(\ell)}\left(\left(1 + \epsilon^{(\ell)}\right)\mathbf{h}_v^{(\ell-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(\ell-1)}\right)$$

where the ego node weighting factor $\epsilon^{(\ell)}$ is a learnable scalar. Therefore, the aggregation function is an element-wise sum operator, which is naturally more expressive than mean- and maximum-based aggregation functions. As discussed, this layer architecture matches the expressive power of the 1-WL test, so it is able to uniquely represent neighborhood distributions, such that information loss is minimized during message passing.

2.3. Semi-supervised learning

In broad terms, semi-supervised learning (SSL) lies between supervised learning, where a dataset consists entirely of labeled samples $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ which are commonly separated into train/validation/test splits for training, model selection and evaluation, and unsupervised learning, where all samples are unlabeled $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$ and the structure of the data is learned from patterns present in the features. Formally, given a set of labeled tuples $\mathcal{D}_L = \{(\mathbf{x}_i, y_i)\}_{i=1}^L$ and a disjoint set of unlabeled instances $\mathcal{D}_U = \{\mathbf{x}_j\}_{j=L+1}^N$, where usually $|\mathcal{D}_L| \ll |\mathcal{D}_U|$, the goal of SSL is to leverage the information in both sets to improve the generalization power of a model (X. Zhu and Goldberg 2009).

The main motivation behind this approach to learning is the fact that in real-life datasets, labeling might be expensive in terms of time consumption, human effort, and/or required domain knowledge. Therefore, SSL aims to maximize the utility of the available information to make better predictions on the unlabeled data. For instance, a case study by Blum and Mitchell 1998 analyses university websites and uses a reduced subset of labeled webpages (which were manually annotated into several categories) to co-train two classifiers on different features of the data, and then jointly determine whether a

specific website corresponds to the category "course homepage" by selecting the most confident predictions of each independent classifier. Evidently, the WWW is a domain where new instances (webpages) are continuously added, so manual annotation is not feasible, yet through co-training, the model is able to increase the utility of the unlabeled data and make more confident predictions.

Abundance of unlabeled data and new unlabeled instances constantly appearing in a dataset are phenomena present in several other domains, e.g. face recognition in social media and spam detection, and techniques that jointly use the information in $\mathcal{D}_L$ and in $\mathcal{D}_U$ have proven to outperform learning merely through supervised approaches. In this section we provide an overview of some of the most prominent techniques and present the task in the context of graph machine learning.

2.3.1. Self-training, Co-training

Self-training is a technique where a classifier learns from its own predictions. It operates by iteratively augmenting the set of labeled samples to make predictions on the unlabeled set (X. Zhu and Goldberg 2009): given some classifier $f : \mathbf{x} \to y$, and given the original sets of labeled $\mathcal{D}_L = \{(\mathbf{x}_i, y_i)\}_{i=1}^{L}$ and unlabeled $\mathcal{D}_U = \{\mathbf{x}_j\}_{j=L+1}^{N}$ instances, the classifier is trained on $\mathcal{D}_L$ and evaluated on $\mathcal{D}_U$. Then a subset $\mathcal{S} = \{(\mathbf{x}_k, \hat{y}_k)\}_{k \in \text{top}-K}$, with $\hat{y}_k = f(\mathbf{x}_k)$, consisting of the K most confident predictions (along with their corresponding feature vectors) is removed from $\mathcal{D}_U$ and added to $\mathcal{D}_L$ to form an augmented training set $\mathcal{D}_L \leftarrow \mathcal{D}_L \cup \mathcal{S}$. Subsequently, f is re-trained on this updated training set and evaluated on the remaining samples, $\mathcal{D}_U \setminus \mathcal{S}$. This procedure is repeated continuously by selecting the K most confident predictions at each iteration. Clearly, this approach relies on the k selected labels being correct in the first place, else it re-trains on mislabeled data, leading to errors that can get reinforced at each iteration.

As previously discussed, in contrast to self-training, where a single classifier f is repeatedly trained, co-training relies on two independent classifiers, $f_1 : \mathbf{x} \to y$ and $f_2 : \mathbf{x} \to y$ (X. Zhu and Goldberg 2009): given the original set of labeled samples $\mathcal{D}_L = \{(\mathbf{x}_i, y_i)\}_{i=1}^{L}$, the features are split into two disjoint sets, $\mathbf{x} = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)})$, so each sample gets two separate subsets of features (a.k.a. "views") $\mathcal{D}_L^{(1)} = \{(\mathbf{x}_i^{(1)}, y_i)\}_{i=1}^{L}$, and $\mathcal{D}_L^{(2)} = \{(\mathbf{x}_i^{(2)}, y_i)\}_{i=1}^{L}$, while $\mathcal{D}_U = \{\mathbf{x}_j\}_{j=L+1}^{N}$ are the unlabeled samples. Each classifier is trained on its corresponding set of features and evaluated on $\mathcal{D}_U$, and a subset of the most confident predictions in each case is drawn from $\mathcal{D}_U$. Then, the most confident predictions from f_1 (along with their corresponding feature vectors) are added to $\mathcal{D}_L^{(2)}$, and analogously for f_2 and $\mathcal{D}_L^{(1)}$. Finally, each model is re-trained on its respective updated set of labeled instances, and the process is repeated for a number of iterations. Therefore, this approach relies on the features used to train one classifier being conditionally independent from those used to train the other one, and on each subset of features being sufficiently informative to let each classifier make accurate predictions (X. Zhu 2008).

2.3.2. Semi-supervised learning on graphs

In graph-based SSL, given a graph $G = (\mathcal{V}, \mathcal{E})$, as defined in Section 2.1, nodes $v \in \mathcal{V}$ represent datapoints, while edges represent relationships between them. Furthermore, each node is associated with a feature vector $\mathbf{x}_v \in \mathbb{R}^d$. $\mathcal{V}$ is split into a labeled[5] node subset $\mathcal{V}_L \subset \mathcal{V}$, whose labels are used during training, and an unlabeled node subset $\mathcal{V}_U = \mathcal{V} \backslash \mathcal{V}_L$. Also, typically $|\mathcal{V}_L| \ll |\mathcal{V}_U|$. In general, the goal is to learn a model f to predict the labels of $\mathcal{V}_U$ using the features of all nodes in $\mathcal{V}$, the labels of the nodes in $\mathcal{V}_L$, the information available in the graph topology, or a combination thereof. Thus, semi-supervised node classification (SSNC) is the subtask of SSL in which the labels of the nodes correspond to –a finite set of– classes, as depicted in Figure 2.3.

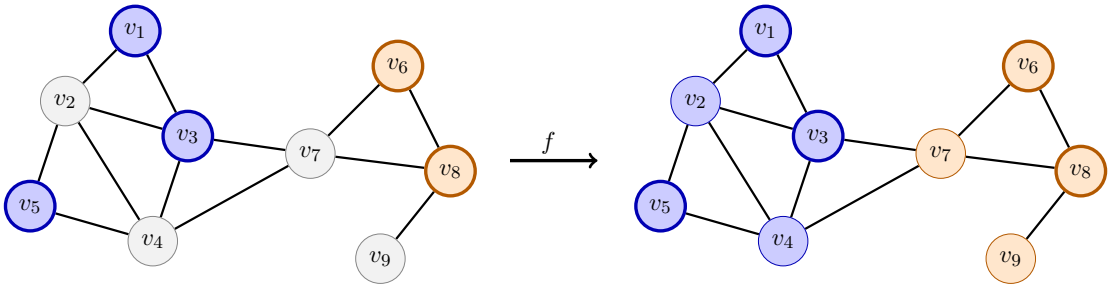


Figure 2.3.: The SSNC task: given sets of labeled and unlabeled samples (represented by colored and gray nodes respectively), predict the classes of the unlabeled nodes using the available labels, all nodes' feature vectors and the graph's structure (edge index).

One of the biggest motivations for graph-based SSL is the ability to use the structural information of graphs to compensate for the limited availability of labels. In traditional machine learning with *i.i.d.* data, having more labeled samples leads to better generalization power under certain assumptions. Conversely, label scarcity leads to high variance and poorer generalization power. In graph SSL, graph topology represents an additional source of information, which allows models to learn not only from features and available labels, but from the inherent relational structure of the data, such that the negative effect of label scarcity can be partially mitigated (Subedi et al. 2025; Q. Li, Han, and X.-M. Wu 2018). For instance, the PubMed citation dataset (Z. Yang, Cohen, and Salakhutdinov 2016) consists of 19,717 nodes and 44,338 edges, and it's public split defines a label rate of barely 0.3% (or 60 nodes) for training, yet graph-based models achieve strong accuracy scores (GCN: 79.0%, Planetoid: 77.2%, ICA: 73.9%), in contrast to graph-agnostic baselines (MLP: 71.4%) (Kipf and Welling 2017). This illustrates how graph structure can be an additional source of information and improve performance beyond pure feature-based learning.

Within SSNC, we mainly distinguish between two paradigms (Z. Yang, Cohen, and Salakhutdinov 2016; William L Hamilton 2020):

- **Transductive setting**: The entire graph (nodes and edges) is visible during

[5] In the context of this work, we also refer to these nodes as the "visible" or "observed" nodes.

training. Specifically, both nodes in $\mathcal{V}_L$ and $\mathcal{V}_U$ (and all corresponding edges) are available to the model during training, and the loss is computed exclusively on $\mathcal{V}_L$ (for GCN, through Eq. (4.4)). During inference, the labels of the nodes in $\mathcal{V}_U$ are predicted, which were already part of the training graph. The original GCN formulation was made considering a transductive setup.

- **Inductive setting**: Inference can be performed on previously unseen nodes $v' \notin \mathcal{V}$, or graphs $G' = (\mathcal{V}', \mathcal{E}')$. These new samples were never used during training. Models like GAT and GraphSAGE naturally accommodate to this setting.

Graph Neural Networks are nowadays the standard approach for SSL on graphs, as they combine learning through latent node representations, as in methods like DeepWalk and node2vec, while learning a classifier by exploiting information from neighboring nodes. In this sense, Planetoid (Z. Yang, Cohen, and Salakhutdinov 2016), represented the bridge between graph embedding methods and neural-based methods. In parallel to graph representation learning, early approaches for SSL in graphs relied on the relational structure of the graph to learn labels from neighboring nodes without explicitly generating node embeddings. Examples are collective classification methods like the Iterative Classification Algorithm (ICA) and label diffusion algorithms like Label Propagation (LP). Some of these classical methods for SSL on graph data are reviewed in Section 3.4.

2.4. Network analysis

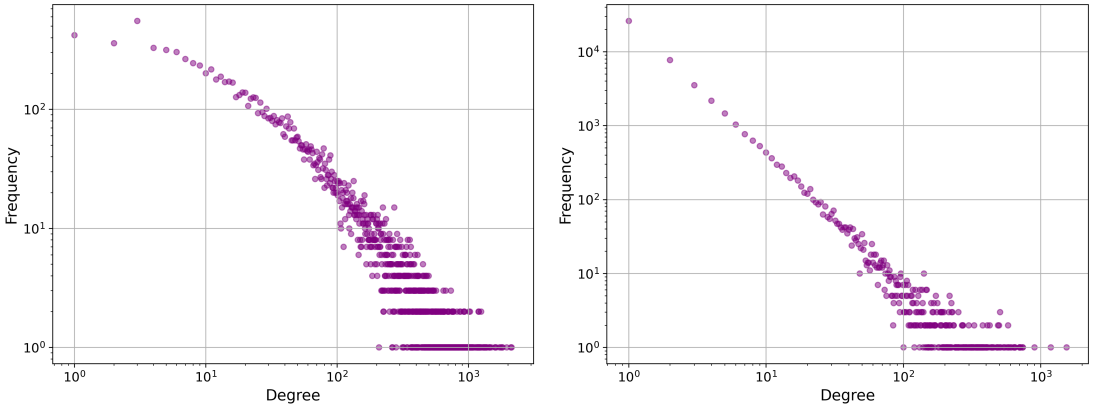
In practice, the terms *graph* and *network* are used interchangeably, though some academics make a terminological distinction: graphs are often analyzed from the abstract, mathematical point of view, while a network often relates to real-world systems like social networks, biological networks or the WWW, where nodes commonly represent specific entities and edges represent relationships between them. The terminology introduced in Section 2.1, particularly the notions of adjacency, degree and Laplacian matrices, neighborhoods and graph attributes, is therefore valid when analyzing networks. This section introduces some further concepts used in network analysis.

2.4.1. Characterizing networks

In network science, networks can be characterized through a number of structural and functional metrics. The definitions here are based mainly on Mark E. J. Newman 2016 and Barabási 2013.

1. **Degree distribution**: given the degree of a node $v \in \mathcal{V}$, $d(v)$ (as defined before), the degree distribution gives the probability distribution of $k = d(v)$. Common patterns are regular distributions, where all nodes have the same degree $P(k) = 1$ if $k = d$, 0 otherwise (so, d-regular graphs have node degree d), and power-law distributions, where node degrees follow a decaying function $P(k) \propto k^{-\gamma}$. Networks following this distribution are called scale-free (Barabasi and Albert 1999).

Empirical networks typically exhibit right-skewed distributions like power-law, but also log-normal, exponential or Poisson-like degree distributions. Figure 2.4 shows the degree distributions of the Tolokers and Questions empirical datasets.



(a) Tolokers. (b) Questions.

Figure 2.4.: Degree distributions (on a log-log scale) of the Tolokers and Questions datasets. Both exhibit heavy-tailed degree distributions, with a vast number of low degree nodes and a low number of very high degree nodes ("hub nodes"), with Tolokers having a more concentrated number of intermediate degree nodes.

2. **Clustering coefficient**: *transitivity* is a property in networks where, if there exist an edge between a pair of nodes (u, v) and between (v, w), then it is likely that there is also an edge between (u, w). In other words, if (u, v, w) define a path of length 2 and transitivity holds, then the path is closed, i.e. it defines a triangle. The clustering coefficient Γ is a measure of transitivity: it quantifies the proportion of paths of length 2 that are closed. Intuitively, if $\Gamma = 1$ then all paths of length 2 form cliques. One can also distinguish between the local clustering coefficient γ_i of a single node v_i, defined as the proportion of pairs of neighbors of v_i that are connected, and the global clustering coefficient for the whole network, defined as $\bar{\Gamma} = \frac{1}{N} \sum_i \gamma_i$.

3. **Centrality**: in general, centrality refers to the notion of some nodes in the network being important according to some metric. Some of these metrics are:

   - *Degree centrality*: the most natural notion, a node is important if many other nodes connect to it, i.e. if its degree is high, as given by the vector of node degrees $\mathbf{d} = \mathbf{A} \cdot \mathbf{1}$.
   - *Eigenvector centrality*: a node is important if it is connected to other important nodes, and its eigenvector centrality is given by the sum of the centralities of its neighbors (Bonacich 1987), weighted by $\frac{1}{\lambda_n(\mathbf{A})}$, where λ_n is the eigenvalue of the dominant eigenvector of $\mathbf{A}$, derived through the power method: $c_i = \frac{1}{\lambda_n(\mathbf{A})} \sum_j A_{ij} c_j$.

- *Katz centrality*: similarly to eigenvector centrality, importance is given by a the centrality of a node's neighbors, and additionally every node is assigned a baseline centrality β_i (Katz 1953): $c_i = \alpha \sum_j A_{ij} c_j + \beta_i$, where $\alpha < \frac{1}{\rho(\mathbf{A})}$ and $\rho(\mathbf{A})$ is the spectral radius of $\mathbf{A}$.
- *Closeness centrality*: a node is important if it has several short paths to other nodes, quantified as the average distance to other nodes: $c_i = \frac{1}{N-1} \sum_{i \neq j} \frac{1}{d_{ij}}$, where d_{ij} is the length of the shortest path between v_i and v_j.
- *Betweenness centrality*: a node is important if it appears in the shortest path between many pairs of nodes: $c_i = \sum_{st} \frac{n_{st}^i}{g_{st}}$, where g_{st} is the total number of shortest paths (geodesic paths) between v_s and v_t, and n_{st}^i is the number of such paths in which v_i is present.

4. **Components**: As described in Section 2.1, in a connected network every node can be reached from every other node. Thus, in this case, the network consists of a single component. In disconnected graphs, each subset of nodes such that there exists a path between every pair of nodes in the subset and no path to nodes in other subsets is called a component. Singleton vertices are also considered components. Furthermore, the adjacency matrix of a disconnected network takes the form of blocks in the diagonal of the matrix (assuming nodes are correctly ordered in the matrix), such that each block represents a component, and the entries around the blocks are 0, meaning there are no connections across components. For example, for a simple 2-component network:

$$\mathbf{A} = \begin{pmatrix} \boxed{\begin{matrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{matrix}} & \mathbf{0} \\ \mathbf{0} & \boxed{\begin{matrix} 0 & 1 \\ 1 & 0 \end{matrix}} \end{pmatrix}$$

   In attributed networks, if each component contains only nodes of one class, then such network is said to be maximally homophilic (in the edge homophily sense, as will be defined in the next section). In the example, this would mean that $y_1 = y_2 = y_3 = c_1$ and $y_4 = y_5 = c_2$ for classes c_1 and c_2.

5. **Similarity**: there exist a number of ways of quantifying whether two nodes are similar. Structural similarity refers to nodes sharing several nodes, so if many of their neighbors overlap, the nodes are similar: $s_{ij} = [\mathbf{A}^2]_{ij} = \sum_k A_{ik} A_{kj}$, where the ij-th element of $\mathbf{A}^n$ gives the number of walks of length n between v_i and v_j. This measure is not normalized, so high degree nodes will tend to connect more and appear to be more similar, to overcome this, we can normalize by the geometric mean of the degrees, which is proportional to the cosine similarity of the nodes' neighborhood vectors $A_{i:}$, $A_{j:}$: $\sigma_{ij} = \frac{s_{ij}}{\sqrt{d(v_i)}\sqrt{d(v_j)}}$. If two nodes have the

same neighbors, then $\sigma_{ij} = 1$. Furthermore, to measure the number of common neighbors with respect to a baseline, we can compare the observed neighborhood vectors with the expected neighborhood assignments on a network where neighbors would be chosen randomly, which is equal to the covariance of the neighborhood vectors: $\text{cov}(A_{i:}, A_{j:}) \propto s_{ij} - \frac{d(v_i)d(v_j)}{N}$. Normalizing by the product of the standard deviations of the vectors, we get the Pearson correlation coefficient: $\rho_{ij} = \frac{\text{cov}(A_{i:},A_{j:})}{\sqrt{\text{var}(A_{i:})}\sqrt{\text{var}(A_{j:})}}$.

6. **Connectivity**: any pair of connected nodes can have more than one path that links them. Such paths can be node-independent if they share no nodes (other than the nodes at the extremes), and edge-independent if they share no edges. The number of node-/edge-independent paths between a pair of nodes represents how strongly connected they are, i.e. their connectivity. This notion is often operationalized through the *cut set*, the minimal set of nodes that would need to be removed to make a pair of nodes disconnected (and analogously for *edge cut sets*). Connectivity is closely related to community structure, as communities typically have higher connectivity of intra-community nodes than inter-community nodes.

7. **Community structure**: graph partitioning and community detection refer to the task of dividing the network into groups according to some criteria. Concretely, graph partitioning is the task of dividing the network into a predefined number of groups of balanced sizes, which is commonly achieved by maximizing the number edges within groups while minimizing the number of edges across groups. Algorithms for graph partitioning like Kernighan-Lin (Kernighan and Lin 1970) commonly operate by minimizing the cut size, i.e. the number of inter-group edges. In contrast, community detection does not constrain a fixed number of groups, and the most common approach is to maximize a network's modularity, defined as:

$$Q = \frac{1}{2E} \sum_{ij} \left(A_{ij} - \frac{d(v_i)d(v_j)}{2E} \right) \delta\left[c_i = c_j\right] \tag{2.7}$$

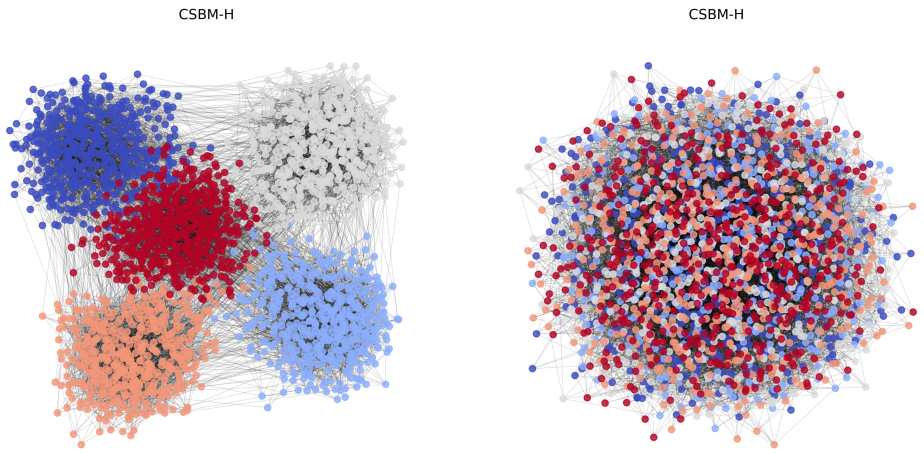
where c_i and c_j are the community assignments of v_i and v_j, and $\delta\left[c_i = c_j\right]$ is the Kronecker delta function, which is 1 if $c_i = c_j$, and 0 otherwise, such that we only count edges of nodes in the same community. Intuitively, modularity represents the difference between the observed number of intra-community edges in the network and the number of edges that would be expected in a random graph with the same degree sequence (a.k.a. *configuration model*), normalized by the actual number of edges, $\frac{1}{2E} = \frac{1}{\sum_v d(v)}$ (each edge appears twice, once per endpoint). So, the higher the modularity, the more nodes tend to form densely connected communities. Algorithms like Louvain (Blondel et al. 2008) and Leiden (Traag, Waltman, and Van Eck 2019) aim to find community assignments that maximize modularity, where the sizes of the communities are not fixed.

2.4.2. Homophily

Assortative and disassortative mixing

The notion of assortative mixing, or *homophily*, refers to the tendency of entities sharing some attribute to connect to each other. In networks, such attributes can be either enumerative characteristics (categorical labels)[6] like gender in social networks, species in biological networks, or topics in citation networks, or scalar (discrete or continuous) characteristics like income in social networks or atomic charge in molecules (Mark E. J. Newman 2016). Additionally, assortative mixing can refer to the structural properties of nodes, for instance, assortative mixing by degree, where high degree nodes (hubs) tend to connect with other high degree nodes while low degree nodes tend to connect to other low degree nodes, as is often the case in social networks (Barabási 2013).

Conversely, in disassortative networks, nodes with different attributes tend to connect to each other. Examples for this pattern are trophic networks, where individuals of one species tend to feed on members of different species, and the internet routing network, where many small routers are wired to highly connected hubs. Figure 2.5 shows examples of synthetic assortative and disassortative networks.



(a) Homophily/assortative mixing. (b) Heterophily/disassortative mixing.

Figure 2.5.: Synthetically generated networks with different levels of (label) homophily, where colors correspond to class membership.

Therefore, a network can be characterized as assortative if a significant number of edges link nodes of the same type, for which it is convenient to define a baseline that shows how often nodes of the same type would connect to each other if the links between

[6]Some works refer to homophily as assortativity specifically at the level of labels. In this work, we adopt it as an umbrella term that encompasses label-based assortativity, but also assortativity at the feature/attribute level and at the structural level, following Zheng, Luan, and Lihui Chen 2024 and Mark E. J. Newman 2016.

them were placed at random considering their degrees (*configuration/null model*), and then compare this with the observed fraction of edges linking nodes of the same type. This is equivalent to the modularity definition in Eq. (2.7), where instead of using the indicator function to denote whether two nodes belong to the same community, we use $\delta[y_i = y_j]$, which is 1 if the class of v_i and v_j is the same, and 0 otherwise:

$$Q = \frac{1}{2E} \sum_{ij} \underbrace{\left(A_{ij} - \frac{k_i k_j}{2E} \right)}_{B_{ij}} \delta[y_i = y_j] \tag{2.8}$$

where $k_i = d(v_i)$, and B_{ij} are the entries of the *modularity matrix* $\mathbf{B}$. Intuitively, the value B_{ij} takes indicates the following patterns:

$$\begin{aligned} B_{ij} > 0: &\quad v_i, v_j \text{ are more strongly connected than expected by chance.} \\ B_{ij} < 0: &\quad v_i, v_j \text{ are less strongly connected than expected by chance.} \\ B_{ij} \approx 0: &\quad v_i, v_j \text{ are connected approximately as expected by chance.} \end{aligned}$$

We previously considered the attribute defining node similarity to be their class membership (an enumerative characteristic), but this reasoning also extends to scalar characteristics. For this, we can define x_i as the value of some numerical attribute of node v_i. Then, the covariance of the node attributes across all edges is also given in terms of the modularity matrix:

$$\operatorname{cov}(x_{\text{src}}, x_{\text{dst}}) = \frac{1}{2E} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2E} \right) x_i x_j = \frac{1}{2E} \sum_{ij} B_{ij} x_i x_j$$

where x_{src} and x_{dst} denote the endpoints (in arbitrary order in the case of undirected networks) of edges. So, the covariance will be positive when the values of the attribute of connected nodes tend to be similar (both above average, both below average), 0 if the attribute varies as expected under the null model, and negative if the values of the attribute tend to be dissimilar (above-average connecting to below-average).

To account for the scale of the numerical attribute and fix the interval of the metric, Mark E. J. Newman 2016 normalizes the covariance by the standard deviations of the endpoint attributes to define the *assortativity coefficient*, or the Pearson correlation coefficient between the endpoint attributes:

$$\rho = \frac{\sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2E} \right) x_i x_j}{\sum_{ij} \left(k_i \delta_{ij} - \frac{k_i k_j}{2E} \right) x_i x_j} = \frac{\operatorname{cov}(x_{\text{src}}, x_{\text{dst}})}{\sqrt{\operatorname{var}(x_{\text{src}})} \sqrt{\operatorname{var}(x_{\text{dst}})}} \tag{2.9}$$

with $\delta_{ij} = 1$ if $i = j$ and 0 otherwise, so that $k_i \delta_{ij}$ contributes only when $i = j$. Now the metric lies in the $-1 \leq \rho \leq 1$ interval, where $\rho > 0$ indicates assortative mixing and $\rho < 0$ indicates disassortative mixing. It is noteworthy that x_i can be any attribute of the node, even structural properties, like its degree (Eq. (2.20)), local clustering coefficient (Eq. (2.21)), centrality, etc. Finally, attributes need not be scalars x_i, but we

can compare entire feature vectors $\mathbf{x}_i$ across edge endpoints, as is the case with Local Similarity, per Eq. (2.18) and Eq. (2.19).

Naturally, homophily can be quantified in several other ways, the previous discussion revolved around the notion of assortativity, the tendency of nodes of the same type to connect more than would be expected under a configuration model. In the following section we review some metrics that are commonly used in GNN literature to quantify homophily from different perspectives: label-based, feature-based and structure-based homophily[7].

Label-based homophily metrics

Edge homophily (J. Zhu, Yan, et al. 2020; Abu-El-Haija et al. 2019) quantifies the proportion of edges whose endpoints share the same label. It can be interpreted as the probability of sampling a random edge that connects nodes of the same class:

$$\boxed{h_{\text{edge}} = \frac{|\{\{u, v\} \in \mathcal{E} : y_u = y_v\}|}{|\mathcal{E}|}} \tag{2.10}$$

where $h_{\text{edge}} \in [0, 1]$. Also, it can be expressed in terms of the adjacency matrix:

$$h_{\text{edge}} = \frac{\sum_{ij} A_{ij}\, \delta[y_i = y_j]}{\sum_{ij} A_{ij}} \tag{2.11}$$

where $\delta[y_i = y_j]$ is the Kronecker delta function. Clearly, this metric is biased in the case of class imbalances: networks with some class(es) concentrating the majority of the nodes will show high edge homophily even under random mixing since the probability of connecting to those classes is higher for nodes of any class. It is also affected by irregular degree distributions across classes: classes with large hubs will inflate h_{edge} as nodes in such classes will naturally connect to each other because of their high degrees even under random mixing.

Node homophily (Pei et al. 2020), similarly to edge homophily, gives the proportion of neighbors that share the same label, averaged over all nodes in the network:

$$\boxed{h_{\text{node}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \frac{|\{u \in \mathcal{N}(v) : y_u = y_v\}|}{d(v)}} \tag{2.12}$$

where $h_{\text{node}} \in [0, 1]$, and the average is taken only over valid (non-isolated) nodes: $\{v \in \mathcal{V} : |\mathcal{N}(v)| > 0\}$. While edge homophily is biased by high-degree nodes, node homophily weights all nodes equally, yet it is also affected by imbalanced classes: highly populated classes will disproportionately contribute more neighbors independently of the ego node's class.

[7] In practice, when discussing homophily, authors usually refer specifically to label-based metrics, especially h_{edge}.

Class homophily (Lim, X. Li, et al. 2021) was proposed to alleviate the problem of class imbalances. It quantifies whether nodes from each class connect more to each other than expected by chance, averaged over all C classes:

$$\boxed{h_{\text{class}} = \frac{1}{C-1}\sum_{c=1}^{C}\left[\frac{\sum_{v:y_v=c}|\{u \in \mathcal{N}(v) : y_u = c\}|}{D_c} - \frac{N_c}{N}\right]_+} \tag{2.13}$$

where $h_{\text{class}} \in [0,1]$ thanks to the $\frac{1}{C-1}$ normalization. D_c is the total degree of class c: $D_c = \sum_{v:y_v=c} d(v)$. The inner term represents the fraction of neighbors of class c nodes that are also in class c, minus a baseline probability of class c nodes, $\frac{N_c}{N}$, which corrects for class imbalances. Therefore, h_{class} quantifies whether class c nodes connect more than expected by chance, and only considers classes where this is the case (i.e. it measures per-class excess homophily), as given by $[x]_+ = \max\{x, 0\}$.

Adjusted homophily (Peel, Delvenne, and Lambiotte 2018; M. E. J. Newman 2002; Platonov, Kuznedelev, Babenko, et al. 2024) builds upon the modularity definition in Eq. (2.8), which compares the observed homophily with a baseline defined by the configuration model. Adjusted homophily measures how much more homophily is observed than expected at random, normalized by the maximum possible excess above chance to correct for class/degree distributions, so it can be considered a normalized measure of modularity:

$$\boxed{h_{\text{adj}} = \frac{h_{\text{edge}} - \sum_{c=1}^{C}\left(\frac{D_c}{2E}\right)^2}{1 - \sum_{c=1}^{C}\left(\frac{D_c}{2E}\right)^2}} \tag{2.14}$$

where $h_{\text{adj}} \in [-1,1]$. Adjusted homophily is also called the *categorical assortativity coefficient*, the equivalent of Eq. (2.9) for enumerative attributes (classes).

Unbiased homophily (Mironov and Prokhorenkova 2024) was proposed as an alternative to class and adjusted homophily, both of which, although they correct for class imbalances, violate 2 desirable properties of homophily measures: *monotonicity*, which requires that adding a homophilic or removing an heterophilic edge must result in an increase in homophily, and *minimal agreement*, which requires that graphs with only heterophilic edges have a consistent value for homophily (class homophily additionally violates other 2 desirable properties, as detailed by the authors). Unbiased homophily was explicitly designed to satisfy all desirable properties. It relies on the *normalized class adjacency matrix* $\mathbf{C}_G \in \mathbb{R}^{C\times C}$, where C is the number of classes of G, whose entries are defined as:

- $c_{ii} = \frac{|\{\{u,v\}\in\mathcal{E}: y_u = y_v = i\}|}{E}$: number of edges with both nodes from the same class, normalized by the total number of edges.
- $c_{ij} = \frac{|\{\{u,v\}\in\mathcal{E}: \{y_u, y_v\} = \{i,j\}\}|}{2E}$: number of edges with nodes from classes i, j ($i \neq j$), normalized by the total number of edges (off-diagonal edges counted twice, so we normalize by $\frac{1}{2E}$).

Unbiased homophily measures the difference between the expected (geometric mean $\sqrt{c_{ii}c_{jj}}$) and observed (c_{ij}) fraction of heterophilic edges, plus normalization:

$$\boxed{h_{\text{unb}} = \frac{\sum_{i<j}\left(\sqrt{c_{ii}c_{jj}} - c_{ij}\right)}{\sum_{i<j}\left(\sqrt{c_{ii}c_{jj}} + c_{ij}\right)}} \tag{2.15}$$

where $h_{\text{unb}} \in [-1, 1]$. Since, $\mathbf{C}_G$ is symmetric, the sums run for $i < j$, else heterophilic edges c_{ij} would be counted twice. It is not affected by class imbalances, and it is also calibrated to take the values -1, 0, $+1$ for fully heterophilic, random and fully homophilic graphs, respectively.

2-hop homophily is a natural extension of node homophily to the second order neighborhood of each node. Quantifying homophily over higher order neighborhoods has been shown to give deeper insights of the structural patterns of the network in certain cases (J. Zhu, Yan, et al. 2020; Cavallo et al. 2022). Several definitions of a higher order neighborhood exist, and they differ mainly on how to handle the lower order neighborhoods and the ego node. The definition used in this work is similar to the one used in J. Zhu, Yan, et al. 2020, but we define the second order neighborhood to be the set of –unique– nodes exactly 2-hops away from the ego node: $\mathcal{N}^{(2)}(v) = \{u \in \mathcal{V} : \text{dist}(u, v) = 2,\ u \neq v\}$. So, the definition used in this work is:

$$\boxed{h^2 = \frac{1}{|\mathcal{V}|}\sum_{v\in\mathcal{V}} \frac{\left|\left\{u \in \mathcal{N}^{(2)}(v) : y_u = y_v\right\}\right|}{|\mathcal{N}^{(2)}(v)|}} \tag{2.16}$$

where $h^2 \in [0, 1]$, and $\mathcal{V}$ excludes nodes that don't have any 2-hop neighbor: $\{v \in \mathcal{V} : |\mathcal{N}^{(2)}(v)| > 0\}$, similarly to h_{node}.

Relative Mutual Information (RMI) (Hussain et al. 2021) is a label-community correlation measure. It relies on the observation that a node's community membership is often informative of its class membership. Given random variables $Y \in \mathcal{Y}$ and $C \in \mathcal{C}$ denoting the graph's label and community assignments respectively, it quantifies the fraction of uncertainty about Y that is removed after knowing C, where C is given by some community detection algorithm like Louvain, see Section 2.4.1:

$$\boxed{\text{RMI} = U(Y|C) = \frac{H(Y) - H(Y|C)}{H(Y)} = \frac{I(Y;C)}{H(Y)}} \tag{2.17}$$

where RMI $\in [0, 1]$, $H(Y)$ is the entropy of Y, a normalizing factor to quantify the original uncertainty about Y before knowing C, and $I(Y;C)$ is the mutual information between the random variables.

Feature-based homophily metrics

Generalized Edge Homophily (Jin et al. 2022) measures the similarity between all edge endpoints across all the graph's edges:

$$h_{\text{GE}} = \frac{1}{|\mathcal{E}|}\sum_{\{u,v\}\in\mathcal{E}} \text{sim}(u, v)$$

where $\text{sim}(u, v)$ can be realized using different pairwise similarity measures. In particular, if we define it as the class indicator function $\delta[y_u = y_v]$, this measure is equivalent to Eq. (2.11). If we consider $\text{sim}(u, v)$ to be the cosine similarity of the feature vectors of each pair of connected nodes, we get a label-agnostic measure of homophily:

$$\boxed{h_{\text{GE}} = \frac{1}{|\mathcal{E}|} \sum_{\{u,v\} \in \mathcal{E}} \frac{\mathbf{x}_u \mathbf{x}_v}{\|\mathbf{x}_u\| \, \|\mathbf{x}_v\|} = \frac{1}{|\mathcal{E}|} \sum_{\{u,v\} \in \mathcal{E}} \cos\left(\mathbf{x}_u, \mathbf{x}_v\right)}$$

where $h_{\text{GE}} \in [-1, 1]$.

Feature Assortativity is used in this work as the extension of the scalar assortativity coefficient defined in Eq. (2.9) to feature vectors $\mathbf{x}_v = (x_v^{(1)}, \ldots, x_v^{(d)}) \in \mathbb{R}^d$ instead of scalar attributes. Two approaches are used in this setting. First, we compute the correlation coefficient between the flattened feature vectors of all edge endpoints $\mathbf{x}_{\text{src}}$, $\mathbf{x}_{\text{dst}}$:

$$\boxed{\rho_{\text{flat}} = \frac{\text{cov}(\mathbf{x}_{\text{src}}, \mathbf{x}_{\text{dst}})}{\sqrt{\text{var}(\mathbf{x}_{\text{src}})}\sqrt{\text{var}(\mathbf{x}_{\text{dst}})}}}$$

Alternatively, it is possible compute the correlation between corresponding features individually, where $\mathbf{x}_{\text{src}}^{(d)}$, $\mathbf{x}_{\text{dst}}^{(d)}$ are the vectors containing feature d over all edge endpoints:

$$\rho_d = \frac{\text{cov}(\mathbf{x}_{\text{src}}^{(d)}, \mathbf{x}_{\text{dst}}^{(d)})}{\sqrt{\text{var}(\mathbf{x}_{\text{src}}^{(d)})}\sqrt{\text{var}(\mathbf{x}_{\text{dst}}^{(d)})}}$$

and then average over all dimensions to define a measure of per-dimension feature assortativity:

$$\boxed{\rho_{\text{dim}} = \frac{1}{d} \sum_d \rho_d}$$

where $\rho_{\text{dim}} \in [-1, 1]$ and $\rho_{\text{flat}} \in [-1, 1]$.

Local Similarity (Y. Chen et al. 2023) is similar to Generalized Edge Homophily: while h_{GE} measured similarity at the level of edge endpoints, h_{LS} measures the average similarity of the features of a node and its neighbors:

$$h_{\text{LS}}(v) = \frac{1}{d(v)} \sum_{u \in \mathcal{N}(v)} \text{sim}(u, v)$$

Concretely, if we consider $\text{sim}(u, v)$ to be the negative Euclidean (L_2) distance between the feature vectors of the ego node and its neighbors, and take the average over all nodes:

$$\boxed{h_{\text{LS-euc}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \frac{1}{d(v)} \sum_{u \in \mathcal{N}(v)} \left(-\|\mathbf{x}_v - \mathbf{x}_u\|_2\right)} \tag{2.18}$$

where $h_{\text{LS-euc}} \in (-\infty, 0]$, so that if $h_{\text{LS-euc}} \to -\infty$ the features are maximally dissimilar and if $h_{\text{LS-euc}} = 0$ the features are identical (maximally homophilic). Alternatively, defining $\text{sim}(u, v)$ as the cosine similarity between corresponding feature vectors:

$$\boxed{h_{\text{LS-cos}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \frac{1}{d(v)} \sum_{u \in \mathcal{N}(v)} \cos\left(\mathbf{x}_u, \mathbf{x}_v\right)} \tag{2.19}$$

where $h_{\text{LS-cos}} \in [-1, 1]$. Also, if we define $\text{sim}(u, v)$ as the class indicator function $\delta[y_u = y_v]$, this measure is equivalent to Eq. (2.12).

Structure-based homophily metrics

Degree Assortativity (Mark E. J. Newman 2016) is the realization of the scalar assortativity coefficient defined in Eq. (2.9), where we define the scalar attribute to be a node's degree $x_i = k_i = d(v_i)$:

$$\boxed{\rho_{\text{deg}} = \frac{\sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2E}\right) k_i k_j}{\sum_{ij} \left(k_i \delta_{ij} - \frac{k_i k_j}{2E}\right) k_i k_j}} \tag{2.20}$$

LCC Assortativity is also the realization of the scalar assortativity coefficient where we define the scalar attribute to be the a node's local clustering coefficient $x_i = \gamma_i$:

$$\boxed{\rho_{\text{lcc}} = \frac{\sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2E}\right) \gamma_i \gamma_j}{\sum_{ij} \left(k_i \delta_{ij} - \frac{k_i k_j}{2E}\right) \gamma_i \gamma_j}} \tag{2.21}$$

where $\rho_{\text{deg}} \in [-1, 1]$ and $\rho_{\text{lcc}} \in [-1, 1]$.

Neighborhood Similarity (Zheng, Luan, and Lihui Chen 2024) is a measure of the similarity of the neighborhood distributions of intra-class and inter-class nodes. The original definition was given by Y. Ma et al. 2023 as Cross-Class Neighborhood Similarity (CCNS), as the similarity of the neighborhood label distributions of the nodes belonging to two classes, c and c':

$$\text{CCNS}(c, c') = \frac{1}{|\mathcal{V}_c||\mathcal{V}_{c'}|} \sum_{u \in \mathcal{V}_c} \sum_{v \in \mathcal{V}_{c'}} \cos(D^{\mathcal{N}}_{uc}, D^{\mathcal{N}}_{vc})$$

where $\mathbf{D}^{\mathcal{N}} \in [0, 1]^{N \times C}$ is the neighbor distribution matrix, whose entries represent the probability that a randomly sampled neighbor of v belongs to class c: $D^{\mathcal{N}}_{vc} = P(Y_u = c \mid u \in \mathcal{N}(v))$, so $D^{\mathcal{N}}_{v:}$ gives the probability distribution of the neighbors of v. Clearly, CCNS is not a scalar but a matrix of similarities across different classes, so Zheng, Luan, and Lihui Chen 2024 define neighborhood similarity as the expected similarity of intra-class node neighborhood distributions, divided by the expected similarity of inter-class

node neighborhood distributions:

$$\boxed{h_{\text{NS}} = \frac{\frac{1}{|\{u,v \in \mathcal{V}\colon y_u = y_v\}|} \sum_{y_u = y_v} S_{uv}}{\frac{1}{|\{u,v \in \mathcal{V}\colon y_u \neq y_v\}|} \sum_{y_u \neq y_v} S_{uv}}}$$

where $h_{\text{NS}} \in [0, \infty)$. $\mathbf{S} \in \mathbb{R}^{N \times N}$ contains the pairwise similarity between the neighborhood distributions of connected nodes: $S_{uv} = \cos\left(D^{\mathcal{N}}_{u:}, D^{\mathcal{N}}_{v:}\right)$. So, it measures structural homophily at the level of node neighborhood distributions: if nodes of the same class tend to have the same neighborhood distributions and nodes of different classes tend to have different neighborhood distributions, then the network will be structurally homophilic.

2.4.3. Synthetic graph generation models

Empirical graphs are structurally heterogeneous, which makes it harder to study specific properties in isolation. Synthetic graphs allow to control the structure of a graph (degree distributions, class assignments, homophily, etc.). Early generative models like the Erdős–Rényi models (Gilbert 1959; Erdős and Rényi 2022) generate edges independently for a predefined set of n nodes with uniform probability $p = P(\{u, v\} \in \mathcal{E})$, yielding a Poisson-like degree distribution for large graphs ($G(n, p)$ model), or directly generate graphs by randomly sampling a graph with n nodes and M edges from the universe of graphs with n nodes and M edges ($G(n, M)$ model). Barabasi and Albert 1999 model preferential attachment by assigning the neighbors of a new node with a probability proportional to their degree, so high degree nodes tend to make more new connections, yielding scale-free graphs.

Stochastic Block Models (SBM) (Holland, Laskey, and Leinhardt 1983) incorporate the ability to influence edge sampling based on class membership. While previous models don't model node classes/labels, the SBM starts by assigning each node v to a class $c \in \mathcal{C}$ (e.g. uniformly at random), and then defines a probability ("block") matrix $\mathbf{P} \in [0, 1]^{C \times C}$, where $C = |\mathcal{C}|$, which defines the probability of an edge connecting each pair of classes, including the same class:

$$\mathbf{P} = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1C} \\ p_{21} & p_{22} & \cdots & p_{2C} \\ \vdots & \vdots & \ddots & \vdots \\ p_{C1} & p_{C2} & \cdots & p_{CC} \end{bmatrix}$$

Therefore, it allows to model community structure and, indirectly, the level of homophily of a graph. The Contextual Stochastic Block Model (CSBM) (Deshpande et al. 2018) builds upon the SBM and incorporates node features, which can be sampled from a probability distribution according to their class membership, e.g. $\mathbf{x}_v \mid c_v \sim \mathcal{N}(\boldsymbol{\mu}_c, \sigma_c^2 \mathbf{I})$, where $\boldsymbol{\mu}_c$ and $\sigma_c^2 \mathbf{I}$ are the mean class vector and the covariance matrix of c. Incorporating features allows ML models to be trained on synthetically generated graphs.

The Contextual Stochastic Block Model for Homophily/Heterophily (CSBM-H) (Luan, Hua, M. Xu, et al. 2024) is a variant of the original CSBM specifically designed to make

homophily a tunable parameter during graph generation. They explicitly parametrize the 2-class CSBM-H as CSBM-H $\left(\boldsymbol{\mu}_0, \boldsymbol{\mu}_1, \sigma_0^2\mathbf{I}, \sigma_1^2\mathbf{I}, d_0, d_1, h\right)$, for classes c_0 and c_1, where $\boldsymbol{\mu}_0,\ \boldsymbol{\mu}_1$ are the mean class vectors, $\sigma_0^2\mathbf{I},\ \sigma_1^2\mathbf{I}$ are the corresponding covariance matrices, $d_0,\ d_1$ are the degrees of the nodes in each class, and h defines how neighbors of the nodes in each class are sampled.

The h parameter helps define the binary *class-compatibility matrix* (a.k.a. neighborhood distribution matrix) $\mathbf{H} \in [0,1]^{2\times 2}$ (J. Zhu, Yan, et al. 2020), whose entries are defined as the fraction of the edges of nodes in class i that connect to nodes in class j (also interpreted as the probability that a neighbor of a node in class i belongs to class j):

$$H_{ij} = \frac{|\{\{u,v\} \in \mathcal{E} : y_u = i, y_v = j\}|}{|\{\{u,v\} \in \mathcal{E} : y_u = i\}|} \tag{2.22}$$

So, in the specific case of the binary CSBM-H:

$$\mathbf{H} = \begin{bmatrix} h & 1-h \\ 1-h & h \end{bmatrix}$$

Therefore, each row represents the probability distribution of the neighborhood of nodes in each class. d_0 and d_1 control how many neighbors are sampled for nodes on each class, so in expectation nodes in c_0 will have hd_0 neighbors in c_0 and $(1-h)d_0$ neighbors in c_1, while nodes in c_1 will have $(1-h)d_1$ neighbors in c_0 and hd_1 neighbors in c_1. Intuitively, larger diagonal values indicate a higher tendency of nodes of the same class to connect to each other, i.e. homophily. Furthermore, the class-conditioned node features are sampled from Gaussians as:

$$\begin{aligned} u \in c_0 : &\quad \mathbf{x}_u \mid c_0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \sigma_0^2\mathbf{I}) \\ v \in c_1 : &\quad \mathbf{x}_v \mid c_1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \sigma_1^2\mathbf{I}) \end{aligned}$$

The CSBM-H model was conceived with the specific goal of studying the effect of different levels of label homophily on node distinguishability and its influence in the expected performance of GNN models, and it can be also extended to the multi-class setting, as discussed in Chapter 4. Figure 2.6 shows a sample binary graph generated using this framework.

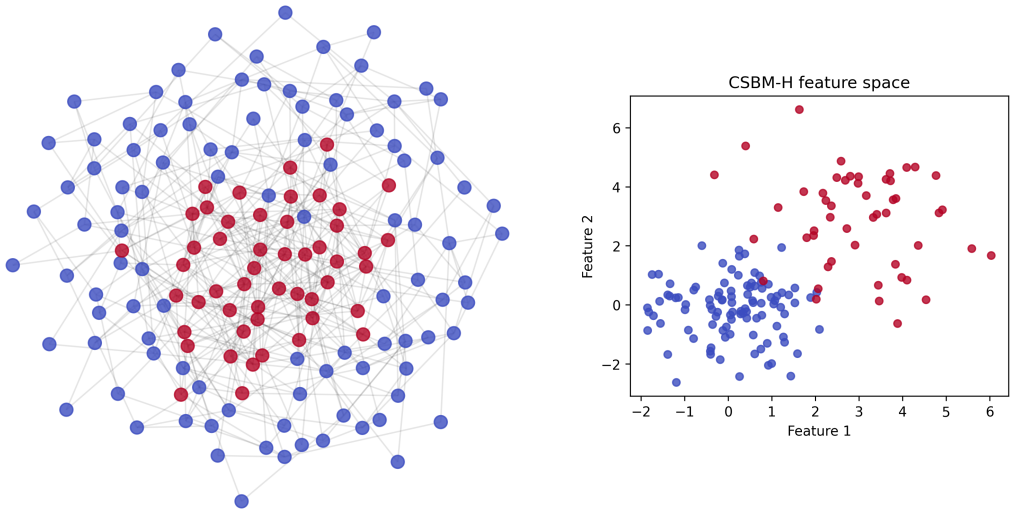


Figure 2.6.: A network generated through the CSBM-H model (node colors indicate class membership, where c_0 is the blue class) and its nodes' 2-dimensional feature space, with parameters $n_0 = 100$, $n_1 = 50$, $d_0 = 2$, $d_1 = 5$, $\boldsymbol{\mu}_0 = (0, 0)$, $\boldsymbol{\mu}_1 = (3, 3)$, $\sigma_0 = 1$, $\sigma_1 = 2$ and $h = 0.7$.

3. Related work

This chapter provides an overview of the work that has been done on understanding the scenarios when graph-aware models are expected to outperform graph-agnostic baselines, dealing with homophilic and heterophilic datasets, predicting GNN performance, and techniques for boosting model performance in the label scarcity regime. Lastly, we bridge the gap between the established literature and the goal of this work.

3.1. When do GNNs outperform graph-agnostic models?

The established assumption on which early GNN architectures relied is that the label distribution of the graph is locally smooth, i.e. nodes of the same type tend to connect to each other (Kipf and Welling 2017; William L. Hamilton, Ying, and Leskovec 2018a; Lim, Hohne, et al. 2021). Even before the Graph Convolutional Network was introduced, early models for semi-supervised node classification relied on regularization to enforce this structural smoothness assumption during learning (D. Zhou et al. 2003; Belkin, Niyogi, and Sindhwani 2006), where a penalty term is used in the loss function to encourage similarity in the embeddings of neighboring nodes. Works like H. Yang, K. Ma, and Cheng 2020 even combine popular GNN layers with (Propagation-)regularization to boost a models' base performance.

Early spectral graph filters shifted from graph regularization to encoding graph structure directly into the model through the graph's Laplacian (Bruna et al. 2013; Defferrard, Bresson, and Vandergheynst 2017) or adjacency matrix (Kipf and Welling 2017; F. Wu et al. 2019). The Graph Convolutional Network simplifies filters to a 1-hop message-passing function combined with a learnable parameter matrix and non-linearity, giving rise to modern GNN layers, where the homophily assumption is typically encoded in the aggregation function. This architectural property, which Luan, Hua, Q. Lu, et al. 2023 also called *edge bias*, proved to be highly effective in node classification tasks on relational datasets (Defferrard, Bresson, and Vandergheynst 2017; William L. Hamilton, Ying, and Leskovec 2018a; Veličković et al. 2018; Luan, Zhao, Chang, et al. 2019). In parallel, an increasing number of works highlighted that GNNs cannot outperform graph-agnostic baselines on every relational dataset (Luan 2023; J. Zhu, Yan, et al. 2020; Lim, Hohne, et al. 2021; Luan, Hua, M. Xu, et al. 2024), which was commonly linked to datasets being heterophilic, where nodes of different types tend to connect to each other.

However, newer works also show that the established homophily $\rightarrow$ good GNN performance, heterophily $\rightarrow$ bad GNN performance relationship is inconsistent. Y. Ma et al. 2023 prove that, under certain conditions, GCNs can also have strong performance on heterophilic datasets and link this performance with the consistency of the neighborhood

patterns of intra-class nodes, and posit that "good heterophily" actually exists. Wang et al. 2024 propose a Heterophilous Stochastic Block Model (HSBM) to model different heterophily patterns and use it to show that class separability is modulated by the distance between class neighborhood distributions after aggregation and the graph's average node degree, such that specific heterophily patterns can be manufactured, in which GCN performance is superior to MLP performance. Luan, Hua, M. Xu, et al. 2024 also link GNN performance to post-aggregation neighborhood distributions and establish that node distinguishability (ND) is optimal when the similarity of the neighborhood distributions of intra-class nodes is maximized, while the similarity of the neighborhood distributions of inter-class nodes is minimized, independently of the level of homophily of a dataset.

Therefore, the most recent literature focuses on the notion of node distinguishability and the factors that influence it rather than on homophily alone to frame the superior performance of GNNs over graph-agnostic baselines. Subedi et al. 2025 model GCN performance jointly as a function of feature quality and homophily and suggest that heuristics that rely on a single factor are not reliable in predicting GCN strength. Zheng, Luan, and Lihui Chen 2024 decompose homophily into label, feature, and structural homophily to isolate the effect of each factor on node distinguishability, and show through their CSBM-3H (Contextual Stochastic Block Model with three types of Homophily) synthetic graph framework that the Tri-Hom metric for quantifying node distinguishability, which accounts for the influence of each type of homophily, is more correlated with GNN performance than any other unidimensional metric. Finally, Luan, Hua, M. Xu, et al. 2024 compare the performance of GNN models with their graph-agnostic counterparts (as shown in Table 2.2) on several empirical datasets and conclude via hypothesis testing that the distance between intra-class node embeddings w.r.t. the inter-class distance is a better indicator of GNN performance than standard homophily metrics, hence ND is a strong predictor of the superiority of GNNs over MLPs.

3.2. GNNs for heterophily

Considering the poor performance that traditional GNN layers achieve on empirical heterophilic datasets (Luan, Zhao, Hua, et al. 2020; J. Zhu, Rossi, et al. 2021; J. Zhu, Yan, et al. 2020), a phenomenon commonly associated with message passing being a signal smoothing mechanism, which in practice often yields less distinguishable node embeddings when aggregating signals from nodes of different classes, several architectures have been designed to specifically work under heterophily while still leveraging the relational nature of these networks. Gong et al. 2024 provide a survey on graph learning on the heterophilic setting, including popular benchmarks, the most relevant tasks and most importantly, the most prominent paradigms for designing heterophilic GNNs.

One common approach to heterophilic GNNs is leveraging higher order neighborhoods during aggregation. MixHop (Abu-El-Haija et al. 2019) is a convolutional architecture where, in contrast to GCN, which is the first-order simplification of ChebNet (Defferrard, Bresson, and Vandergheynst 2016), the learned representations depend not only on

the immediate neighbors but on higher order neighborhoods via powers of the degree-normalized adjacency matrix $\hat{\mathbf{A}}$. H_2GCN (J. Zhu, Yan, et al. 2020) showed state-of-the-art performance on both homophilic and heterophilic datasets by enforcing three design choices in the architecture: separating neighbor aggregation from the ego node embedding, aggregating information from higher order neighborhoods and letting the output layer be a combination of intermediate node representations.

Another approach is exploring homophily beyond immediate neighborhoods, i.e. enriching the definition of a node's neighborhood by making aggregation work over a more informative subset of nodes. Geom-GCN (Pei et al. 2020) perform aggregation on the immediate neighbors of a node, plus a "structural neighborhood" consisting of those nodes whose distance in the latent space to the ego node is below some threshold, thus capturing information from potentially informative nodes. CPGNN (J. Zhu, Rossi, et al. 2021) learns an empirical compatibility matrix (as defined in Eq. (2.22)) which is in turn used to inform propagation, enabling a generalization of other GNN models to homophilic and heterophilic settings.

Beyond these approaches, Gong et al. 2024 mention Graph Transformers (Q. Wu et al. 2023), which enable selective, weighted aggregation over arbitrary nodes beyond the immediate neighborhood (in contrast to GAT (Veličković et al. 2018)), as well as Discriminative Message Passing (Bo et al. 2021), which enable message filtering via e.g. signed aggregation weights, to discriminate between informative and noisy messages, as alternative approaches to designing GNNs that generalize to both homophilic and heterophilic datasets.

3.3. Estimating GNN performance beyond homophily

As mentioned before, GNN performance has historically been estimated through a graph's level of label homophily, which has two main limitations: unidimensional metrics inevitably disregard other potentially informative attributes of the dataset, and secondly, several works have shown that the relationship between GNN performance and homophily is not necessarily monotonic, as was discussed in the previous section. For this reason, a number of metrics have been used to estimate GNN performance beyond the conventional definitions of homophily.

Homophily is usually a scalar, a single measure that summarizes some property of the network, so information at the local level can be lost, especially when the network is structurally heterogeneous (Suresh et al. 2021; Peel, Delvenne, and Lambiotte 2018). To address this limitation, Suresh et al. 2021 use assortativity at a node level instead of as a global metric and show that this metric is highly correlated with the predictive performance of a number of GNNs, plus, they propose a framework for rewiring the original graph to increase its assortativity by encoding both structural and proximity patterns. K. Rizi et al. 2025 model homophily at different group sizes within the original graph, and propose a generative model that can accommodate different local homophily patterns to study the effects of these patterns on information diffusion.

Beyond homophily, GNN performance has been analyzed from the perspective of how

informative its structural patterns are for node classification. Platonov, Kuznedelev, Babenko, et al. 2024 define the concept of Label Informativeness (LI), an information-theoretic measure that quantifies how informative a neighbor's label is about the ego node's label, and that is able to distinguish between informative and uninformative heterophilic patters, hence it shows a stronger correlation with GNN performance than traditional measures. Hussain et al. 2021 also propose to inform model selection through an information-theoretic correlation measure between a graph's community structure and ground-truth labels (as described in Section 2.4.2), which is particularly useful in homophilic graphs, where communities tend to align with ground-truth labels. Y. Ma et al. 2023 propose Cross-Class Neighborhood Similarity, (CCNS) which measures the similarity between the neighborhood distributions of nodes of each pair of classes, and argue that GNN performance is associated to whether the distributions of nodes in the same class are similar, which makes them aggregate similar neighborhoods, and whether the distributions of nodes in different classes are dissimilar, such that their embeddings end up being distinguishable after neighbor aggregation.

Following the notion of Y. Ma et al. 2023 of estimating performance based on the expected separability of classes, GNN suitability has been studied through the lens of the effect of message passing on the distinguishability of node representations. Wang et al. 2024 define pairwise class *separability gains* which are positive if a pair of classes is more distinguishable after aggregation, and negative otherwise, indicating whether GNN message passing will benefit classification over classification using a label-agnostic baseline, and link these separability gains to pre-aggregation neighborhood distributions and average node degrees. Luan, Hua, Q. Lu, et al. 2023 propose two metrics, Normalized Total Variation (NTV), which measures the change in a graph signal after applying a filter, normalized by the magnitude of the signal, (therefore, if NTV is low, aggregation will preserve label signal, else it distorts the signal), and Normalized Smoothness Value (NSV), a measure of the smoothness of signals, an indicator of homophily and therefore of the potential usefulness of graph filters. Luan, Hua, M. Xu, et al. 2024 also link GNN performance to post-aggregation neighborhood distributions and establish that node distinguishability (ND) is optimal (quantified via Probabilistic Bayes Error and Generalized Jeffreys Divergence) when the distance between intra-class node embeddings is lower than the distance between inter-class embeddings. Furthermore, they operationalize ND through a Classifier-Based Performance Metric (CPM), a statistical measure of the potential benefit of aggregating features, and highlight this metric can be used also in the scarce label setting.

Several of the previously discussed measures share the notion of class signal preservation after message passing. For instance, in Wang et al. 2024, the distance between neighborhood distributions determines the class-discriminative signal after aggregation, while degree moderates the variance of the aggregated embeddings, which decreases when the average node degree is high. Rubin, Loomba, and Jones 2025 study the limitations of message passing and propose a signal-to-noise ratio (SNR) to quantify the amount of relevant information w.r.t. noise that arrives to a node through message passing, which is bounded by so-called *class bottlenecks*, which are ultimately determined by the graph's topology.

3.4. The label scarcity regime

As discussed in 2.3, the task of maximizing the utility of the –typically scarce– labeled nodes to predict the missing labels predates GNNs. Early methods exploited graph structure to make informed predictions about the unlabeled data. The Label Propagation algorithm (LP) (X. Zhu and Ghahramani 2002) iteratively diffuses labels through the graph's edges using a random-walk-based transition matrix to update the soft class assignments of each node until these class assignments converge. Belief Propagation (BP) (Pearl 2022) also relies on diffusion and naturally works on Probabilistic Graphical models by iteratively updating each node's marginal distribution over classes ("belief") through messages encoding the likelihood of each class. The Iterative Classification Algorithm (Neville and Jensen n.d.) further extends these approaches by also incorporating node features, training two classifiers for label prediction, one relying only on the ego features for an initial label prediction on the unlabeled nodes, and the other relying on the features plus the neighboring nodes' class assignments.

Furthermore, techniques from SSL for label augmentation have been applied to graph learning. Q. Li, Han, and X.-M. Wu 2018 use co-training to complement GCN classification with the predictions from a random walk model, which can leverage information from the global graph structure, and self-training to expand the label set of a GCN based on its own most confident predictions. Z. Zhou et al. 2023 propose stabilized self-training to mitigate the effect of imbalanced and low-confidence pseudo-labels during training and use negative sampling regularization to boost accuracy on the GCN and DAGNN (Thost and J. Chen 2021). Y. Li, Yin, and Ling Chen 2022 use pseudo-labeling for enriching the training set by selecting representative nodes based on their informativeness about their neighborhood, given by a mutual information criterion.

Regularization-based approaches like the NormProp algorithm (Zhang et al. 2025) was designed to augment the supervision signal by leveraging the homophily assumption to enforce local consistency through an "Homophilous Regularization" loss. Alternatively, Active Learning approaches guide node selection for manual labeling based on the node features or the graph's structural attributes, e.g. through feature propagation and K-Medoid-based clustering (Y. Wu et al. 2021).

Finally, a number of works provide diagnostic studies of the performance of GNNs in the label scarce setting. Z. Zhou et al. 2023 emphasize the high variance of the predicted label distributions when training a GCN and a DAGNN on the Cora benchmark with 1 labeled node per class, meaning that the predicted labels depend heavily on the sampled training nodes, and show that pseudo-labeling can mitigate this negative effect. Q. Li, Han, and X.-M. Wu 2018 associate the performance degradation of a 2-layer GCN in the label-scarce setting to label signals not being able to propagate across the global structure of the graph, but also highlight the risk of over-smoothing in the case of deep GCNs, which makes the resulting node embeddings undistinguishable from each other. This smoothing nature of GCNs is the reason why they often thrive on empirical homophilic graphs. Subedi et al. 2025 associate GCN performance on the Amazon Computers dataset with the interaction between homophily and feature quality and conclude that the gain in performance (measured through the F1 score) w.r.t. the feature-only baselines

(Logistic Regression and Support Vector Machine) consistently increases as label scarcity becomes more pronounced.

3.5. Research gap

Most of the previous work in the label-scarce setting revolves around leveraging the available labels, features and graph topology to improve model performance (Section 3.4). Our approach doesn't focus on boosting the predictive power of GNNs, but rather on studying the effect of label scarcity on the performance of the GCN using its graph-agnostic counterpart, the MLP, as baseline. Ultimately, we aim to get a broader understanding of when GNNs outperform graph-agnostic models in different dataset-specific and supervision regime-specific conditions (Section 3.1).

We study the effect of label scarcity on a wide spectrum of homophily levels, both in synthetic and empirical datasets. Though a number of heterophily-specific layers have been designed (Section 3.2), we want to investigate if the canonical GCN can perform well in heterophilic datasets and under what circumstances. Furthermore, previous diagnostic studies of label scarcity have analyzed the performance of GNNs predominantly on empirical homophilic datasets. We use the CSBM-H framework to model different levels of homophily and control feature generation and graph topology in order to study the mechanisms that influence node distinguishability and validate the observations on several homophilic and heterophilic benchmark datasets.

Finally, as mentioned before, relying only on label-based metrics to estimate GNN performance has important limitations (Section 3.3): unidimensional metrics are underpowered since GNN performance is known to be influenced by a combination of factors, which is particularly evident in empirical networks, and full label availability is unfeasible in real-world scenarios. We aim to predict, with a certain degree of confidence, whether to use graph-aware or graph-agnostic models in a given dataset using a limited budget of labeled nodes (the same nodes that would be used to train ML models), plus the information encoded in the features and topology of the graph.

4. Approach

In this chapter we review the methodology used to test and evaluate each of the research questions by means of two independent pipelines: a synthetic dataset pipeline and an empirical dataset pipeline.

Use of AI-assisted tools. During the software development of this work, I used OpenAI's GPT-5.5 and Anthropic's Claude Opus 5.5 models, particularly for code refactoring, optimization and debugging. All suggestions were reviewed and I remain fully responsible for the entire content of this thesis.

4.1. Task definition

For the descriptive task (**RQ1**), the goal is to compare and characterize the performance of a representative graph-aware architecture (GCN) with that of its graph-agnostic counterpart (MLP). For this, it is necessary to evaluate the performance of the models on different dataset-specific and supervision regime-specific conditions. In this sense, we are given graphs $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ can be further split into exclusive subsets $\mathcal{V}_{\text{train}} \subset \mathcal{V}$, $\mathcal{V}_{\text{val}} \subset \mathcal{V}$, $\mathcal{V}_{\text{test}} \subset \mathcal{V}$, each of which defines an induced subgraph of the original dataset. For instance, $G[\mathcal{V}_{\text{train}}]$ is the subgraph defined by $\mathcal{V}_{\text{train}}$, including all those edges from G whose endpoints belong to the subgraph. We then train the models using different label rates (which define $|\mathcal{V}_{\text{train}}|$) and characterize the advantage/disadvantage of the GCN (gain) visually and quantitatively.

For the predictive task (**RQ2**), we test the extent to which several homophily metrics are able to predict the performance advantage of the GCN w.r.t. the MLP under equivalent architectural conditions and training procedure. For this, we define label-, feature- and structure-based proxies as follows:

- Label-based proxies, $\hat{h}_{\text{label}} = f(\mathbf{A}, \mathcal{Y}_{\text{train}})$, use $G[\mathcal{V}_{\text{train}}]$ as input to compute an estimate of the true homophily metric on the original graph (h_{label}).

- Feature-based proxies, $h_{\text{feature}} = f(\mathbf{A}, \mathbf{X})$, are label agnostic, so they can be computed directly on G without any sub-sampling. We assume that all nodes' features are readily available, unlike labels.

- Analogously to feature-based proxies, structure-based proxies, $h_{\text{structure}} = f(\mathbf{A})$, operate on the original graph and leverage its structural properties. Therefore, we assume that all nodes (and corresponding edges) are available to extract their topological attributes.

We distinguish between pre-aggregation (e.g. h_{node}, CCNS, h_{GE}), and post-aggregation (e.g. separability gains) metrics, and we only consider pre-aggregation metrics since the objective is to predict which model to select before performing any training on $\mathcal{V}_{\text{train}}$, $\mathcal{V}_{\text{val}}$, $\mathcal{V}_{\text{test}}$ based in the information available in the (sub)graph.

The experimental pipeline was constructed in Python and we used PyTorch and PyTorch Geometric (PyG) for the implementation and training of the neural models and for handling graph datasets. NetworkX was used for graph structural analysis and visualization, and the statistical analysis was conducted in R. The experiments were executed on a MacBook Air with Apple M2, 8 CPU cores and 16 GB RAM, and on an NVIDIA DGX Spark compute server with an NVIDIA GB10 GPU and 20 ARM CPU cores [1].

4.2. Pipeline description

The main roles of each pipeline are as follows. Through the synthetic pipeline, we generate datasets with controlled levels of homophily, controlled feature generation processes and controlled structural properties in order to study the mechanisms that affect the performance of the GCN in isolation, and we evaluate a number of homophily metrics as proxies to model the gain behavior under label scarcity. Through the empirical pipeline, we evaluate the validity of the findings made on the synthetic pipeline and the generalization power of the metrics tested to estimate the gain.

4.2.1. Synthetic pipeline

Figure 4.1 depicts the overall architecture for generating synthetic datasets, training graph-aware and graph-agnostic models on them and evaluating the performance of each model. The pipeline comprises the following modules:

1. **Input**. Initially, an unconnected CSBM-H graph is generated, where labels are assigned to nodes according to a predefined number of classes and nodes per class, and node features are sampled from class-conditional Gaussian distributions using user-defined mean vectors and covariance matrices. The CSBM-H graph is then wired according to the homophily parameter, h. Since labels and features were generated before making any connections, it is possible to rewire graphs to generate new instances with different h.

2. **Splits**. Stratified train/validation/test splits are sampled from the generated graph, where the number of nodes in the training set is controlled by a pipeline parameter, lr[2].

[1] The code for downloading the datasets, reproducing the experiments and generating the figures in this work are publicly available in the following GitHub repository: `https://github.com/LuisVasquez123/understanding-homophily-gnn`.

[2] Within the scope of this work, lr corresponds to label rate and not learning rate, as is usually the case in the field.

3. **Training**. The splits are used to train the GCN (transductively) and MLP, which have the same underlying architecture (number of layers, hidden dimensionality) and share training hyperparameters. The accuracy score from each model is used to compute the gain for the specific (h, lr) instance.

4. **Homophily Estimators**. On each run, a new instance can be generated, and the subgraph induced by the training nodes is used to compute label-based proxies for gain estimation. Feature- and structure-based proxies are computed using the whole graph without the labels.

5. **Evaluation**. After generating and training a number of (h, lr) instances, the results are aggregated and evaluated in terms of the performance of the models on different dataset-specific and supervision regime-specific conditions, the correlation of the homophily estimations with the resulting gains, and the deviation from the full-label homophily estimation in the case of the label-based metrics.

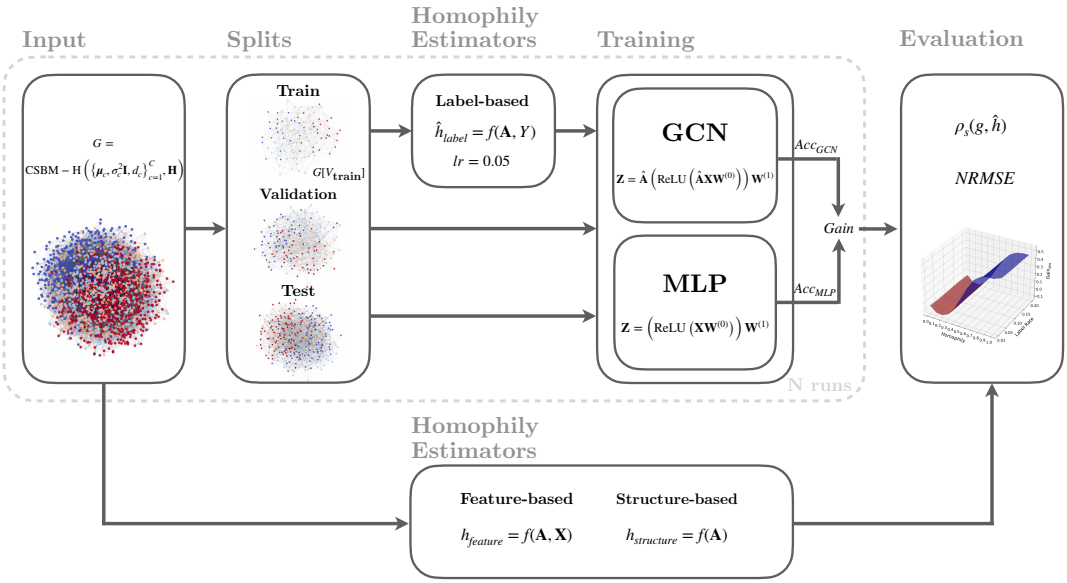


Figure 4.1.: Synthetic pipeline, which uses CSBM-H graphs as inputs and evaluates homophily on a predefined supervision regime.

4.2.2. Empirical pipeline

Figure 4.2 depicts the overall architecture for training graph-aware and graph-agnostic models on empirical datasets at different label rates and evaluating the performance of each model. As there is no graph generation process, the empirical pipeline is a simplified version of the synthetic case:

1. **Input**. Each of the selected datasets represents a graph generated from a real-world network, so the labels, features and graph connections are given, therefore homophily is a fixed, graph-specific property.

2. **Splits**. Although some datasets provide public train/validation/test masks, we sample new splits using custom random/stratified split generators, where the number of training nodes is controlled by the parameter lr. The training split, which represents an induced subgraph of the original dataset, is stored, as it is used to compute label-based estimators of performance.

3. **Training**. The splits are used to train the GCN (transductively) and MLP, which have the same underlying architecture and share training hyperparameters. The accuracy score from each model is used to compute the gain for the dataset under the current supervision regime.

4. **Homophily Estimators**. For efficiency, and to align with a real-life setting, homophily estimation is coupled with the splits used for training. The subgraph induced by the training nodes is used to compute label-based proxies for gain estimation. Feature- and structure-based proxies are computed using the entire graph without the labels.

5. **Evaluation**. After generating and training a number of instances per dataset, each of which has a new randomly sampled train/validation/test split for the current lr, the results are aggregated and evaluated by assessing the correlation of the homophily estimations with the resulting gains, and the deviation from the full-label homophily estimation in the case of the label-based metrics.

4.3. Synthetic graph generation

As described in Section 2.4.3, (Luan, Hua, M. Xu, et al. 2024) defined the Contextual Stochastic Block Model for Homophily (CSBM-H) to make homophily a user input when generating graphs. They specifically parametrize the 2-class CSBM-H, and we extend the model to generate multiclass graphs. For this purpose, it is convenient to define the class-compatibility matrix $\mathbf{H} \in [0,1]^{C\times C}$ in the multiclass setting, building from Eq. (2.22):

$$H_{ij} = \begin{cases} h, & i = j, \\ \dfrac{1-h}{C-1}, & i \neq j. \end{cases}$$

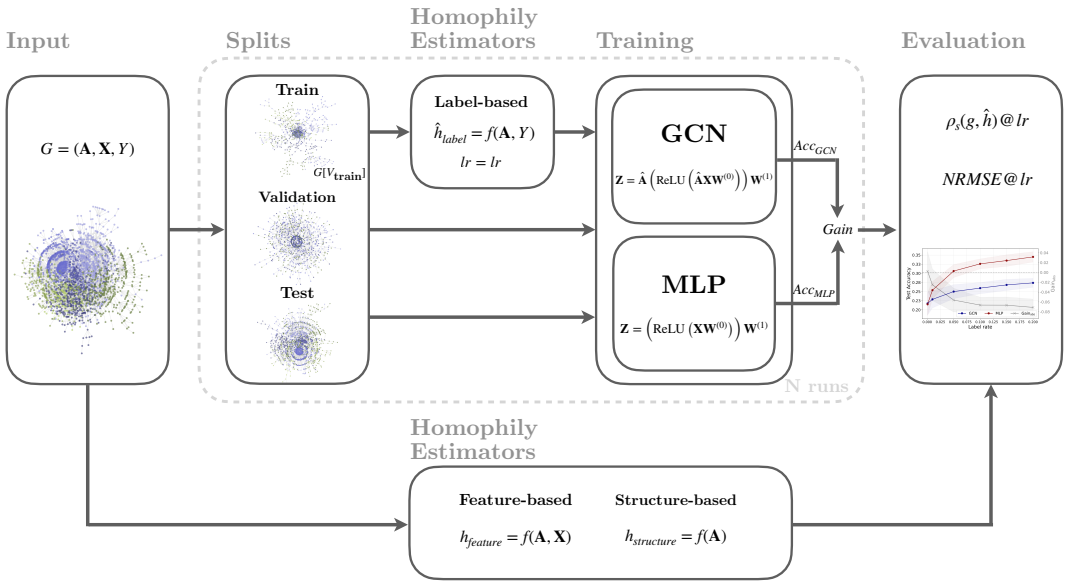


Figure 4.2.: Empirical pipeline, which uses real-world graphs as inputs and evaluates homophily on different label rates per dataset.

where, as in the 2-class setup, the diagonal entries H_{ii} describe the intra-class connectivity, while the off-diagonal entries H_{ij} describe the inter-class connectivity. In matrix form:

$$\mathbf{H} = \begin{bmatrix} h & \frac{1-h}{C-1} & \cdots & \frac{1-h}{C-1} \\ \frac{1-h}{C-1} & h & \cdots & \frac{1-h}{C-1} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1-h}{C-1} & \frac{1-h}{C-1} & \cdots & h \end{bmatrix} \tag{4.1}$$

So, a multiclass CSBM-H is parametrized as CSBM-H $\left(\left\{\boldsymbol{\mu}_c, \sigma_c^2\mathbf{I}, d_c\right\}_{c=1}^{C}, \mathbf{H}\right)$, where the level of homophily is encoded in $\mathbf{H}$. We can observe that the diagonal elements correspond by definition to Eq. (2.10), although due to the stochastic nature of edge sampling, empirically h_{edge} and each H_{ii} might slightly differ. It is also worth noting that the implementation balances the distribution of the heterophilic edges, i.e. each neighbor has an equal probability of $\frac{1-h}{C-1}$ of belonging to each of the $C-1$ foreign classes. Additionally, to prevent the empirical class distributions from getting distorted during the neighbor sampling process because of class and degree imbalances, for the experimental phase we standardize the number of nodes and their degrees across classes, $n_1 = n_2 = \cdots = n_C = n$ and $d_1 = d_2 = \cdots = d_C = d$.

Furthermore, we define the class mean vectors as $\boldsymbol{\mu}_c = a \cdot \mathbf{e}_c \in \mathbb{R}^C$, where $\mathbf{e}_c$ is the c-th basis vector of $\mathbb{R}^C$ and C is the number of classes, a convenience that makes the class means orthogonal basis vectors scaled by a. The user-defined factor a allows us to control feature separability and guarantees that class distances are uniform across each pair independently of the number of classes: $\|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\| = \sqrt{2}a$ for all $i \neq j$. Also, in the experiments we set $\sigma_1^2 = \sigma_2^2 = \cdots = \sigma_C^2 = \sigma^2$. So, the class-conditioned node features are sampled from Gaussians as:

$$\mathbf{x}_v \mid y_v = c \sim \mathcal{N}\left(\boldsymbol{\mu}_c, \sigma_c^2 \mathbf{I}\right) \tag{4.2}$$

Throughout the experiments in the synthetic pipeline, we use the parameters defined in Table 4.1 independently of the number of classes, unless otherwise stated.

Table 4.1.: Parameters for CSBM-H graph generation.

Parameter	Value
n	500
d	5
h	$[0, 0.1, \ldots, 0.9, 1]$
a	1
$\boldsymbol{\mu}_c$	$\mathbf{e}_c$
σ	1

4.4. Empirical datasets

For the empirical evaluation, we use 15 benchmark datasets for node classification loaded from the PyTorch Geometric framework, some of which were specifically designed to serve as challenging benchmarks in the heterophilic setting:

- The Planetoid datasets (Cora, CiteSeer, Pubmed) (Z. Yang, Cohen, and Salakhutdinov 2016) come from citation networks, where nodes represent publications and edges are citations between them.
- WebKB (Pei et al. 2020) contains datasets from university websites, where nodes are pages and edges are hyperlinks between them.
- The WikipediaNetwork datasets (Rozemberczki, Allen, and Sarkar 2021) come from Wikipedia articles on the topics of chameleons and squirrels, where nodes are specific articles and edges are hyperlinks between them.
- Similarly to the WikipediaNetwork datasets, WikiCS (Mernyei and Cangea 2022) is based on Wikipedia articles on the topic of computer science.
- The Actor dataset (Pei et al. 2020) is also Wikipedia-based, but it is a co-occurrence network where nodes represent actors and links between them represent the actors being mentioned in the same article.

- The HeterophilousGraphDataset (Platonov, Kuznedelev, Diskin, et al. 2024) family contains a variety of heterophilic empirical datasets coming from different websites, where nodes typically denote users/entities and edges represent some relationship (e.g. purchasing the same items, answering a question), plus the Minesweeper synthetic dataset, which is inspired in the namesake game, where the nodes are cells in the grid, the edges denote neighboring cells, and the goal is to predict which nodes contain mines.

A summary of the empirical datasets is provided in 4.2.

Table 4.2.: Empirical benchmark datasets summary.

Source	**Dataset**	**Type**
Planetoid	Cora	Citation network
	CiteSeer	Citation network
	PubMed	Citation network
WebKB	Cornell	Webpage hyperlink network
	Texas	Webpage hyperlink network
	Wisconsin	Webpage hyperlink network
WikipediaNetwork	Chameleon	Wikipedia hyperlink network
	Squirrel	Wikipedia hyperlink network
WikiCS	WikiCS	Wikipedia hyperlink network
Actor	Actor	Actor co-occurrence network
HeterophilousGraphDataset	Roman-Empire	Wikipedia article network
	Amazon-ratings	Amazon co-purchasing network
	Minesweeper	Synthetic grid network
	Tolokers	Crowdsourcings network
	Questions	Q&A interaction network

For the split generation process, we rely on multiple seeds to sample new splits per run following either a random or stratified approach. On the early days of GNNs, experimental pipelines stuck to a single fixed public split for training and evaluation. Later, Shchur et al. 2019 pointed out that training on a single split could be misleading and that changes in the training procedure and the hyperparameters could significantly change how models rank. So, it was proposed to resample the nodes to generate multiple train/validation/test splits and train models with multiple random initializations. With this robustness improvement, results across architectures would be more fairly comparable. However, that also implied that pipelines still relied on a single supervision regime, i.e. a fixed number if nodes per split, such as the well-known 20 labeled nodes per class split for Cora. One of the main objectives of this work is to evaluate different supervision regimes beyond the standard proportion of training nodes, as is described in Section 4.6. This way, we test the robustness of the models when the number of visible nodes is increasingly scarce and we are able to characterize the performance of the each model under different training conditions.

For preprocessing, after loading a dataset, reverse edges are added, such that for every $\{u, v\}$ in the graph's edge index, $\{v, u\}$ is also present, an assumption that the GCN formulation makes for message passing, else the model would enforce message passing following the direction specified in the dataset's edge index. Furthermore any self loops $\{u, u\}$ are removed, since the GCN architecture adds them automatically during training.

4.4.1. Exploratory Data Analysis

The selected datasets are structurally heterogeneous, as shown in Table 4.3. Node counts range from 183 in Cornell and Texas to 48,921 in Questions, edges range from 554 in Cornell to 1,038,000 in Tolokers. While some datasets possess a large number of classes, others like Minesweeper, Tolokers and Questions only have 2, and, as can be seen in Figure 4.3, classes might be highly imbalanced. In particular, we notice that in the Questions dataset, 97.9% of the nodes belong to a single class, which leads to highly biased homophily values for the measures not correcting for class imbalances. Furthermore, the feature dimensionality ranges from 7 in the case of the Minesweeper synthetic dataset to 3,703 for CiteSeer. Some graphs are more densely connected, like Tolokers, while the majority are rather sparse, a common condition on empirical networks.

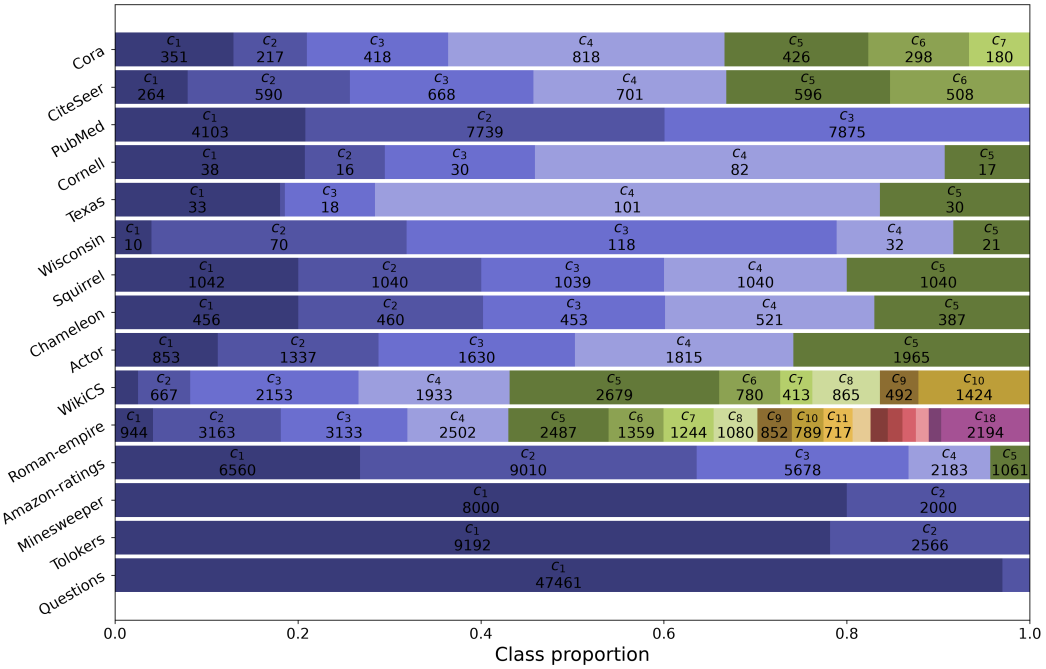


Figure 4.3.: Class distributions of the empirical datasets.

Importantly, some datasets are graphs with a single connected component, while Cora, CiteSeer and WikiCS are not, which has an impact in message passing, since isolated nodes have no neighbors to aggregate information from. Class imbalances also affect the sampling process in some cases by capping the proportion of nodes that can be placed in

the training split at extreme label scarcity. For instance, for small datasets like Cornell and Texas, label rates of 5% and 1% are both realized as 1 node per class.

Table 4.3.: Empirical dataset characterization.

Dataset	# Nodes	# Edges	# Classes	Feature dim.	Avg. degree
Cora	2,708	10,556	7	1,433	3.90
CiteSeer	3,327	9,104	6	3,703	2.74
PubMed	19,717	88,648	3	500	4.50
Cornell	183	554	5	1,703	3.03
Texas	183	558	5	1,703	3.05
Wisconsin	251	900	5	1,703	3.59
Chameleon	2,277	62,742	5	2,325	27.55
Squirrel	5,201	396,706	5	2,089	76.27
WikiCS	11,701	431,206	10	300	36.85
Actor	7,600	53,318	5	932	7.02
Roman-Empire	22,662	65,854	18	300	2.91
Amazon-Ratings	24,492	186,100	5	300	7.60
Minesweeper	10,000	78,804	2	7	7.88
Tolokers	11,758	1,038,000	2	10	88.28
Questions	48,921	307,080	2	301	6.28

Additionally, it is possible to get an intuition of the homophily patterns of each dataset through their empirical class compatibility matrices. By removing the normalization from Eq. (2.22), we get the matrix of raw counts of the connections between each pair of classes, $\mathbf{H}^{(\text{count})} \in \mathbb{N}^{C\times C}$, defined as:

$$H_{ij}^{(\text{count})} = |\{\{u, v\} \in \mathcal{E} : y_u = i, y_v = j\}| \tag{4.3}$$

For instance, for a highly homophilic dataset, PubMed ($h_{\text{edge}} = 0.8024$), the empirical normalized (Eq. (2.22)) and unnormalized (Eq. (4.3)) class compatibility matrices are:

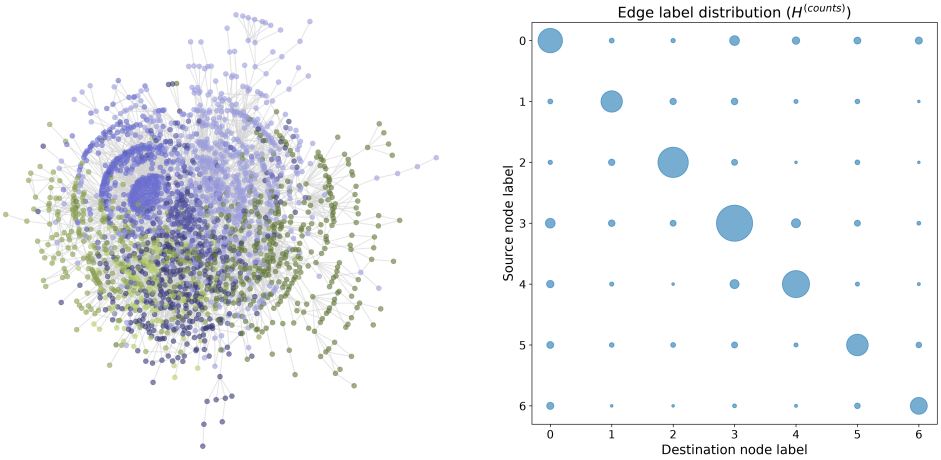
$\mathbf{H} =$

	c_1	c_2	c_3
c_1	0.66	0.11	0.24
c_2	0.05	0.85	0.10
c_3	0.10	0.09	0.82

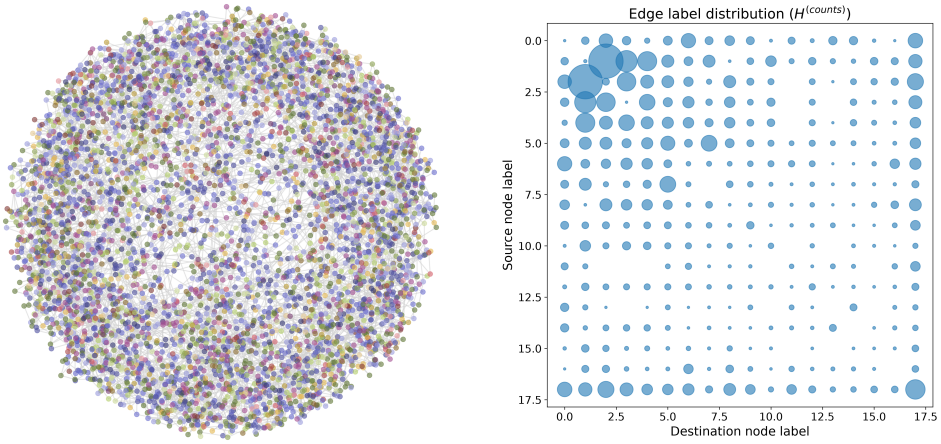
$\mathbf{H}^{(\text{count})} =$

	c_1	c_2	c_3
c_1	$10,424$	$1,670$	$3,724$
c_2	$1,670$	$29,126$	$3,365$
c_3	$3,724$	$3,365$	$31,580$

Clearly, a high number of edges join nodes from the same classes, and the classes are not balanced, highlighting the need for class and degree correction in homophily measures. Still, in this case, standard uncorrected measures give a reasonably accurate overview of its homophilic nature. Figure 4.4 further visualizes one representative homophilic (Cora) and one representative heterophilic dataset (Roman-Empire), plus their class-wise bubble plots, a visual representation of their unnormalized class compatibility matrices. Cora, which has 7 classes, is highly homophilic ($h_{\text{edge}} = 0.81$) and thus strongly dominant on the diagonal entries of the bubble plot, while Roman-Empire, which has 18 classes, shows more connections across foreign classes (off-diagonal), it is therefore highly heterophilic ($h_{\text{edge}} = 0.0469$).



(a) Cora.



(b) Roman-Empire.

Figure 4.4.: Network structure and edge distribution bubble plots for Cora and Roman-Empire. Bubble sizes represent the number of edges between pairs of classes. Roman-Empire's graph visualization is based on a sub-sampling of 5,000 nodes.

4.5. Proxies in the label-scarce setting

For the descriptive task, we use the metrics listed in Table 4.4 as predictors of GCN gain, whose general definitions are those listed in Section 2.4.2. They operate on undirected graphs without self-loops.

As mentioned, label-based metrics normally operate on a graph with full label visibil-

Table 4.4.: Homophily metrics used in the predictive task.

Label-based	**Feature-based**	**Structure-based**
Node homophily	Generalized Edge Homophily	Degree Assortativity
Edge homophily	Feature Assortativity	LCC Assortativity
Class homophily	Feature Assortativity	Neighborhood Similarity
Adjusted homophily	Local Similarity (cosine)	
Unbiased homophily	Local Similarity (Euclidean)	
2-hop homophily		
Relative Mutual Information		

ity. Our approach to computing these measures is two-fold. For a given graph G, we first compute the ground-truth value of the metric using all of its labels (h_{label}), and then we use the visible nodes to generate a label induced subgraph $G[\mathcal{V}_{\text{train}}]$ and the metrics are computed on this subgraph ($\hat{h}_{\text{label}}$). This yields an estimation under label scarcity of the metric, which can be compared with the ground-truth. In general, the metrics can be directly computed using Eq. (2.12), Eq. (2.10), Eq. (2.13), Eq. (2.14), Eq. (2.15) by treating $G[\mathcal{V}_{\text{train}}]$ as input.

In the case of 2-hop homophily (Eq. (2.16)), the full graph structure is used to find 2-hop neighbors, but only nodes with visible labels are considered for the computation of homophily, i.e. 1-hop neighbors with no visible labels are still used to find 2-hop neighbors. In the case of RMI (Eq. (2.17)), community detection is realized through the Louvain algorithm on the whole graph in an unsupervised fashion. Then, mutual information and entropy are calculated on the visible nodes.

In the case of feature-based (h_{feature}) and structure-based ($h_{\text{structure}}$) metrics, since labels are not used for the computation, the obtained values can be considered ground-truth values independently of label visibility. Also, they operate on the entire edge index, and feature-based metrics have access to all nodes' feature vectors. Additionally, since h_{NS} is operationalized through the proposition that the similarity between the neighborhood distributions of intra-class nodes can be a good predictor of GNN performance, it can be considered a label-structure hybrid.

4.6. Training

You, Ying, and Leskovec 2021 provide a framework to systematically explore design choices for GNNs. The model design space consists on the intra-layer design (e.g. batch normalization, dropout rate, activation function, aggregation operator), inter-layer design (e.g. skip connections, pre/post-processing layers, message passing layers), and learning configuration (hyperparameters including batch size, learning rate, optimizer and training epochs). For our setup, we keep the architectural and optimization choices mostly fixed across models, as the independent variables of interest are homophily and label rate. Also, we aggregate over several runs to analyze the variability of the results,

which becomes relevant especially at low label rates. We follow the task formulation of Kipf and Welling 2017, i.e. semi-supervised node classification, where all nodes are available during the forward pass and the loss is computed on the labeled nodes.

As mentioned before, the approach for resampling the same number/proportion of nodes on the training set leaves open the question whether a specific supervision regime is adequate. Thus, we make the level of supervision both for homophily estimation and for model training a tunable parameter. The number of training nodes is controlled by the parameter $lr \in \{0.20, 0.15, 0.10, 0.05, 0.01, \text{min}\}$, where min is the smallest possible label rate, representing 1 node per class, and applies exclusively to the empirical datasets. On the other hand, the validation and test splits are also randomly sampled on each run and fixed to 10% and 20%, respectively, where nodes sampled on each split are mutually exclusive[3].

4.6.1. Models

Architecturally, we follow the base design described in Kipf and Welling 2017. This consists of 2 GCN layers, with ReLU activation on the intermediate layer, hidden dimensionality of 16 and dropout rate of 0.5. The implementation of the convolutional layers, automatically adds self-loops to the graph's edge index. Therefore, for an input graph G with N nodes, E edges and C classes, our GCN instantiation is:

$$\begin{aligned}\mathbf{H}^{(0)} &= \mathbf{X} \\ \mathbf{H}^{(1)} &= \text{ReLU}(\hat{\mathbf{A}}\mathbf{H}^{(0)}\mathbf{W}^{(0)}) \\ \mathbf{Z} &= \hat{\mathbf{A}}\mathbf{H}^{(1)}\mathbf{W}^{(1)}\end{aligned}$$

where $\mathbf{X} \in \mathbb{R}^{N \times d_0}$ is the feature matrix and d_0 is the graph's feature dimensionality given either by the synthetic graph generator or the attributes of an empirical graph, $\mathbf{H}^{(1)} \in \mathbb{R}^{N \times 16}$ is the intermediate node representation, $\hat{\mathbf{A}} \in \mathbb{R}^{N \times N}$ is the degree-normalized adjacency matrix with self loops, $\mathbf{Z} \in \mathbb{R}^{N \times C}$ are the output logits, $\mathbf{W}^{(0)} \in \mathbb{R}^{d_0 \times 16}$ and $\mathbf{W}^{(1)} \in \mathbb{R}^{16 \times C}$ are the learnable parameter matrices.

We design the MLP architecture to be the direct graph-agnostic equivalent of the GCN, as shown in Table 2.2, including activation, hidden dimensionality and dropout. Therefore, the 1-layer setting is equivalent to multinomial Logistic Regression. By keeping these parameters consistent across architectures, we can attribute the resulting performance gap to the architectural difference between the models, i.e. the presence/absence of a neighbor aggregation function.

Furthermore, the training protocol for the MLP is the same as the GCN. For each run, the weights of each model are independently initialized, and we use full batch training.

[3] Works like Shchur et al. 2019 use splits like (20 per class)/(30 per class)/rest, others like Kipf and Welling 2017 use (20 per class)/500/1000, we follow a fixed proportions on the validation/test sets approach, similar to J. Zhu, Yan, et al. 2020 and Platonov, Kuznedelev, Diskin, et al. 2024, in order to make different supervision regimes comparable and independent of the size of the validation/test sets.

We use the standard cross entropy loss on the output logits (log-softmax internally computed), computed as a mean exclusively over the labeled nodes, per Eq. (4.4):

$$\mathcal{L} = -\frac{1}{|\mathcal{V}_{\text{train}}|} \sum_{v \in \mathcal{V}_{\text{train}}} \log \hat{Y}_{v,y_v} \tag{4.4}$$

4.6.2. Hyperparameters

For each model, training is performed for as many as 200 epochs with early stopping, and model selection is based on the validation loss. We use Adam (Kingma and Ba 2014) as optimizer, which implements L2 regularization according to the weight decay factor. No grid search is performed on either pipeline to analyze the effect of homophily and label rate on model performance and to isolate these effects from hyperparameter tuning, which would hinder a fair comparison of the models. A summary of the hyperparameters used in our pipelines is provided in Table 4.5

Table 4.5.: Training hyperparameters used for both models.

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.01
Weight decay	5×10^{-4}
Maximum epochs	200
Early stopping	Yes
Early stopping patience	50 epochs

4.7. Gain characterization

Once the models have been trained on a specific split of the input graph (either synthetic or empirical) we aim to quantify whose performance is superior given the current dataset-specific and supervision regime-specific conditions. Per the standard approach (Shchur et al. 2019; Z. Yang, Cohen, and Salakhutdinov 2016; Q. Li, Han, and X.-M. Wu 2018; Kipf and Welling 2017), we use test set accuracy as the main performance metric, defined as:

$$\text{Acc} = \frac{|\{v \in \mathcal{V}_{\text{test}} : \hat{y}_v = y_v\}|}{|\mathcal{V}_{\text{test}}|}$$

Then, we define the notion of *gain* as the relationship between the accuracies of the models on a given run, i.e. the gap between GCN and MLP performance. This notion has already been implicitly used to compare GNNs with graph-agnostic architectures in works like Y. Ma et al. 2023 and J. Zhu, Yan, et al. 2020. Different measures are proposed, which help interpret the relationship in different ways. We define the absolute gain, $\text{Gain}_{\text{abs}} \in [-1, 1]$, as the –signed– difference between model accuracies:

$$\text{Gain}_{\text{abs}} = \text{Acc}_{\text{GCN}} - \text{Acc}_{\text{MLP}} \tag{4.5}$$

Intuitively, Gain_{abs} is positive when GCN outperforms MLP, negative when MLP outperforms GCN, and 0 when their accuracies are equal. Relative gain, $\text{Gain}_{\text{rel}} \in [-100\%, \infty]$, is the normalized version of the absolute gain when considering the MLP accuracy as baseline. It can be interpreted as the expected improvement (in %) we observe when using GCN w.r.t. using MLP:

$$\text{Gain}_{\text{rel}} = \frac{\text{Acc}_{\text{GCN}} - \text{Acc}_{\text{MLP}}}{\text{Acc}_{\text{MLP}}} \times 100\%$$

Finally, we define the gain ratio as the proportion of GCN accuracy w.r.t. MLP accuracy, with $\text{Ratio} \in [0, \infty]$. It is therefore easily interpretable as GCN accuracy as a multiple of MLP accuracy:

$$\text{Ratio} = \frac{\text{Acc}_{\text{GCN}}}{\text{Acc}_{\text{MLP}}}$$

Clearly, it holds that if $\text{Ratio} > 1$, then GCN outperforms MLP and $\text{Ratio} = 1$ indicates equal performance. The ratio is an alternative yet equivalent view of the relative gain, and we use it because of its interpretability. Also, the relative gain and the ratio are undefined at $\text{Acc}_{\text{MLP}} = 0$ and they are not bounded on the positive side, so their values can theoretically explode, conditions that in practice we seldom observe.

To visualize the gain, on the synthetic pipeline, since homophily is a tunable parameter, we characterize the GCN/MLP gap as a function of the graph's homophily and the label rate used for training, such that we can observe the relationship between the independent (h, lr) and dependent (gain) variables as a surface:

$$\text{Gain} = f(h, lr)$$

On the empirical pipeline, since homophily is fixed for every dataset, after model training, we visualize the effect of the supervision regime on the performance (GCN and MLP accuracy) and gain behavior on the datasets for increasing levels of label scarcity:

$$\text{Gain} = f(lr)$$

4.8. Evaluation

On the main pipelines, runs across each (h, lr) (synthetic datasets) and (dataset, lr) (empirical datasets) configuration are aggregated and we report mean and standard deviation of the accuracies of each model and the gains. Furthermore, depending on the specific subject of study, we evaluate the experimental results according to the following guidelines.

4.8.1. Ground-truth homophily

Ground-truth homophily values (h_{label}) are computed using the original graphs $G = (\mathcal{V}, \mathcal{E})$, while estimates ($\hat{h}_{\text{label}}$) are computed on subgraphs induced by the visible node set $G[\mathcal{V}_{\text{train}}]$. Feature- and structure-based estimators (h_{feature}, $h_{\text{structure}}$) are considered

ground-truth values, as they are computed on G independently of whether nodes are labeled or not. As will be shown in the next chapter, we use h_{unb} as the base measure to classify empirical datasets into the homophilic, neutral and heterophilic regimes, given the desirable properties it satisfies, as mentioned in Section 2.4.2. On the synthetic dataset, reference homophily is considered to be the h input parameter in the CSBM-H model.

For the descriptive task on the empirical datasets, we compute pairwise correlation coefficients (Pearson correlation with Bonferroni correction) across every pair of metrics h_{label} at full label visibility to understand the relationship between the metrics and visualize if redundancies exist. Such values are then used to compare estimations $\hat{h}_{\text{label}}$, computed at specific label rates, in the evaluation phase after training.

4.8.2. Node distinguishability

On the synthetic pipeline, when analyzing the effect of GCN neighbor aggregation (Section 5.1.1), we define a simple signal-to-noise metric to quantify the separability gains after applying GCN layers. Luan, Hua, M. Xu, et al. 2024 evaluate node distinguishability through the Negative Generalized Jeffreys Divergence, a measure of the difference between two probability distributions equivalent to the symmetrized KL divergence. This metric consists of an Expected Negative Normalized Distance (ENND) measuring the distance between classes, normalized by their variances, and a Negative Variance Ratio (NVR) term, which accounts for the difference in variance across classes. In our setup, since we keep the class variances equal by construction, the NVR term before aggregation collapses to 0 and the expression simplifies to:

$$D_{\text{NGJ}} = \text{ENND} = -\frac{1}{2}\left(\frac{\|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|}{\sigma}\right)^2 \tag{4.6}$$

Then, the classes are highly distinguishable (separable) when D_{NGJ} is more negative. Clearly, separability is ultimately a function of the distance between class means, normalized by the variance, which can be interpreted as a signal-to-noise ratio, where the signal modulates inter-class separation, while noise controls intra-class variability:

$$D_{\text{NGJ}} = -\frac{1}{2}\,(\text{SNR})^2$$

So, in practice, we can quantify node distinguishability on the feature vectors of two classes through an empirical signal-to-noise ratio (which is simply a re-parametrization of Eq. (4.6)) computed on the mean class vectors and their pooled standard deviation, which assumes class variances are approximately the same:

$$\text{SNR}(\mathbf{X}) = \frac{\|\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_2\|}{\hat{\sigma}} \tag{4.7}$$

where $\hat{\boldsymbol{\mu}}_c = \frac{1}{n_c}\sum_{i \in C_c} \mathbf{x}_i$, $\hat{\sigma}^2 = \frac{\hat{\sigma}_1^2 + \hat{\sigma}_2^2}{2}$ is the pooled variance when $n_1 = n_2$, and $\hat{\sigma}_c^2$ are the empirical class variance estimates. We evaluate this SNR in the original features as well as the low-pass filtered features to measure how distinguishable nodes are after one and two rounds of aggregation under different homophily conditions.

4.8.3. Effect of homophily and supervision regime

Additionally to the gain characterization procedure, a statistical test is performed on the synthetic pipeline to investigate whether the GCN/MLP performance gap is affected by each of the independent variables, homophily and label rate, and whether their interaction also plays a role in modulating the output. Since every graph is used across all (h, lr) combinations (a total of 11 homophily levels and 5 label rates in the experimental pipeline), and since we also model the interaction between the independent variables, we use a Two-Way Repeated Measures ANOVA (Analysis of Variance) test. The formula of our model in Wilkinson–Rogers notation is:

$$gain \sim h * lr + (1|subject) \tag{4.8}$$

where *gain* is the dependent variable, $h * lr$ denotes that we analyze the main effects of each of the independent variables plus their interaction $(h + lr + h : lr)$, and $(1|subject)$ is the random intercept term for *subject* (a.k.a. grouping factor), i.e. we account for the variability across runs instead of estimating individual intercepts.

Furthermore, in the scenario where the interaction term in Eq. (4.8) turns out to be significant, signaling that the variables jointly and not independently affect the gain, we use a post-hoc test to further investigate, for every level of homophily, whether the effect of the label rate is statistically significant. Since we only evaluate one independent variable (lr), we use a single-effects model, a One-Way Repeated Measures ANOVA for lr separately on every homophily level. The formula of the model in Wilkinson–Rogers notation is:

$$gain \sim lr + (1|subject) \qquad \text{for each} \quad h = h_i$$

where *gain* is the dependent variable, lr is the main effect of the independent variable, and $(1|subject)$ is the random intercept term for *subject*, which serves the same purpose as the grouping factor in Eq. (4.8).

4.8.4. Gain estimation

For the predictive task, we are interested in assessing the extent to which each homophily metric is able to model the gain behavior. We mainly evaluate the alignment of a metric's estimations with the resulting gain, both under full label visibility and under label scarcity conditions, through the Spearman rank correlation $\rho_s \in [-1, 1]$ between the two variables of interest, $\rho_s(\text{Gain}, h)$, and equivalently for estimations under label scarcity, $\rho_s(\text{Gain}, \hat{h})$. We use this measure on the one hand because it is scale-unaware and rank-based, in contrast to value-based measures like Pearson correlation, therefore it primarily measures whether the ordering of the samples is preserved independently of their specific values. On the other hand, it is able to model non-linear, monotonic relationships, which is relevant given that the specific association between the variables of interest is not assumed to be linear.

Additionally, we evaluate the adequateness of an estimator under label scarcity by calculating the overall deviation (error) w.r.t. the ground-truth values through the

Normalized Root Mean Squared Error (NRMSE). For a single instance for which we estimate homophily using a label-based metric h, we define the error at such sample as:

$$e_i = \frac{|h_i - \hat{h}_i|}{h_{max} - h_{min}}$$

where the denominator is used to normalize the error for measures that live in the $[-1, 1]$ interval (h_{adj}, h_{unb}), making results comparable. By aggregating over all instances, we get the NRMSE, a measure of the overall quality of the estimator under label scarcity:

$$\text{NRMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2} \tag{4.9}$$

In the synthetic pipeline, we take the NRMSE by aggregating over every independently generated graph (micro-averaged), whereas in the empirical pipeline, given some label rate, we compute dataset-specific errors, and then estimate global NRMSE $@lr$ as the mean over those errors (macro-averaged).

5. Experiments and Results

This chapter outlines the experiments performed to tackle each of the research questions by means of the synthetic and empirical dataset pipelines. Each pipeline aims to partially answer each research question under different dataset-specific and supervision regime-specific conditions and their roles are complementary. As such, Section 5.1.1, Section 5.1.2, Section 5.2.1 and Section 5.2.2 primarily tackle **RQ1** (descriptive task), while Section 5.1.3 and Section 5.2.3 mainly focus on answering **RQ2** (predictive task). Finally, we present the most relevant findings in each pipeline.

5.1. Synthetic datasets

5.1.1. GCN feature aggregation

We start by investigating the effect of raw feature aggregation on small 2-class CSBM-H graphs with different levels of homophily. Initially, features for each class are generated per Eq (4.2), with equidistant class means $\boldsymbol{\mu}_1 = [2, 0]$, $\boldsymbol{\mu}_2 = [0, 2]$, i.e. $\boldsymbol{\mu}_c = 2\mathbf{e}_c$, and $\sigma_1^2 = \sigma_2^2 = 1$, such that the expected distance between class means is $\|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\| = 2\sqrt{2} \approx 2.828$. Neighbors are stochastically sampled, with $n_1 = n_2 = 10$ and $d_1 = d_2 = 3$, and we use homophilic ($h = 0.9$), neutral ($h = 0.5$) and heterophilic ($h = 0.1$) graphs for comparison.

We perform 2 rounds of feature aggregation according to the propagation rule from Kipf and Welling 2017, i.e. using the degree-normalized adjacency matrix with self loops $\hat{\mathbf{A}}$, which yields the update rule $\mathbf{X}^{(k)} = \hat{\mathbf{A}}^k\mathbf{X}$. We also discard the nonlinearity, such that we can isolate the raw effect of the aggregation operator, an approach already proposed in F. Wu et al. 2019, which highlights that the biggest benefit of the convolution comes from aggregation and not from the layer-wise nonlinearities. Therefore, after one and two rounds of low-pass filtering, respectively:

$$\mathbf{X}^{(1)} = \hat{\mathbf{A}}\mathbf{X}, \qquad \mathbf{X}^{(2)} = \hat{\mathbf{A}}^2\mathbf{X}$$

We evaluate the Signal-to-Noise Ratio (SNR) before and after aggregation, as defined in Eq. (4.7) by generating a small CSBM-H instance, sampling its labels and class-dependent features, and using this unlabeled graph to wire the nodes according to the corresponding edge index. We repeat this process for 100 runs and report average and standard deviation values, as shown in Table 5.1. We observe that the SNR monotonically increases on the homophilic and heterophilic regimes. Therefore, the effect of neighbor aggregation in these regimes is beneficial for node distinguishability, as it yields increasingly separable classes.

This phenomenon directly challenges the belief that GNNs invariably fail under heterophily, and can be intuitively explained as follows. Given a node from class c_1, it will have approximately $hd_1 = (0.1)(3) = 0.3$ neighbors in c_1 and $(1-h)d_1 = (1-0.1)(3) = 2.7$ neighbors in c_2, therefore, one round of feature aggregation will produce embeddings that are similar to those of the nodes in c_2. Analogously, after one round of feature aggregation, nodes in c_2, which will have mostly neighbors from c_1, will get embeddings similar to those of the opposite class, so nodes "switch" identities and remain distinguishable. Since the SNR measures the absolute distance between empirical class centers and not their position, it captures distinguishability independently of the actual location of the classes. After another round of aggregation, nodes swap positions again and reinforce the separability gain.

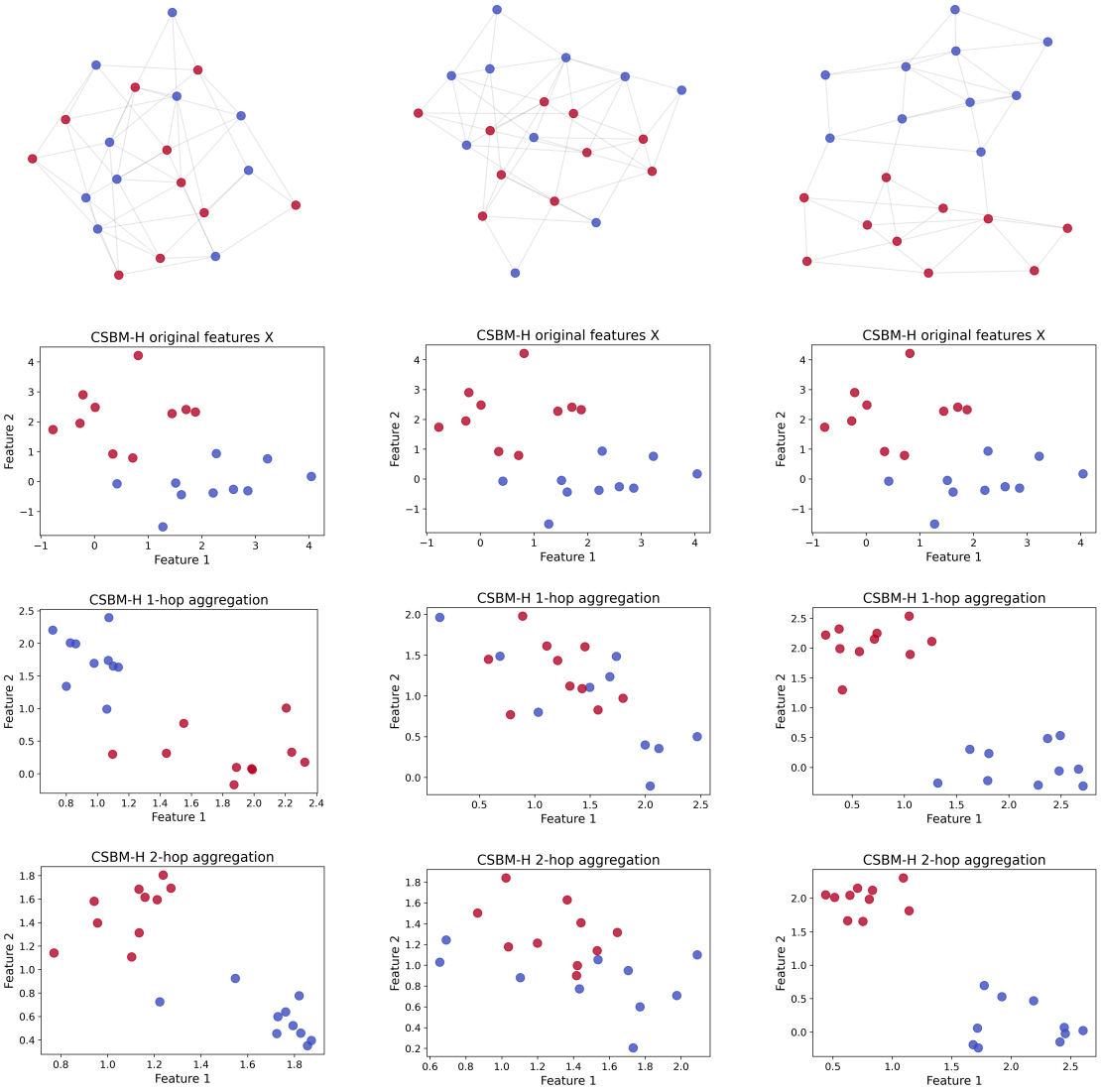
Table 5.1.: SNR on the original and aggregated features for different homophily regimes.

h	SNR($\mathbf{X}$)	SNR($\mathbf{X}^{(1)}$)	SNR($\mathbf{X}^{(2)}$)
0.1	2.920 ± 0.531	3.097 ± 0.771	3.636 ± 0.962
0.5	2.920 ± 0.531	1.083 ± 0.589	1.635 ± 0.533
0.9	2.920 ± 0.531	5.820 ± 1.339	7.590 ± 2.029

Under homophily, nodes tend to have neighbors from the same class, so embeddings tend to be smoothed, leading to more compact and separable classes, which explains the significant gain in distinguishability. This aligns with literature stating that homophily is beneficial for GCN-style feature averaging (Kipf and Welling 2017; Y. Ma et al. 2023; Platonov, Kuznedelev, Diskin, et al. 2024).

Furthermore, this reasoning explains why on the neutral regime, GCN aggregation hurts separability, as nodes tend to have the same number of neighbors on every class, so aggregation mixes signals from different sources and effectively pulls the class means towards each other. Most of the mixing is done after the first aggregation round, explaining the sharp drop in separability, while the increase in the SNR after an additional round is explained by the constant noise reduction due to smoothing after every round, with little change in the distance between the already mixed signals. This U-shape behavior of node distinguishability after low-pass filtering (high under homophily and heterophily and low in the neutral regime) is what Luan, Hua, M. Xu, et al. 2024 call the "mid-homophily pitfall" in the 2-class CSBM-H model.

The effect of the GCN feature aggregation function is visualized in Figure 5.1 on a single CSBM-H instance, which starts with the same underlying features and labels, and is rewired to achieve the corresponding levels of homophily. From Figure 5.1a, we observe that the graph convolution has the effect of switching node identities: after aggregation, the location of the features is swapped, such that $\mathbf{x}_i$ for $i \in c_1$ (blue class) tend to move towards $\boldsymbol{\mu}_2 = [0, 2]$, and $\mathbf{x}_j$ for $j \in c_2$ (red class) tend to move towards $\boldsymbol{\mu}_1 = [2, 0]$. After another round, the nodes remain distinguishable and tend to move towards their original cluster centers. Meanwhile, the reduction in variance leads to more compact clusters after each round.



(a) $h = 0.1$. (b) $h = 0.5$. (c) $h = 0.9$.

Figure 5.1.: A representative instance of the 2-class CSBM-H model, rewired to achieve different levels of homophily, and feature distributions of the original features $\mathbf{X}$, the 1-hop aggregated features $\mathbf{X}^{(1)}$, and the 2-hop aggregated features $\mathbf{X}^{(2)}$.

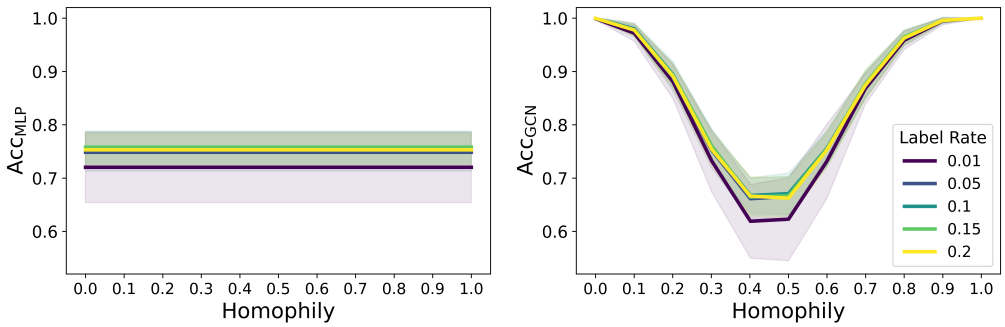
It is noteworthy that this behavior is specific to this heterophily patterns modeled in the CSBM-H framework, with highly consistent and distinctive neighbor distributions across classes, as characterized in Wang et al. 2024. This implies that heterophily is not sufficient for GCN aggregation to increase node distinguishability, as is usually observed in empirical datasets, where neighbor aggregation usually leads to less separable classes. In the 2-class CSBM-H model, neighborhood distributions are highly consistent on each class, real-world heterophilic networks rarely have uniform neighborhood

patters, so aggregation mixes signals from different sources, introducing noise on intra-class node embeddings. This also shows that global homophily measures are inherently underpowered, as they don't show how local homophily patterns behave.

Under homophily (Figure 5.1c), features are smoothed on each round. Signal, which modulates inter-class separation, increases, while noise, which controls intra-class variability, decreases, leading to a steady increase in the SNR. Continuous feature aggregation leads to the over-smoothing phenomenon (Q. Li, Han, and X.-M. Wu 2018), which in general is harmful for performance. For our 2-layer setting, this effect starts to be visible after 2 steps of aggregation, but does not harm separability. In the neutral regime (Figure 5.1b), it is clear that signal mixing hurts node distinguishability, clusters which were initially roughly separable, get mixed together, hinting that graph-agnostic models might outperform GNNs in this setting, as will be analyzed afterwards.

5.1.2. Training on CSBM-H graphs

The synthetic pipeline aims to analyze the effect of homophily, a tunable dataset-specific variable, and label rate, a supervision regime-specific variable, on the performance gap between GCN and MLP. We generate 2-class CSBM-H graphs, train both models on all different combinations of the independent variables, and track the gain to characterize its behavior. The specific parameters used in the generative process are those described in Table 4.1, and a total of 100 instances per (h, lr) combination are generated. We use label rates $lr \in \{0.20, 0.15, 0.10, 0.05, 0.01\}$, with fixed validation and test set proportions.



(a) MLP. (b) GCN.

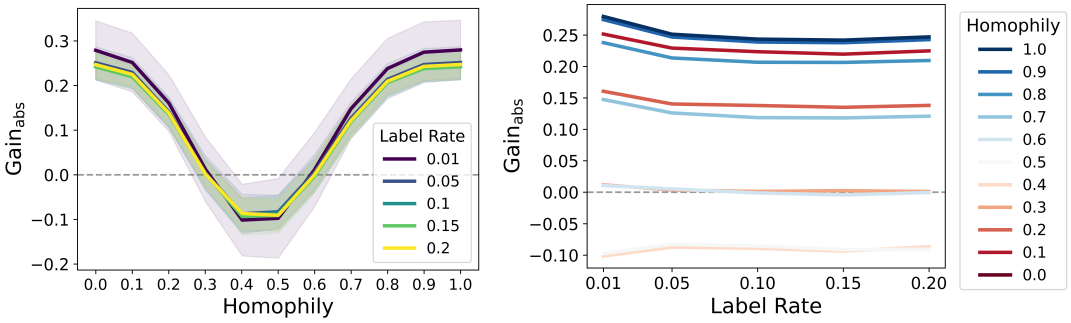
Figure 5.2.: MLP and GCN test set accuracy along all homophily values, with values averaged over runs and ±1 standard deviation bands. Coloring indicates different label rates for both plots.

Figure 5.2 shows the effect of homophily over the entire spectrum $h \in \{0, 0.1, \ldots, 0.9, 1\}$ on each model separately when trained on using different label rates. Clearly, homophily has no effect on the performance of the MLP, hence its accuracy is constant w.r.t. h, as visualized in Figure 5.2a. Furthermore, we see that Acc_{MLP} is almost constant when trained using increasingly scarce labels, while for the lowest label rate $lr = 0.01$, which corresponds to $|\mathcal{V}_{\text{train}}| = 10$, or 5 labeled nodes per class, accuracy is significantly im-

pacted by the low supervision, with mean and standard deviation 0.720 ± 0.067, indicating high variability on the label-scarce regime.

On the other hand, as seen in the previous experiment, in the 2-class CSBM-H model, intermediate homophily levels are expected to have a negative effect on the performance of GCN due to signal mixing. Conversely, for high homophily and high heterophily levels, consistent neighborhood patterns lead to good classification accuracies. This effect is confirmed in Figure 5.2b, where we observe the highest accuracy on $h = 0$ and $h = 1$. As h approaches the extremes, the expected GCN accuracy becomes less variable and closer to perfect classification performance with little influence of the label rate, showing the model is able to capture and leverage homophilic/heterophilic patterns even under very low supervision. Variability is therefore lowest on the extremes and higher as we move towards the middle, being highest under the neutral regime at extreme label scarcity, with accuracy consistently reaching values well below the MLP baseline.

Separately, Figure 5.2a and Figure 5.2b give an intuition of the regimes on which it is more appropriate to use a GCN and on which aggregation is harmful and a graph-agnostic model is preferable. We use the absolute gain definition in Eq. (4.5) to quantify the performance gap between GCN and MLP as it is bounded, thus comparable across different (h, lr) configurations. Evidently, there exist transition regimes (heterophily-neutral and neutral-homophily) on which it is unclear which model leads to a better classification performance, and the gain gives a clean threshold to understand where each model becomes superior, with $\text{Gain}_{\text{abs}} = 0$ representing the boundary. As seen in Figure 5.3a, these transition values occur at $h \approx 0.3$ and $h \approx 0.6$, therefore in the neutral regime ($0.4 \leq h \leq 0.5$), the gain is expected to be negative (MLP has superior performance), while beyond this interval, both in the homophily and heterophily direction, the gain is consistently expected to be positive, independently of the supervision regime. Additionally, the point of least class distinguishability is $h \approx 0.5$ (approximately, since GCN aggregation uses self-loops, unlike the aggregation method used in the original paper). We also observe that the variability of the gain is nonzero on the extremes, which is expected due to the noise introduced by the MLP measurements, especially at low label rates.



(a) Absolute gain across homophily levels.

(b) Absolute gain across label rates.

Figure 5.3.: Gain characterization under different homophily and label rate levels.

Importantly, Figure 5.3a shows that the supervision regime does not play a major role in determining the gain behavior. In other words, aside from the transition regimes, for graphs with a certain degree of homophily, training with increased label scarcity does not change the decision of which model to employ, although for $lr = 0.01$ the actual value of the difference in performance can significantly fluctuate. Figure 5.3b shows the mean gain values across the label rate spectrum, which makes the previous claim more evident: for a given homophily label, the gain is mostly consistent for every label rate. In practice, this implies that even under label scarcity, we are able to predict with confidence which model will have an advantage, especially as we move towards the more extreme homophily/heterophily regimes, represented with the darkest hues.

Intuitively, it is possible to join the previously analyzed perspectives, i.e. the "homophily view" and the "supervision regime view", into a surface defined by the two independent variables, where the absolute gain values resulting from each combination represent the regions where we should select each of the models, as well as the transition region, defined by the plane $\text{Gain}_{\text{abs}} = 0$. In Figure 5.4, the blue region is the positive gain region ($\text{Acc}_{\text{GCN}} > \text{Acc}_{\text{MLP}}$), while the red region represents the negative gain region ($\text{Acc}_{\text{MLP}} > \text{Acc}_{\text{GCN}}$) along the entire (h, lr) spectrum. This visualization makes the U-shaped behavior of the gain clear, and shows how in the neutral regime, GCN aggregation reduces node distinguishability and thus hurts performance, and it also shows how the preference for a model is primarily determined by homophily.

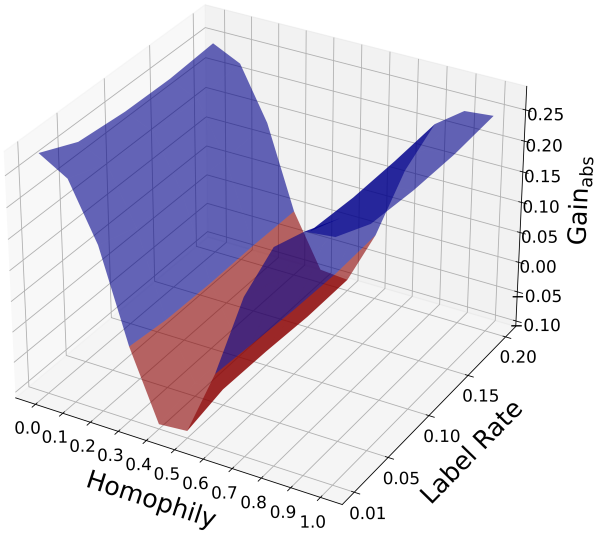


Figure 5.4.: Absolute gain as a function of h and lr for 2-class CSBM-H graphs.

Statistical analysis

In connection with the descriptive task, we aim to investigate if the effect of homophily and of label rate on the gain are statistically significant, and more importantly, whether they jointly or independently influence the obtained gains. We use a Two-Way Repeated Measures ANOVA to quantify the amount of the gain's variability that is explained by each factor, including their interaction. The test design is *repeated measures* as each of the 100 runs (subjects) contributes one gain value to each (h, lr) combination, so we account for observations being correlated, and *two-way* because we want to model the interaction term, i.e. whether the effect of label rate on the gain depends on the level of homophily the specific graph has. So, we fit the following model:

$$y_{s,h,lr} = \mu + \alpha_h + \beta_{lr} + (\alpha\beta)_{h,lr} + \pi_s + \epsilon_{s,h,lr} \tag{5.1}$$

where:

$y_{s,h,lr}$: observed gain for subject s at (h, lr).
μ : overall mean.
α_h : main effect of homophily at level h.
β_{lr} : main effect of label rate at level lr.
$(\alpha\beta)_{h,lr}$: interaction effect of (h, lr).
π_s : subject random effect.
$\epsilon_{s,h,lr}$: residual error.

This partitions the observed gain into an overall mean shared across all observations, a main effect of homophily and a main effect of label rate (fixed effects), an interaction term between the fixed effects, a random intercept that accounts for the inherent differences across subjects, and a residual accounting for everything not explained by the model. For our model with 11 homophily values, 5 label rates and 100 runs, the solver is approximating 1 global mean, 10 homophily coefficients, 4 label rate coefficients, 40 interaction coefficients, a random intercept for each subject, assumed to come from a single distribution with a shared variance $\mathcal{N}(0, \sigma_s^2)$, and a residual value per observation. Therefore, the model defines the following null-hypotheses:

$$H_0^{(1)} : \quad \alpha_{0.0} = \alpha_{0.1} = \cdots = \alpha_{1.0} = 0$$
$$H_0^{(2)} : \quad \beta_{0.2} = \beta_{0.15} = \cdots = \beta_{0.01} = 0$$
$$H_0^{(3)} : \quad (\alpha\beta)_{h,lr} = 0 \qquad \forall\, lr, h$$

The first null-hypothesis states that there is no effect of homophily on the gain, i.e. the gain does not differ across homophily levels. The second null-hypothesis states that there is no effect of the supervision regime on the gain, i.e. the gain is expected to be the same independently of the supervision regime. The third null-hypothesis states that each fixed effect acts independently on the gain, in other words, how much the label rate impacts the gain does not depend on how homophilic the graph is in the first place, and vice-versa.

Furthermore, the test relies on the following assumptions: there are no extreme outliers within each (h, lr) cell, the dependent variable is approximately normally distributed within each (h, lr) cell (tested using the Shapiro-Wilk test and Q-Q plots), and sphericity, i.e. the variance of the differences between repeated measurements is approximately the same across all pairs of levels for each factor (h and lr), which is handled by the model implementation via Greenhouse-Geisser correction.

Table 5.2.: Two-Way Repeated Measures ANOVA test results.

Effect	DF_n	DF_d	F	p	η^2_G
h	4.73	468.58	9168.03	<0.001***	0.899
lr	2.12	209.42	5.89	0.003**	0.030
$h : lr$	9.29	919.88	9.25	<0.001***	0.021

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

We use a custom script in R to test the assumptions, perform the Two-Way Repeated Measures ANOVA, and we obtain the results shown in Table 5.2, where DF_n and DF_d represent the degrees of freedom of the numerator and denominator after Greenhouse-Geisser correction, $F = \frac{MS_{\text{effect}}}{MS_{\text{error}}}$ is the test statistic, p is the corresponding p-value, and η^2_G is the generalized eta-squared value, which measures the size of each effect[1]. We observe, based on the p-values, a statistically significant ($p \leq 0.001$) effect of homophily, a significant effect of label rate and a significant interaction term. An η^2_G of 0.899 for h indicates that an overwhelming proportion of the variability in the gain can be attributed to homophily. Therefore, homophily is a very dominant factor in comparison to the rest of the effects, so we confirm the previous observation that graph structure dominates over label rate in determining the gain.

The effects of lr ($\eta^2_G = 0.030$) and the interaction term $h : lr$ ($\eta^2_G = 0.021$) are comparatively much smaller, so they are secondary in determining the gain: if we are able to reliably estimate how homophilic a CSBM-H graph is, we can say that the performance gap between the models will be primarily determined by this structural property. The smaller effect of label rate also has important practical implications: labeling effort can potentially be reduced since we expect homophily to be the main driver of gain. The interaction term is also small but statistically significant, meaning that the effect of label rate on the gain is not the same across all homophily levels, indicating that at some levels, the number of available labels actually modulates the performance gap. From Figure 5.3a, we can visually validate this claim, for some homophily levels, especially in the extremes, there is a difference between the gains obtained when training at different label rates, which motivates the following follow-up tests.

Since there is a statistically significant interaction between the fixed effects, we perform a post-hoc test to determine for which homophily values we actually care about the number of labeled nodes. The test corresponds to a One-way Repeated Measures ANOVA for lr, performed on each h, whose goal is to compare the lr means within each

[1]We also perform a sensitivity analysis to validate the robustness of the results in presence of some mild outliers in the data.

level of homophily to determine for which kind of graphs the label rate actually matters and for which the gain is expected to be the same independently of the number of labels. The single-effects model for every homophily level is defined as:

$$\begin{aligned} h_{0.0}: &\quad y_{s,lr} = \mu^{(h_{0.0})} + \beta_{lr} + \pi_s + \epsilon_{s,lr} \\ h_{0.1}: &\quad y_{s,lr} = \mu^{(h_{0.1})} + \beta_{lr} + \pi_s + \epsilon_{s,lr} \\ &\quad \vdots \\ h_{1.0}: &\quad y_{s,lr} = \mu^{(h_{1.0})} + \beta_{lr} + \pi_s + \epsilon_{s,lr} \end{aligned}$$

where:

$$\begin{aligned} y_{s,lr}: &\quad \text{observed gain for subject } s \text{ at } lr. \\ \mu^{(h_i)}: &\quad \text{overall mean for homophily level } h_i. \\ \beta_{lr}: &\quad \text{main effect of label rate at level } lr. \\ \pi_s: &\quad \text{subject random effect.} \\ \epsilon_{s,lr}: &\quad \text{residual error.} \end{aligned}$$

Similarly to Eq. (5.1), the model partitions the observed gain at a specific homophily level into a main effect of label rate, a random intercept that accounts for the inherent differences across subjects, and a residual accounting for everything not explained by the model. Each test is performed independently for a specific h_i and has an independent null-hypothesis:

$$\begin{aligned} H_0^{(h_{0.0})}: &\quad \beta_{0.2} = \beta_{0.15} = \cdots = \beta_{0.01} \\ H_0^{(h_{0.1})}: &\quad \beta_{0.2} = \beta_{0.15} = \cdots = \beta_{0.01} \\ &\quad \vdots \\ H_0^{(h_{1.0})}: &\quad \beta_{0.2} = \beta_{0.15} = \cdots = \beta_{0.01} \end{aligned}$$

Each null-hypothesis states that there is no effect of label rate on the gain at a given homophily level, i.e. the gain is expected to be the same for h_i independently of the supervision regime. As we observed in the Two-way Repeated Measures ANOVA test, this is not the case in at least one homophily level. The results shown in Table 5.3 make this claim explicit: we see statistically significant effects ($p \leq 0.001$) both at $h \leq 0.2$ and at $h \geq 0.7$. In this regimes, we reject the null hypothesis that the effect of label rate is the same at every level.

Therefore, label rate modulates the advantage of GCN over MLP in the homophilic and heterophilic regimes. This is most evident at $h = 0.0$ and $h = 1.0$ in Figure 5.3a, where $lr = 0.01$ appears to yield a slightly higher gain, i.e. a bigger GCN advantage. Conversely, on intermediate homophily levels, a change in the supervision regime is not expected to affect the performance gap. However, it should be noted that across all homophily levels, the variance of the expected gain is highest when training at extreme label rates. So, we can characterize this as a supervision-variability trade-off: training at more scarce label rates potentially boosts the advantage of a model, at the cost of making the estimation less predictable. Still, under high homophily and high heterophily, this

variability mainly comes from the MLP performance degrading at extreme label rates, GCN performance is still expected to stay consistent even when trained at $lr = 0.01$, as can be verified in Figure 5.2b.

Table 5.3.: One-Way Repeated Measures ANOVA results.

h	DF_n	DF_d	F	p	η^2_G
0.0	2.28	225.75	14.65	<0.001***	0.101
0.1	2.17	215.02	10.19	<0.001***	0.075
0.2	2.46	243.05	7.09	<0.001***	0.053
0.3	2.40	237.50	1.18	0.313	0.008
0.4	2.35	233.14	1.60	0.201	0.012
0.5	2.06	204.16	1.17	0.315	0.009
0.6	2.05	202.99	1.49	0.227	0.011
0.7	2.52	249.42	8.23	<0.001***	0.060
0.8	2.38	235.48	10.10	<0.001***	0.075
0.9	2.25	223.14	14.16	<0.001***	0.098
1.0	2.27	224.76	15.01	<0.001***	0.103

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

Multiclass CSBM-H

The previous pipeline on 2-class CSBM-H graphs gave us meaningful insights about the role that homophily and label rate play in the performance gap between the models. In the 2-class setting, we showed that graph structure is the main driver of GCN performance, and label rate mostly affects the variability of the models. To get closer to a real-world scenario while still controlling the variables of interest, we extend the analysis to the multiclass setting.

We mostly replicate the experimental pipeline of the 2-class setting, including the graph generation process and the training procedure, as described in Section 4.3. For a C-class CSBM-H model, parametrized as CSBM-H $\left(\left\{\boldsymbol{\mu}_c, \sigma_c^2\mathbf{I}, d_c\right\}_{c=1}^{C}, \mathbf{H}\right)$, with $n_1 = n_2 = \cdots = n_C = 500$ and $d_1 = d_2 = \cdots = d_C = 5$, class connectivity is defined by $\mathbf{H} \in [0,1]^{C\times C}$, which contains on its diagonal values the user-specified parameter h, and on the off-diagonal the inter-class connectivity. For instance, for a 4-class graph with $h = 0.4$, neighbor sampling is performed according to the following distributions, from Eq. (4.1):

$$\mathbf{H} = \begin{bmatrix} 0.4 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.4 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.4 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.4 \end{bmatrix}$$

So, in this setting, for a node in c_1, around 40% of the neighbors are expected to be intra-class nodes, and 60% are expected to be inter-class nodes with equal probability of belonging to each of the foreign classes.

Moreover, in all cases, we control class separability through separability parameter $a = 1$, let class means be orthogonal to each other, and sample features from Gaussian distributions according to Eq. (4.2), with $\boldsymbol{\mu}_c = \mathbf{e}_c \in \mathbb{R}^C$ and $\sigma_1^2 = \sigma_2^2 = \cdots = \sigma_C^2 = 1$. As in the previous setting, for each C we perform a sweep across every $h \in \{0.0, 0.1, \ldots, 0.9, 1.0\}$ and $lr \in \{0.20, 0.15, 0.10, 0.05, 0.01\}$, with a total of 100 instances per (h, lr). Since lr is defined as a proportion, the number of training nodes per class for a specific label rate is the same in every C.

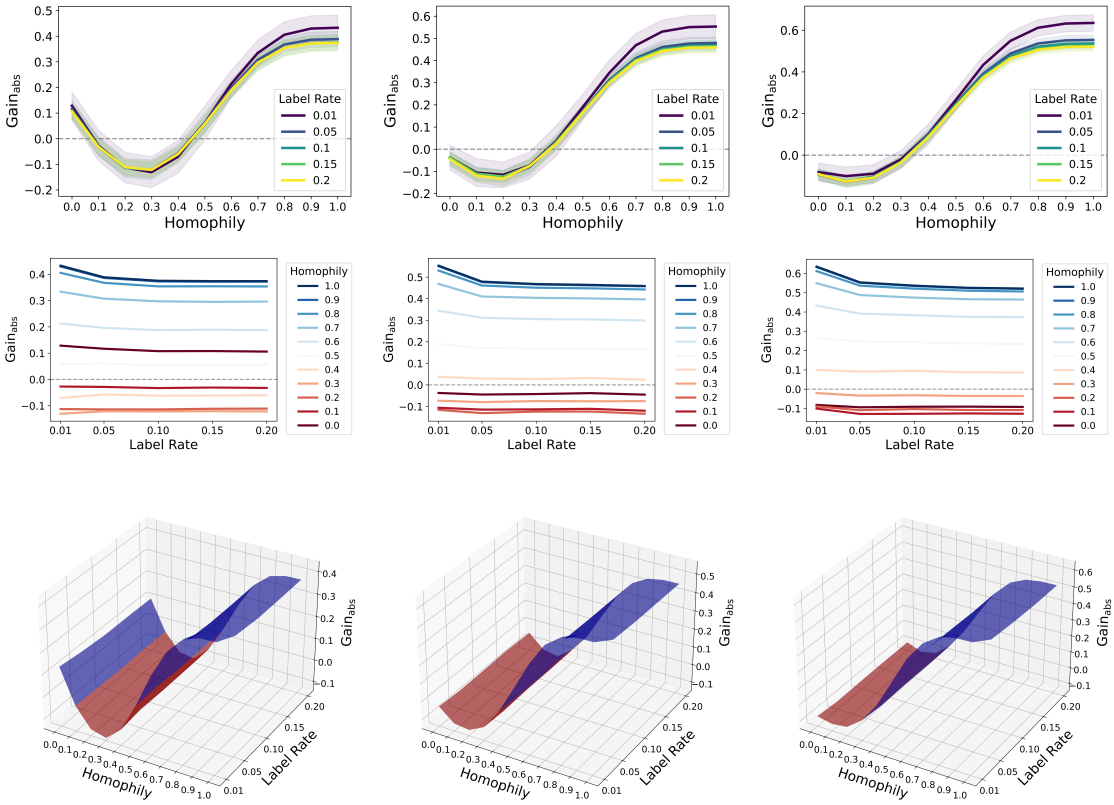
Figure 5.5 shows the homophily view and the supervision regime view as well as the surface defined by the two views for each C. The top row shows a clear pattern: in the multiclass setting, gain is also primarily determined by homophily, and their relationship is not symmetric anymore. On each case, there exists a point of least distinguishability where the gain is maximally negative, and it corresponds to $h \approx \frac{1}{C}$ (not exactly due to the presence of self-loops). Additionally, the relationship is increasingly monotonic, and as the number of classes increases, the positive gain region increases both in size, where decreasingly homophilic graphs profit from neighbor aggregation starting from $h \approx 0.45$ ($C = 3$), $h \approx 0.35$ ($C = 4$), and $h \approx 0.3$ ($C = 5$), and magnitude, where on homophilic graphs, the attained maximum gain increases.

Furthermore, we observe that at the lowest label rate, GCN advantage grows w.r.t. higher label rates on highly homophilic graphs, which becomes clearer when analyzing the second row of plots. Across the whole lr spectrum, gain is approximately constant, and it slightly increases at $lr = 0.01$ on homophilic graphs since MLP performance drops and GCN accuracy is near perfect (plots not shown), as in the 2-class setting. So, the effect of label rate is conditioned on homophily: when neighbor information is consistent, even at scarce labels, GCN is able to exploit graph structure and maximize its advantage.

The surface plots on the bottom row show that, in the case of $C = 3$ there exists a small heterophily regime where the gain is positive, so neighbor aggregation is actually able to make nodes from each class more distinguishable. This effect vanishes for $C = 4$ and $C = 5$, as signals get diluted with an increasing number of foreign classes to aggregate from. Additionally, homophily is the dominant factor in determining the gain, the change in the gain axis is much more pronounced along the h axis than along the lr axis.

Our statistical analysis, summarized in Table 5.4 also backs the previous evidence: even though in every C the fixed effects and the interaction effect are significant, the magnitude of the effect of homophily is overwhelmingly higher, and it grows stronger with C. Therefore, graph structure gains importance in determining GCN advantage: it helps more if classes as separable, and it hurts more if they are entangled. As well, the label rate and interaction effects grow with C, signaling their importance also increases as graphs become more complex. So, the way label rate modulates the relationship between homophily and gain becomes more pronounced on graphs with several classes. Another consequence is that label rate has a statistically significant effect along more homophily levels, as shown in the single-effects analysis. See Appendix A for the rest of the ANOVA diagnostics.

All-in-all, as in the 2-class CSBM-H graphs, we observe a notorious non-linear, non-monotonic influence of homophily and a comparatively smaller influence of label rate,



(a) $C = 3$. (b) $C = 4$. (c) $C = 5$.

Figure 5.5.: Gain plots for multiclass CSBM-H graphs with $C \in \{3, 4, 5\}$.

whose main role is modulating the effect of homophily on the gain. In the multiclass setting, we confirm that the effect of label scarcity effect is indeed tied to homophily, as it amplifies the value of graph information, especially as we increase the number of classes. These analysis provides insights into the interplay of these variables in a controlled environment (fixed degree distributions, number of nodes per class and separability of inter-class features), and the role each plays in the GCN/MLP performance gap, and motivates the next experiment: how to predict the gain under label scarcity conditions in such structurally homogeneous graphs, and whether the findings can be generalized to the more complex setting of empirical networks.

5.1.3. Gain estimation

The previous analysis shed light into a common claim in literature: homophily is a good indicator of GNN performance. In the CSBM-H framework, homophily is a parameter of the model and neighbor sampling is guided by it according to the class compatibility matrix (Eq. (4.1)). We now disregard the fact that we know the ground-truth parameter h from which graphs are generated and test the ability of the homophily measures listed

Table 5.4.: Two-way repeated-measures ANOVA results for $C \in \{3, 4, 5\}$.

	h			lr			$h : lr$		
C	F	p	η^2_G	F	p	η^2_G	F	p	η^2_G
3	22989.7	<0.001***	0.964	10.64	<0.001***	0.049	22.94	<0.001***	0.062
4	41386.4	<0.001***	0.979	38.08	<0.001***	0.154	54.28	<0.001***	0.124
5	77043.6	<0.001***	0.989	102.53	<0.001***	0.312	98.13	<0.001***	0.213

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

in Table 4.4 to estimate the gain using the information available in the graph's labeled nodes, features and structure. Ultimately, the goal is to predict whether message passing will help or not on a scenario where the "true" homophily cannot be computed directly.

We fix the data generation process of the 2-class CSBM-H model according to Table 4.1, and to test the robustness of each metric across the whole homophily spectrum. We generate 500 graphs independently (no rewiring), each with a new h in the $[0, 1]$ interval, sampled uniformly at random, and model training is done as described in Section 4.6. To align with the label rates commonly used in literature, and to avoid redundancy in the analysis, we restrict the evaluation to one specific supervision regime, with $lr = 0.05$ both for training and for the definition the label-induced subgraph $G[\mathcal{V}_{\text{train}}]$ on which label-based metrics compute the estimator $\hat{h}_{\text{label}}$.

Label-based estimators

As was previously discussed (and depicted in Figure 5.3a), the relationship between the input parameter h and the absolute gain is non-monotonic for 2-class CSBM-H graphs, with a turning point at the minimum, $h \approx 0.5$. Therefore, to analyze the relationship between homophily estimators and the gain, we separate the graphs into two regimes, a homophilic regime for graphs with $h \geq 0.5$ and a heterophilic regime for graphs with $h < 0.5$, which in each case yields monotonic behaviors. Importantly, on each regime, we compute each metric on the same 500 graphs, and the gain is also shared by h_{label} and $\hat{h}_{\text{label}}$, i.e. we only train the models once for each graph at $lr = 0.05$, so the two measurements share the gain for a given instance.

Figure 5.6 shows the scatterplots of the homophilic instances and their corresponding gain and label homophily estimations. The top row shows the relationship for the metrics computed on the entire set of labels. We observe positive monotonic relationships and a high correlation between several measures, particularly h_{node}, h_{edge}, h_{adj}, h_{unb}. Each measure consistently predicts that, as homophily grows, the gain will also grow, such that for graphs closer to the neutral regime, the estimated gain becomes increasingly negative. The bottom row shows the estimations under label scarcity, where the metrics degrade due to the steep loss of label signal: 5% of labels for a 1000 node graph represents a total of 25 nodes per class, from which the metrics have to make a new estimation, so some nodes are expected to be isolated and therefore contribute no information.

In the heterophilic regime, the relationship between gain and h_{label} is inverted in most

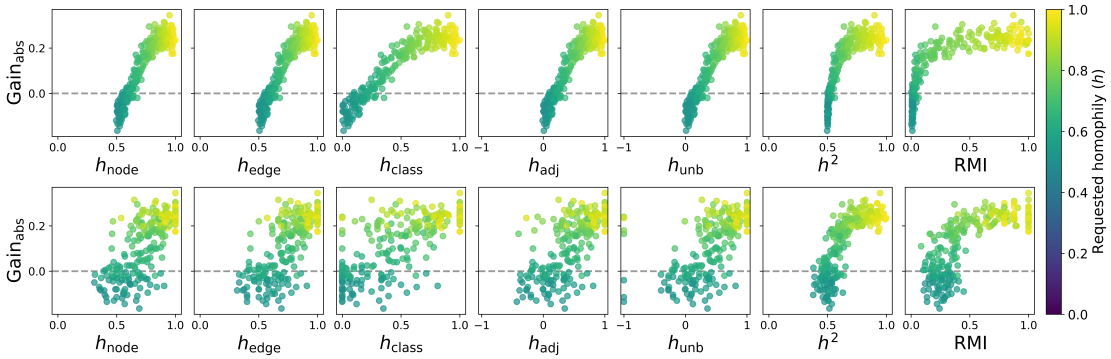


Figure 5.6.: Label-based estimators on the homophilic regime ($h \geq 0.5$). The top row shows the relationship between Gain_{abs} and h_{label} for each metric computed on the entire graph G, and the bottom row shows the relationship between Gain_{abs} and $\hat{h}_{\text{label}}$ for each metric computed on $G[\mathcal{V}_{\text{train}}]$. Each point represents a CSBM-H graph and color represents the value of the CSBM-H parameter h.

cases, showing negative correlations except for h^2, as shown in Figure 5.8. The behavior of h^2 is expected: for heterophilic graphs, second order neighbors tend to have the same class membership as the ego node, and the more heterophilic a graph is, the higher the GCN advantage is expected to be, as shown in the previous section. Also, we see h_{class} mostly collapses: per definition, it only aggregates over classes whose nodes connect to each other more than expected by chance, which is more rarely the case in heterophilic graphs. RMI also collapses in the full visibility scenario: community detection relies on modularity, i.e. communities are assigned based on a higher intra-community than inter-community edge density, and on heterophilic graphs, communities will have mixed nodes, which is not informative of class membership, in contrast to homophilic graphs where nodes of the same class tend to define dense, detectable communities. In the label scarce setting, sparse, isolated nodes can form singleton communities and thus communities can appear informative of class membership. These findings highlight the inadequacy of some metrics for capturing the GCN/MLP relationship under label scarcity on the heterophilic regime.

From all the measures, 2-hop homophily appears to be the most robust when computed with a reduced subset of the labels, as it still strongly captures the behavior of the gain on both regimes. This result is consistent with our experimental design (2-layer GCN and MLP): the graph-aware model aggregates information from its 2-hop neighborhood, while 2-hop homophily measures the extent to which the 2-hop neighbors share the same label, which is the case in high homophily and high heterophily, which in turn we showed to be optimal conditions for GCN performance. Furthermore, it inherently leverages graph structure, it collects more information per node than its 1-hop equivalent thanks to its larger receptive field, which is especially beneficial in the label-scarce setting.

We formally evaluate the ability of all metrics to model the gain through the Spearman rank correlation $\rho_s \in [-1, 1]$, which evaluates whether the ordering of the graphs in terms of gain aligns to the ordering in terms of h_{label} or $\hat{h}_{\text{label}}$ (either in the positive or

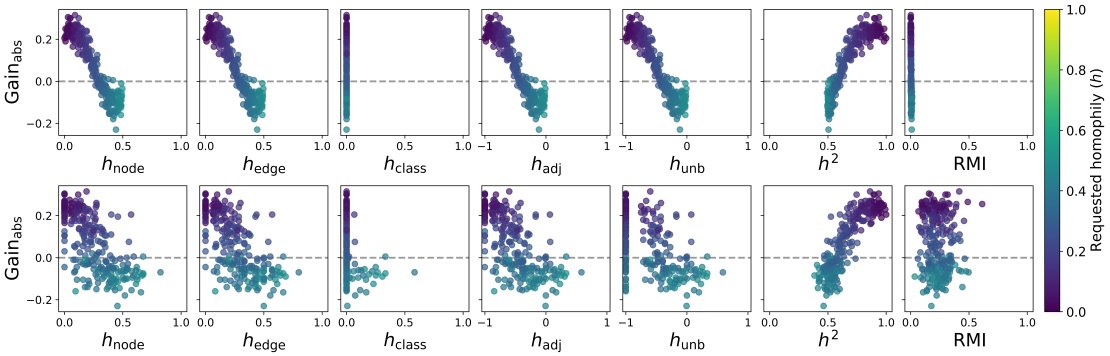


Figure 5.7.: Label-based estimators on the heterophilic regime ($h < 0.5$). The top row shows the relationship between Gain_{abs} and h_{label} for each metric computed on the entire graph G, and the bottom row shows the relationship between Gain_{abs} and $\hat{h}_{\text{label}}$ for each metric computed on $G[\mathcal{V}_{\text{train}}]$. Each point represents a CSBM-H graph and color represents the value of the CSBM-H parameter h.

Table 5.5.: Spearman correlation ρ_s between label-based homophily metrics and Gain_{abs}. We highlight the strongest correlation in each case.

Regime		h_{node}	h_{edge}	h_{class}	h_{adj}	h_{unb}	h^2	RMI
$h \geq 0.5$	h_{label}	0.91	0.91	0.91	0.91	0.91	0.91	0.90
	$\hat{h}_{\text{label}}$	0.73	0.73	0.71	0.71	0.70	0.84	0.80
$h < 0.5$	h_{label}	−0.93	−0.93	–	−0.93	−0.93	0.93	−0.83
	$\hat{h}_{\text{label}}$	−0.71	−0.72	−0.36	−0.72	−0.62	0.88	−0.11

negative direction). Under label scarcity, a good estimator should preserve the trend observed under full visibility, else the joint pattern changes and the estimation becomes meaningless. Table 5.5 shows the Spearman correlation between the two variables both under full visibility and under label scarcity. In the homophilic regime, ρ_s is extremely high for all measures, and remains high for h^2 under label scarcity, dropping from 0.91 to 0.84, still a strong correlation. Similarly, in the heterophilic regime, correlation is the highest for h^2 (in absolute terms, we don't care whether the relationship is positive or negative), dropping from 0.93 to 0.88. So, this metric is the most adequate in terms of preserving the original gain-homophily trend.

Finally, we follow-up the evaluation by computing the average error incurred by each estimator w.r.t. the ground truth values across the 500 instances, normalized to account for different scales, as defined by Eq.(4.9). This follows from the observation that Spearman correlation does not account for the size of the deviations from the ground truth as long as the ordering is preserved. Table 5.6 confirms that, in expectation and across all levels of homophily, h^2 degrades the least when computed on $G[\mathcal{V}_{\text{train}}]$.

We also plot the error distribution across all samples on each metric, which shows how much estimations deviate from the ground truth values at the individual level and see a high concentration of low errors for h^2, as shown in Figure 5.8 (the very high

Table 5.6.: Normalized root mean squared error (NRMSE) between the estimated and ground-truth label-based homophily metrics. We highlight the lowest expected error.

	h_{node}	h_{edge}	h_{class}	h_{adj}	h_{unb}	h^2	RMI
NRMSE	0.1192	0.1169	0.1557	0.1233	0.1629	0.0479	0.2270

concentration of zero-error samples in h_{class} is due to graphs collapsing to 0 in the heterophilic regime). So, the estimations made with h^2 are expected to closely follow the original homophily, making it the most robust label-based metric.

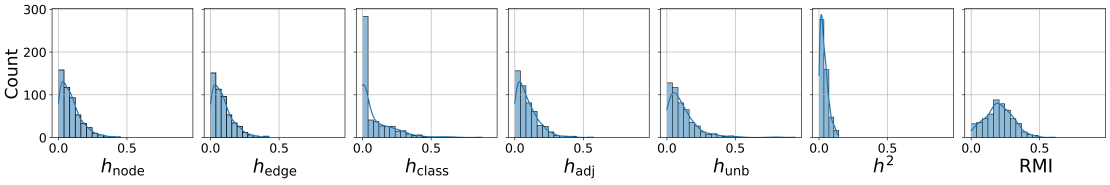


Figure 5.8.: Label-based estimators error distribution ($e_i = \frac{|h_i - \hat{h}_i|}{h_{\max} - h_{\min}}$) for each metric with the corresponding smooth KDE line. The y-axis represents the number of observations in the corresponding error range.

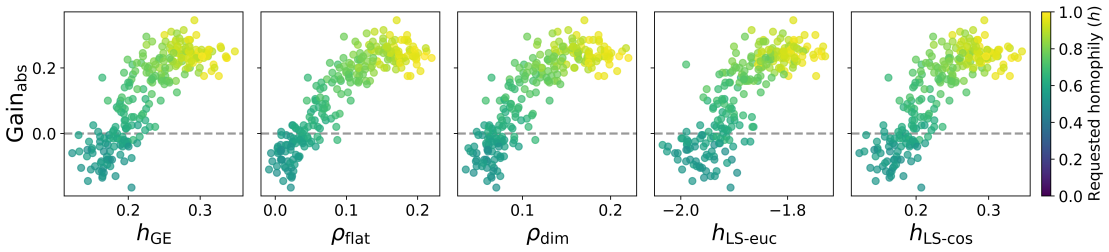
Feature- and structure-based estimators

The feature- and structure-based proxies for gain estimation are computed on the same 500 instances generated in the previous section. We also follow a similar evaluation approach, except that, because there is no ground truth at full label visibility since we are not using label information, we rely solely on Spearman correlation to quantify the level of alignment of each metric with the gain.

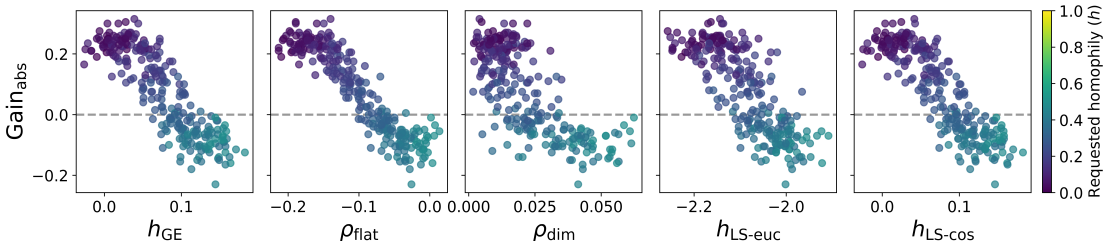
Figure 5.9 shows the scatterplots of the homophilic and heterophilic instances and their corresponding gain and feature homophily. Figure 5.9a shows the homophilic graphs, where we observe positive monotonic trends and, as a result, a high correlation between all measures and the gain, the same trends observed in the label-based metrics. This alignment with label homophily is expected, as in CSBM-H graphs, the feature generation process is class-conditioned: nodes from the same class share the same underlying generative parameters, therefore, the higher the ground-truth homophily h is, the higher the tendency of nodes with similar features to be connected will be (positive correlation).

On the other hand, Figure 5.9b shows the alignment for heterophilic instances. For feature-based metrics in the heterophilic regime, the relationship between gain and homophily is inverted, showing negative correlations that also cleanly align with label-based estimators. So, increasingly heterophilic graphs show a decreasing alignment between connectivity and feature similarity (negative correlation). These results for feature-based metrics are validated by the highly positive Spearman correlation values for homophilic graphs and highly negative values for heterophilic graphs, as shown in Table 5.7.

As mentioned in Section 2.4.2, h_{GE} and h_{LS} are the feature similarity equivalents of



(a) Homophilic regime ($h \geq 0.5$).



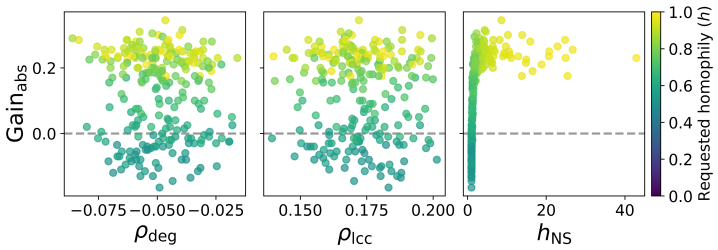
(b) Heterophilic regime ($h < 0.5$).

Figure 5.9.: Feature-based estimators on the homophilic and heterophilic regimes. Each point represents a CSBM-H graph and color represents the value of the CSBM-H parameter h.

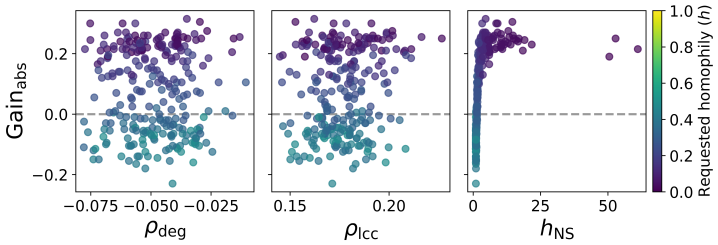
h_{edge} and h_{node} respectively, where we replace the class indicator function $\delta[y_u = y_v]$ with a measure of the similarity between the feature vectors of connected nodes $\text{sim}(u, v)$. Similarly, feature assortativity measures like ρ_{flat} and ρ_{dim} measure the correlation between the feature vectors of connected nodes, which are increasingly similar with increasing h and increasingly dissimilar with decreasing h. Hence, such a strong alignment is cleanly represented on synthetic graphs, but not likely to be observed in empirical graphs where the coupling between label homophily and feature similarity can be much weaker and noisy.

Figure 5.10 shows the scatterplots of the homophilic and heterophilic instances and their corresponding gain and structure homophily. In the case of structure homophily estimators, both in the homophilic (Figure 5.10a) and heterophilic (Figure 5.10b) regimes, we observe no systematic relationship with ρ_{deg} (Degree Assortativity) and ρ_{lcc} (LCC Assortativity). Given the CSBM-H setup, which specifically controls for class connectivity while keeping degrees approximately constant across nodes, the results are consistent with the generative process and variations can be attributed to stochasticity.

On the other hand, h_{NS} (Neighborhood Similarity) is strongly correlated with the gain as it quantifies the extent to which intra-class neighborhood distributions are similar ($\cos(D^{\mathcal{N}}_{u:}, D^{\mathcal{N}}_{v:})$, where $y_u = y_v$) and inter-class neighborhood distributions are dissimilar ($\cos(D^{\mathcal{N}}_{u:}, D^{\mathcal{N}}_{v:})$, where $y_u \neq y_v$). In CSBM-H graphs, edge sampling is controlled by the class compatibility matrix $\mathbf{H}$, so for nodes in c_1, the empirical neighborhood distribution will be $D^{\mathcal{N}}_{v:} \approx [h,\ 1-h]$, and for nodes in c_2, $D^{\mathcal{N}}_{v:} \approx [1-h,\ h]$ (provided nodes have the same average degree and the classes are balanced). Thus, the similarity be-



(a) Homophilic regime ($h \geq 0.5$).



(b) Heterophilic regime ($h < 0.5$).

Figure 5.10.: Structure-based estimators on the homophilic and heterophilic regimes. Each point represents a CSBM-H graph and color represents the value of the CSBM-H parameter h.

tween the neighborhood distributions of intra-class nodes will be maximal (numerator: $\cos(D^{\mathcal{N}}_{u:}, D^{\mathcal{N}}_{v:}) \approx 1$) and the similarity between the neighborhood distributions of inter-class nodes will be minimal (denominator: $\cos(D^{\mathcal{N}}_{u:}, D^{\mathcal{N}}_{v:}) \approx 0$) on both extremes, $h = 0$ and $h = 1$:

$$h = 0: \quad \mathbf{H} \approx \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad h = 1: \quad \mathbf{H} \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

whereas for $h \approx 0.5$, the neighborhood distributions for both classes become $D^{\mathcal{N}}_{v:} \approx [0.5,\ 0.5]$, i.e. indistinguishable from each other. Therefore, neighborhood similarity (numerator/denominator) will monotonically increase with higher homophily and higher heterophily, the same pattern exhibited by GCN accuracy, signaling its very high positive correlation with the gain, as shown in Table 5.7. So, among all feature- and structure-based measures, h_{NS} is the strongest predictor, as it captures how distinguishable the neighborhood distributions are, which is directly related to whether aggregation will produce separable embeddings after message passing.

Table 5.7.: Spearman correlation ρ_s between feature- and structure-based metrics and Gain_{abs}. We highlight the strongest correlation in each case.

Regime	h_{GE}	ρ_{flat}	ρ_{dim}	$h_{\text{LS-euc}}$	$h_{\text{LS-cos}}$	ρ_{deg}	ρ_{lcc}	h_{NS}
$h \geq 0.5$	0.83	0.88	0.86	0.84	0.83	−0.04	−0.06	0.91
$h < 0.5$	−0.87	−0.92	−0.73	−0.81	−0.88	−0.01	0.13	0.93

5.2. Empirical datasets

5.2.1. Characterizing label homophily measures

The goal of the empirical datasets pipeline is to evaluate the extent to which the conclusions derived from the synthetic datasets generalize to real-world networks. The exploratory data analysis in Section 4.4 provided an initial understanding of the composition of the empirical graphs and showed they are structurally heterogeneous: non-uniform class distributions, irregular degree distributions, and heterogeneous, high-dimensional and unnormalized feature vectors. All the previous factors we were able to control in the synthetic scenario, which was helpful in understanding how to best predict the performance advantage of graph-aware models over graph-agnostic models by isolating the role that homophily plays in GNN performance.

Table 5.8.: Homophily metrics at full label visibility, ranked by h_{unb} and separated according to their homophily regime (homophilic, neutral, heterophilic).

Dataset	h_{node}	h_{edge}	h_{class}	h_{adj}	h_{unb}	h^2	RMI
Cora	0.8252	0.8100	0.7657	0.7711	0.9200	0.7282	0.6226
WikiCS	0.6774	0.6543	0.5675	0.5790	0.8569	0.3645	0.5257
CiteSeer	0.7166	0.7355	0.6267	0.6707	0.8520	0.6758	0.6121
PubMed	0.7924	0.8024	0.6641	0.6860	0.7653	0.7424	0.3915
Amazon-ratings	0.3757	0.3804	0.1266	0.1403	0.2769	0.3658	0.0948
Tolokers	0.6344	0.5945	0.1801	0.0925	0.1016	0.6508	0.1201
Chameleon	0.2471	0.2299	0.0411	0.0324	0.0742	0.2539	0.1512
Questions	0.8980	0.8396	0.0790	0.0209	0.0583	0.9087	0.0341
Squirrel	0.2172	0.2221	0.0307	0.0067	0.0270	0.2143	0.0156
Minesweeper	0.6829	0.6828	0.0094	0.0094	0.0145	0.6798	0.0053
Actor	0.2199	0.2167	0.0064	0.0027	0.0053	0.2136	0.0051
Wisconsin	0.1552	0.1778	0.0461	-0.1733	-0.2851	0.4262	0.1231
Cornell	0.1110	0.1227	0.0383	-0.2202	-0.4675	0.4074	0.1471
Roman-Empire	0.0460	0.0469	0.0208	-0.0467	-0.4913	0.0843	0.0329
Texas	0.0567	0.0609	0.0000	-0.2937	-0.8050	0.5505	0.1025

We start the analysis on empirical datasets by defining ground-truth label-homophily values on the graphs at full label visibility. We compute the label-based measures defined in Table 4.4, and rank the datasets in terms of h_{unb}, as shown in Table 5.8. We use unbiased homophily as the primary metric to describe a dataset's homophily mainly due to its robustness to class imbalances and irregular degree distributions. Using metrics like h_{node} or h_{edge} will yield biased estimations of true homophily, as can be seen on the Questions ($h_{\text{node}} = 0.898$, $h_{\text{edge}} = 0.8396$) and Minesweeper ($h_{\text{node}} = 0.6829$, $h_{\text{edge}} = 0.6828$) datasets, which have drastic class imbalances as shown in Figure 2.4, leading to overestimations of their true level of homophily.

Also, h_{unb} provides a clean separation between the homophilic, heterophilic and neutral regimes: the datasets in the homophilic group (Cora, WikiCS, CiteSeer, PubMed)

have a higher intra-class connectivity than expected at random, those in the neutral group (Amazon-ratings, Tolokers, Chameleon, Questions, Squirrel, Minesweeper, Actor) display approximately the same level of mixing to that expected if the edge connections were generated independently of the nodes' labels, and the datasets in the heterophilic group (Wisconsin, Cornell, Roman-Empire, Texas) show a higher inter-class connectivity than expected at random. Since this metric was designed for comparability across datasets by measuring the level of connectivity w.r.t. a baseline following the same class distributions as the original graph, and calibrated to represent fully heterophilic graphs at $h_{\text{unb}} = -1$, random mixing at $h_{\text{unb}} = 0$ and fully homophilic graphs at $h_{\text{unb}} = 1$, we select it for future analysis as the reference for homophily, as was the case with the h parameter within CSBM-H graphs.

We study the relationship between the measures by computing pairwise correlations across the empirical datasets. We use the Pearson correlation coefficient to quantify the degree of agreement of the numerical values of the measures, which is 1 in the case of perfect positive linear correlation and -1 in the case of perfect negative linear correlation. The upper triangle in Figure 5.11 shows the correlations between each pair of metrics at full label visibility as well as their associated p-values to test the null-hypothesis $H_0 : \rho = 0$ (no correlation in the population). We mark statistically significant correlations (after Bonferroni correction with $\alpha = 0.05$ to get the significance threshold $\alpha' = \frac{0.05}{\binom{7}{2}} \approx 0.0024$) with an asterisk. High absolute correlation values indicate strong linear relationships, as is the case between h_{node} and h_{edge}, indicating redundancy in the information they capture.

The lower triangle shows the pairwise scatterplots of the measurements on each dataset. We include linear regression fits with 95% confidence intervals to visualize the degree of linear association between the measures. The association is particularly high between pairs of biased (e.g. h_{node}–h_{edge}) and normalized (e.g. h_{adj}–h_{unb}) matrics, and not as strong between them due to the effect of class imbalances, as mentioned before. Also, we see h^2 tends to assign high values to heterophilic datasets (e.g. Wisconsin, Cornell, Texas, see Table 5.8), as was the case in the synthetic datasets since second order neighbors tend to match the class of the ego node. This is not the case on Roman-Empire given the high number of classes in the dataset. The diagonal shows the empirical histograms of the individual measures. In the case of h_{unb}, we see the separation into 3 regimes, whereas other measures tend to misclassify datasets like Questions as highly homophilic.

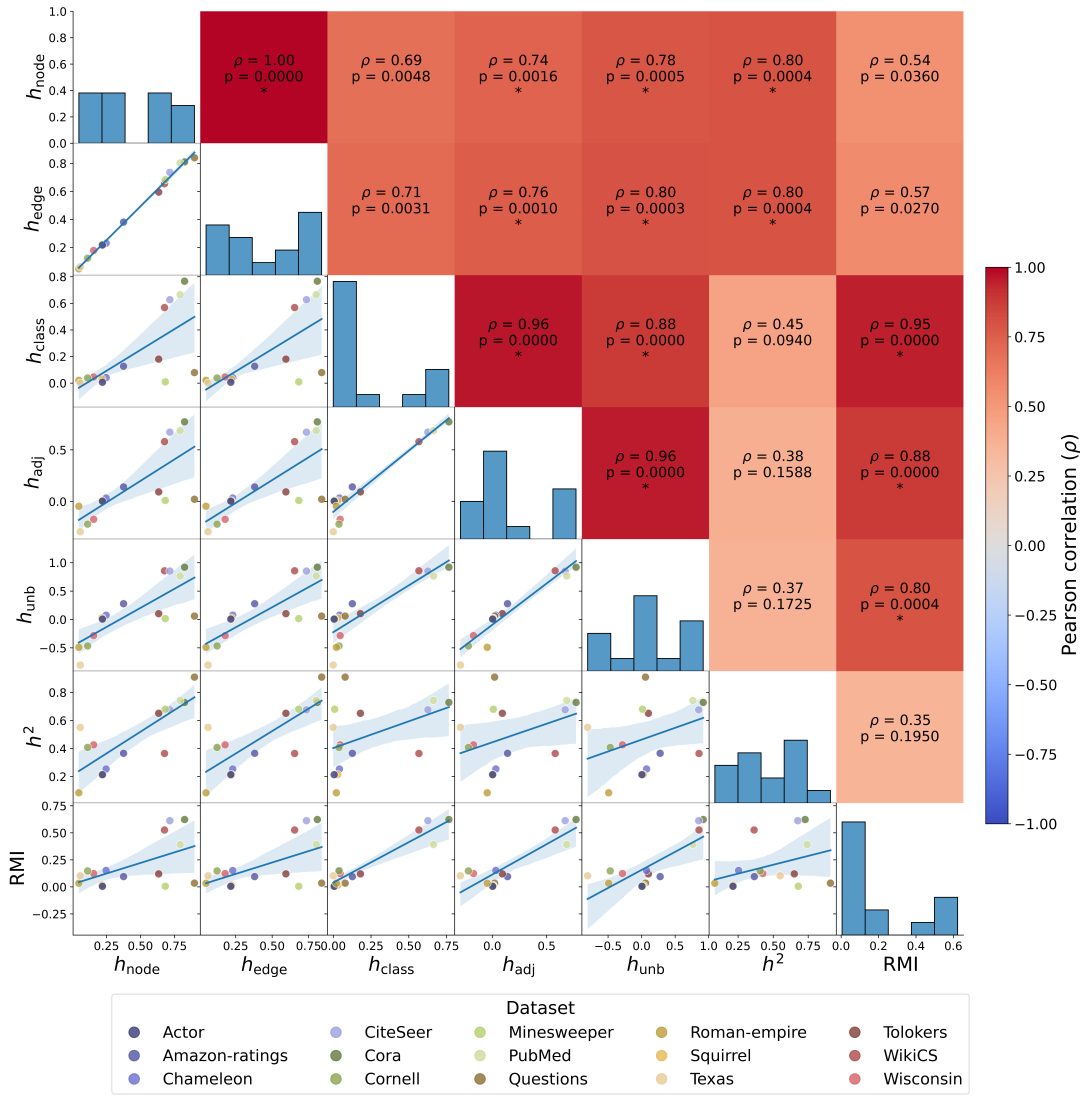


Figure 5.11.: Pairwise relationships between homophily measures. The upper triangle shows the pairwise Pearson correlation coefficients, with Bonferroni correction ($\alpha' \approx 0.0024$), where $*$ denotes $p < \alpha'$. The lower triangle shows the pairwise scatterplots and linear regression lines with 95% confidence intervals. The diagonal shows the marginal distribution of each measure.

5.2.2. Performance degradation analysis

Similarly to Section 5.1.2, we test the effect of label scarcity on the performance gap between GCN and MLP on empirical datasets. For each dataset, we sample a different train/validation/test split 50 times following Shchur et al. 2019, train the two models on each split, and visualize the distribution of the test accuracies for each model. We use a random sampling strategy, meaning that for a specific label rate, we sample the

same number of nodes per class, independently of the size of the classes. The splits used for training are saved to later be used to generate the label-induced subgraphs to compute $\hat{h}_{\text{label}}$ for the gain estimation analysis in Section 5.2.3. We use label rates $lr \in \{0.20, 0.15, 0.10, 0.05, 0.01, \text{min}\}$, where min denotes 1 node per class on each dataset, with fixed validation and test set proportions.

Figure 5.12 shows a snapshot of the resulting test accuracy distributions of each model on all datasets at the highest label rate, $lr = 0.20$, as a representative supervision regime to visualize the performance of each model independently. The datasets are ranked by h_{unb} from left to right to distinguish every regime. We observe, as is usually reported in literature, that in heterophilic regimes, MLP yields a higher accuracy in expectation, with extreme variability in the case of Texas, Cornell and Wisconsin, which arises from the fact that the datasets are comparatively much smaller than the rest, making every misprediction on the test set have a big impact on the final accuracy. In the case of GCN, aggregating over several classes (5 on each dataset) mixes signals and amplifies the negative effect of split size, making test set predictions even more uncertain. This illustrates a common conclusion made in literature, heterophily hurts GCN performance, yet, as shown in Section 5.1.1, under certain conditions, this is not forcefully the case.

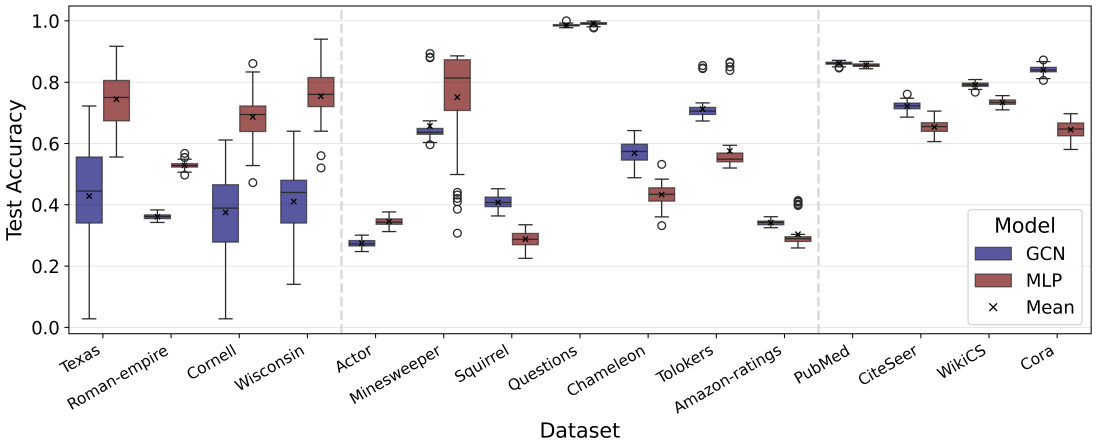


Figure 5.12.: Distribution of test set accuracies for GCN and MLP over 50 runs on each dataset at $lr = 0.20$. Datasets are grouped into heterophilic (left panel), neutral (middle panel) and homophilic (right panel) according to h_{unb}.

On the other hand, GCN has a higher accuracy in expectation on homophilic datasets, and it benefits the most on Cora, the most homophilic dataset, making it clear that neighbor aggregation makes classes more separable when message passing reinforces the identity of each class. In the neutral regime, which model to choose is unclear. It is worth mentioning the case of the Questions dataset, which shows near perfect accuracy on both models, with very low variability. This comes from its highly imbalanced nature, with 97.9% of the nodes belonging to one class, so even naïve classifiers would achieve high accuracies, one of the reasons why works like Subedi et al. 2025 evaluate model performance using F1-scores.

Naturally, we want to study the degradation of the accuracy of the models and to characterize the performance gap between GCN and MLP as we approach extreme label scarcity. We study the behavior of the performance of the models and the gain on each homophily regime separately to find common patterns across datasets in the similar levels of homophily. Figure 5.13, Figure 5.14 and Figure 5.15 show the aggregated (mean ± 1 standard deviation) accuracy and absolute gain results for representative datasets of each regime when trained with every label rate up to 1 labeled node per class:

- **Homophily**. Figure 5.13 shows the accuracy and absolute gains of the most homophilic dataset, Cora ($h_{\text{unb}} = 0.92$), whose gain behavior is representative of that of homophilic datasets[2]. On homophilic datasets, we observe low variability in the accuracy of either model as we decrease the number of training nodes. The gain behavior is consistent for homophilic datasets from $lr = 0.20$ down to $lr = 0.01$: increased label scarcity leads to an increase in the advantage of GCN, as structural consistency compensates for the lack of supervision signal, so message passing is especially beneficial as we decrease the number of labeled samples. At $lr = \text{min}$, the gain behavior is much less predictable, for Cora we see a steep dip in the expected gain, but for other datasets (e.g. WikiCS, see Appendix B) there is even a slight increase in the gain. A common pattern is that gain variance significantly increases under extreme label scarcity. For every homophilic dataset we see positive gains across the entire spectrum, so GCN would be the preferred model independently of the number of labeled nodes.

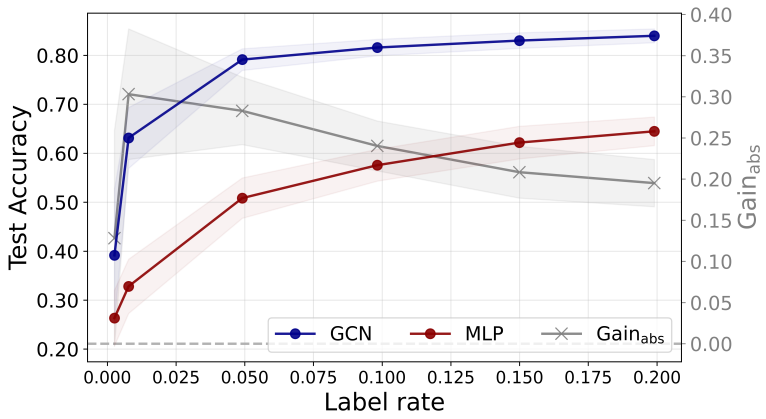


Figure 5.13.: Accuracy and gain plot for a representative homophilic dataset, Cora ($h_{\text{unb}} = 0.92$), trained at different levels of label scarcity. Label rates are approximate due to sampling.

- **Heterophily**. Figure 5.14 shows the accuracy and absolute gains of Roman-Empire ($h_{\text{unb}} = -0.4913$). As opposed to homophilic datasets, MLP performance

[2] As a reference, Cora's public split is approximately 5.2%, or 20 nodes per class, and Kipf and Welling 2017 report an accuracy of 81.5% (GCN) and 55.1% (MLP), which are on-par with our results for the 5% label rate. They however, make their evaluation only on that specific split.

is consistently superior, with the other heterophilic datasets showing high variability across the entire spectrum due to their small number of nodes (see Appendix B). The gain is also consistently negative, even when accounting for its variability. In contrast to Cora, in Roman-Empire, gain significantly increases under extreme label scarcity, signaling a steeper decrease in MLP performance with low supervision. In fact, in Wisconsin, Cornell and Texas there is a constant increase in gain as we reduce supervision, although it remains negative throughout. So, considering the homophilic and heterophilic datasets, there is a clear association between expected gain and the homophily regime.

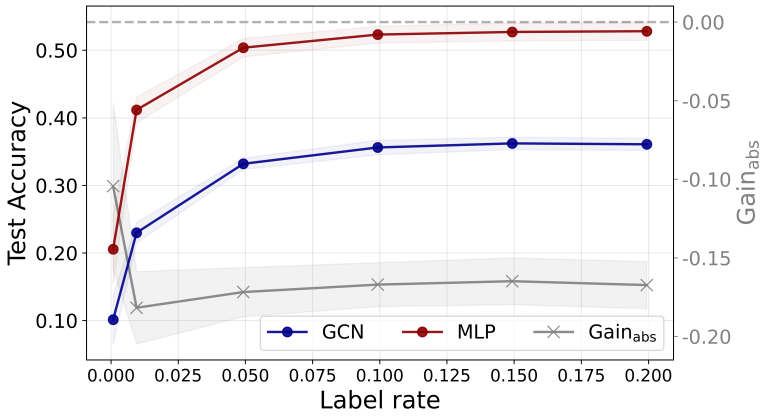
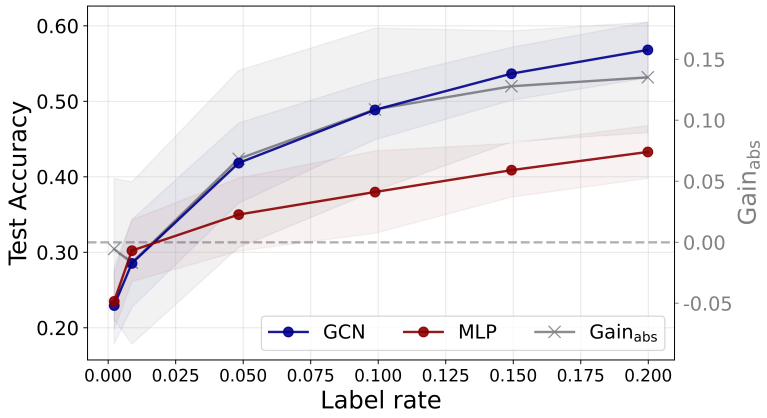


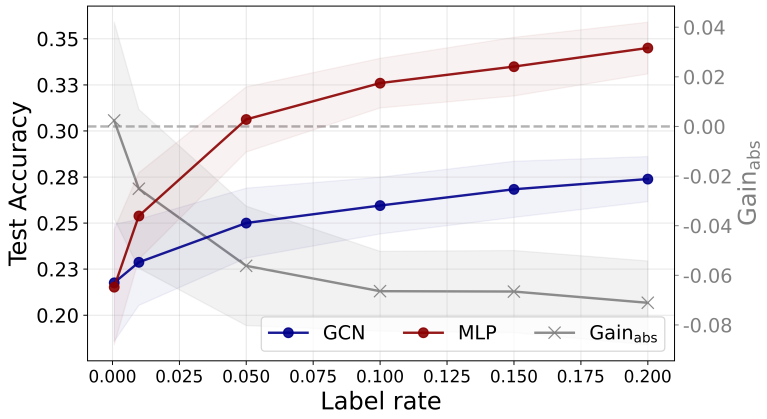
Figure 5.14.: Accuracy and gain plot for a representative heterophilic dataset, Roman-Empire ($h_{\text{unb}} = -0.4913$), trained at different levels of label scarcity. Label rates are approximate due to sampling.

- **Neutral**. Figure 5.15 shows two representative datasets from the neutral regime, Actor ($h_{\text{unb}} = 0.0053$) and Chameleon ($h_{\text{unb}} = 0.0742$), which display opposing accuracy and gain patterns. While Chameleon shows mostly higher GCN accuracy and positive gains, Actor actually behaves more similarly to an heterophilic dataset, with superior MLP performance and therefore negative gains throughout, which leads to a natural observation: homophily is not reliable way to predict GCN advantage in the neutral regime. Wang et al. 2024 study the difference in the performance of the models by analyzing the neighborhood distributions of the nodes on each class for each dataset and characterize them as "good heterophily" in the case of Chameleon, and "bad heterophily" in the case of Actor.

  In fact, by computing the empirical class-compatibility matrix of each dataset, we can observe the main argument of the study: whether classes become more separable after aggregation is related to the *type* of heterophily patterns that the classes have (plus the average degree), i.e. the distance between their neighborhood distributions, and not directly to whether the graph as a whole is homophilic or



(a) Chameleon.



(b) Actor.

Figure 5.15.: Accuracy and gain plot for a representative neutral datasets, Chameleon ($h_{\text{unb}} = 0.0742$) and Actor ($h_{\text{unb}} = 0.0053$), trained at different levels of label scarcity. Label rates are approximate due to sampling.

heterophilic:

$$\mathbf{H}_{\text{Ch}} = \begin{array}{c|ccccc} & c_1 & c_2 & c_3 & c_4 & c_5 \\ \hline c_1 & 0.18 & 0.42 & 0.12 & 0.13 & 0.15 \\ c_2 & 0.37 & 0.19 & 0.15 & 0.13 & 0.16 \\ c_3 & 0.10 & 0.14 & 0.24 & 0.25 & 0.28 \\ c_4 & 0.09 & 0.11 & 0.22 & 0.26 & 0.31 \\ c_5 & 0.10 & 0.12 & 0.23 & 0.29 & 0.27 \end{array} \qquad \mathbf{H}_{\text{Ac}} = \begin{array}{c|ccccc} & c_1 & c_2 & c_3 & c_4 & c_5 \\ \hline c_1 & 0.11 & 0.16 & 0.21 & 0.25 & 0.26 \\ c_2 & 0.11 & 0.17 & 0.20 & 0.26 & 0.26 \\ c_3 & 0.11 & 0.17 & 0.20 & 0.26 & 0.26 \\ c_4 & 0.11 & 0.17 & 0.21 & 0.26 & 0.25 \\ c_5 & 0.12 & 0.17 & 0.21 & 0.24 & 0.26 \end{array}$$

Clearly, Chameleon ($\mathbf{H}_{\text{Ch}}$) has imbalanced, distinctive neighborhood distributions (for each class, information is mostly incorporated from one –different– foreign class), leading to an increase in separability when aggregating information from neighbors, while the classes in Actor ($\mathbf{H}_{\text{Ac}}$) are rather homogeneous and similar to

each other, such that GCN aggregation will have the negative effect of aggregating similar information for each class, making them less distinguishable after message passing. So, graph-agnostic models would be a natural choice in this kind of neighborhood patterns.

The plots for the rest of the datasets can be found in see Appendix B.

All-in-all, the analysis on the neutral regime shows how the advantage of GCN is more related to internal structural patterns than simply to global homophily, as the latter is approximately the same for datasets in this regime. Naturally, homophilic datasets like Cora share this structural property of having distinctive neighborhood distributions: their class compatibility matrices are diagonal-dominant, thus their classes are distinctive and will potentially benefit from neighbor aggregation. Interestingly, empirical heterophilic datasets like Cornell can also have distinctive neighborhood patterns, but given its low average degree ($d = 3.03$, and similarly for Wisconsin, Texas and Roman-Empire, see Table 4.3), each node may receive insufficient signal from its neighbors to reinforce its class identity, resulting in an overall negative effect of neighbor aggregation. This illustrates how homophily is in general a good proxy for the performance of GCN in real-world datasets, yet it is not directly the factor that governs it, which becomes more evident in datasets with irregular neighborhood distributions, multiple classes and limited neighborhood signals.

As documented in Y. Li, Yin, and Ling Chen 2022; Lee et al. 2022; Z. Zhou et al. 2023, at extreme label rates, GCN performance consistently decreases while variability increases, so there is also a supervision-variance trade-off, yet we observe that under certain conditions, the gain actually tends to increase with lower label rates. This is due to the fact that GCN accuracy degrades more slowly than MLP accuracy with less supervision, showing how GCN is able to leverage graph structure even in label-scarce settings provided that neighborhood structure contains informative and distinctive patterns for aggregation to reinforce class identity.

5.2.3. Gain estimation

In real-world data, label scarcity is the rule rather than the exception, so we are interested in predicting the potential GCN advantage in a scenario where we have limited access to node labels. On the synthetic datasets pipeline, we showed that it is possible to make confident predictions of the gain under label scarcity since we were able to isolate the structural mechanisms that lead to high GCN performance, and there exist metrics that closely track those mechanisms (in particular, h^2 and h_{NS}). Once again, the goal is to predict whether message passing will help or not on more complex datasets where neither graph structure not feature quality are controlled. Ultimately we aim to give practitioners tools to select the more suitable model in incomplete information scenarios before any training is performed.

To align with real-world scenarios, we assume we only have those labels that will be used to train the models available, plus the entire graph's connections and all nodes' feature vectors, as is usually the case in graph ML (evidently, an edge where either

endpoint is missing is discarded when building $G[\mathcal{V}_{\text{train}}]$ and computing $\hat{h}_{\text{label}}$, yet it still takes part in the training process and the computation of h_{feature} and $h_{\text{structure}}$). On synthetic datasets, we evaluated graphs on a single label rate of $lr = 0.05$ to study the mechanisms that drive GCN performance, yet in real-world scenarios, the amount of available labeled nodes is usually not a predefined parameter, so we evaluate the robustness of the metrics on all empirical datasets considering different degrees of label scarcity.

Label-based estimators

As was the case in the synthetic pipeline, we mainly evaluate the ability of every metric to model the gain through the Spearman rank correlation $\rho_s \in [-1, 1]$. Under different levels of label scarcity, a good estimator should model the behavior of the gain obtained when training at such label rates. In the case of label-based metrics, for a specific lr, given that there are 50 instances (splits) per dataset, we take the mean $\hat{h}_{\text{label}}$ of the 50 splits per dataset and correlate the resulting values with the mean Gain_{abs} of the same 50 splits per dataset to get $\rho_s(\text{Gain}_{\text{abs}}@lr, \hat{h}_{\text{label}}@lr)$. For ground-truth homophily ($h_{\text{label}}@1.0$, or simply h_{label}), we compute $\rho_s(\text{Gain}_{\text{abs}}@lr, h_{\text{label}}@1.0)$ using the values listed in Table 5.8. Finally, we use an additional evaluation criteria (NRMSE $@lr$) to analyze the robustness of label-based proxies, as described in Section 4.8.4.

Table 5.9.: Spearman correlation between label-based estimators and the absolute gain obtained when training models at different label rates.

		Training lr				
		0.20	0.15	0.10	0.05	0.01
$h_{\text{label}}@1.0$	h_{node}	0.5429	0.5500	0.5679	0.5286	0.7363
	h_{edge}	0.5643	0.5714	0.5857	0.5464	0.7527
	h_{class}	0.6036	0.6286	0.6321	0.5393	0.8791
	h_{adj}	0.7714	0.7786	0.7964	0.6643	0.9505
	h_{unb}	0.7964	0.8071	0.8179	0.7036	0.9670
	h^2	0.1000	0.0964	0.1286	0.3107	0.5110
	RMI	0.3929	0.4000	0.4321	0.5500	0.7308
$\hat{h}_{\text{label}}@lr$	h_{node}	0.7036	0.7143	0.6750	0.6434	0.8006
	h_{edge}	0.6929	0.6964	0.6679	0.6220	0.7912
	h_{class}	0.5143	0.5071	0.5786	0.6631	0.8352
	h_{adj}	0.7179	0.7250	0.7393	0.5929	0.3022
	h_{unb}	0.7536	0.7429	0.7525	0.6344	0.6795
	h^2	0.3107	0.3321	0.4607	0.5862	0.7967
	RMI	0.1179	0.0143	0.0143	0.1071	0.2582

Table 5.9 shows the Spearman correlation between the two variables under different levels of label scarcity. The first block uses homophily values computed from the full set of labels ($h_{\text{label}}@1.0$), whereas the second block uses homophily estimates computed from the labels available at the corresponding label rate ($\hat{h}_{\text{label}}@lr$). Clearly, the normalized

label-based metrics (h_{class}, h_{adj}, h_{unb}) are particularly effective in modeling the gain behavior. Ground-truth h_{unb} strongly correlates with the gains when training models at every label rate. Also, correlation values for all measures spike at $lr = 0.01$ since we don't compute gains for the smaller datasets, Cornell and Texas, as the minimum label rate of 1 node per class is already reached at $lr = 0.05$. This can be seen in the bottom plot of Figure 5.16, which shows the average gains, including ± 1 standard deviation bars, for each dataset at all label rates, and ground-truth h_{unb} as reference.

For the homophily estimates at $lr \in \{0.20, 0.15, 0.10\}$, h_{unb} has the highest values and for $lr \in \{0.05, 0.01\}$, h_{class} surpasses it. Measures not correcting for class imbalances like h_{node} will inherently produce biased homophily estimations when class structure is irregular, so even though gain is actually aligned with how homophilic a graph is, their estimations of true homophily will be far off in the first place, so correlations are weak. Metrics like h_{unb} give more accurate estimations of true homophily, which in empirical datasets we showed to align with model performance: homophilic datasets tend to have positive gains and heterophilic datasets tend to have negative gains. Therefore, results suggest that metrics designed to give unbiased estimates of homophily tend to predict the advantage of graph-aware models more accurately, even when computed on smaller subgraphs of the original dataset.

The comparatively lower degradation of h_{unb} can be explained by the following phenomena. First, it relies on class-level pooling: surviving edges in $G[\mathcal{V}_{\text{train}}]$ are pooled into the normalized class adjacency matrix before the estimate is computed, as shown in Eq.(2.15), so the final estimation is based on class-level edge counts and not on individual node/edge counts. In contrast, metrics like h_{node} compute homophily ratios individually for every node and then aggregate over all nodes, so noisy estimations at the individual level equally affect the final value. Secondly, measures like h_{unb} are already robust to class imbalances, which can become more pronounced when subsampling nodes uniformly at random, leading to more biased estimations in the case of uncorrected measures. Finally h_{unb} is calibrated to take the values -1, 0, 1 for fully heterophilic, random and homophilic graphs respectively, so estimations from very small subgraphs will fall back to consistent reference values and remain comparable across graphs.

Figure 5.16 shows how ground-truth h_{unb} models the gain behavior at every label rate. We see positive, nearly monotonic relationships, and we include linear regression lines with 95% confidence intervals to highlight the near-linear relationship between the variables. As discussed in the previous section, homophilic graphs tend to have positive gains independently of the label rate, while heterophilic graphs tend to have negative gains, and neutral datasets can produce positive or negative gains depending on their structural composition (and also the number of nodes used for training). Furthermore, we observe the supervision-variance trade-off: as we decrease the number of training nodes, gains become less predictable. This is particularly inconvenient in neutral datasets, whose gain estimations are particularly variable at low label rates, and can switch between higher GCN and higher MLP performance depending on the sampled nodes of a specific split[3]. Therefore, homophily (as a general concept) aligns with the performance gap

[3]Q. Li, Han, and X.-M. Wu 2018 give an expression to estimate a lower bound for the number of

between graph-aware and graph-agnostic models, meaning that label-based metrics are good predictors as long as they are robust to structurally heterogeneous graphs.

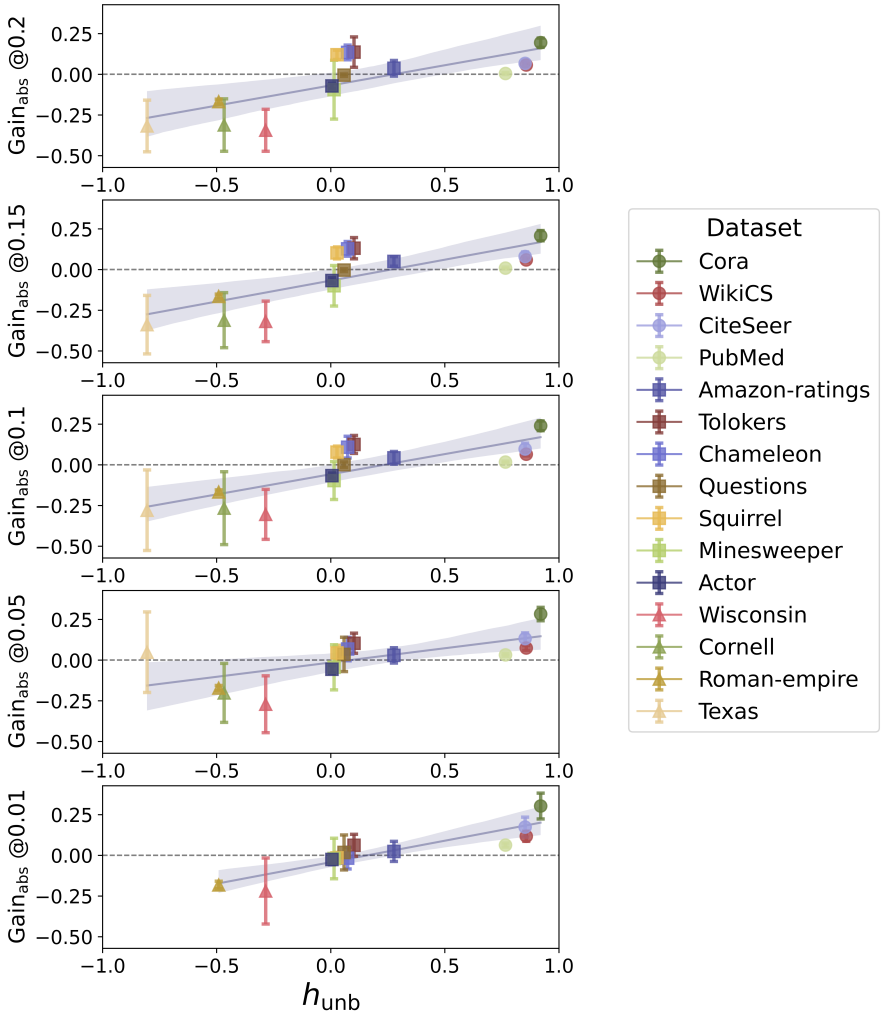


Figure 5.16.: $\text{Gain}_{\text{abs}}@lr$ plots for different supervision regimes with lines of best linear fit and 95% confidence intervals. Observations represent every dataset's mean gain after training at the given label rate, with ± 1 standard deviation bars. Markers indicate the homophily regime of each dataset.

In addition to their correlation with the gain, we assess the adequacy of label-based metrics from the perspective of its deviation from the estimate at full label visibility. We therefore use $\text{NRMSE}@lr$, to quantify the quality of $\hat{h}_{\text{label}}$. Importantly, in every case,

labels (η) needed for training GCNs as a function of the number of layers τ of the model (which influences signal propagation) and the average degree $\hat{d}$ and number of nodes in the graph N, given by the expression $(\hat{d})^{\tau} \cdot \eta \approx N$. For our GCN architecture and the Cora dataset, this amounts to approximately 178 nodes, i.e. $lr \approx 0.066$.

estimators are exclusively compared to h_{label}, therefore no statement is made about how well they align with the gain. Hence, NRMSE should be considered as secondary when evaluating their alignment with the performance of the models, the primary criterion is their correlation with the dependent variable.

Table 5.10.: Normalized root mean squared error (NRMSE @lr) between the estimated and ground-truth label-based homophily metrics at different label rates. We highlight the lowest error at each label rate.

	lr				
	0.20	0.15	0.10	0.05	0.01
h_{node}	0.077	0.092	0.099	0.105	0.237
h_{edge}	0.073	0.087	0.091	0.099	0.233
h_{class}	0.040	0.046	0.051	0.067	0.222
h_{adj}	0.031	0.041	0.064	0.102	0.277
h_{unb}	0.049	0.061	0.073	0.115	0.406
h^2	0.091	0.098	0.110	0.154	0.192
RMI	0.087	0.111	0.145	0.219	0.298

Table 5.10 shows the average errors incurred by each metric when computed at different label rates w.r.t. the original measurement, which were obtained by averaging the per-dataset NRMSE at a given label rate. h_{unb} has a slightly higher degradation when decreasing the amount of labels, yet in practice, since it is highly correlated with the other normalized measures, h_{class} and h_{adj}, which show the lowest degradation except from $lr = 0.01$, they can be used as proxies to the true h_{unb}, especially in empirical datasets. This showcases the robustness of normalized homophily metrics when computed on different subgraphs of the original datasets. In any case, how much a metric deviates from its ground-truth value does not necessarily interfere with its ability to model gain behavior, as shown in the previous analysis.

Feature- and structure-based estimators

For the feature- and structure-based metrics, given that there is only one ground-truth value for each dataset independently of the label rate, we directly correlate these values with the mean gains obtained at a given lr to obtain $\rho_s(\text{Gain}_{\text{abs}}, h_{\text{feature}})$ or $\rho_s(\text{Gain}_{\text{abs}}, h_{\text{structure}})$. Table 5.11 shows a significantly weaker alignment of these metrics with the gain, which is expected in empirical datasets. In the case of feature-based metrics, feature generation is not a class-conditioned controlled mechanism, as was the case in CSBM-H graphs, so feature similarity may arise independently of the underlying classes. In other words, highly assortative features don't necessarily align with ground-truth labels. Furthermore, even if features are actually informative of the underlying class, feature vectors can be sparse, noisy, and heterogeneous within the same class, so measures of pairwise similarity like h_{LS} and h_{GE} can yield low similarity even though features are still informative of their class.

Table 5.11.: Spearman correlation between feature- and structure-based estimators with the absolute gain obtained when training models at different label rates.

	lr				
	0.20	0.15	0.10	0.05	0.01
h_{GE}	-0.2607	-0.2643	-0.2500	-0.0357	0.1484
ρ_{flat}	-0.1536	-0.1571	-0.1393	0.0607	0.3297
ρ_{dim}	0.2464	0.2536	0.2857	0.3357	0.6978
$h_{\text{LS-euc}}$	0.2536	0.2464	0.2500	0.0536	0.1319
$h_{\text{LS-cos}}$	-0.2821	-0.2893	-0.2714	-0.0750	0.1319
ρ_{deg}	0.2536	0.2607	0.2786	0.1429	0.2033
ρ_{lcc}	0.0143	0.0321	0.0643	0.0714	0.1374
h_{NS}	0.1429	0.1357	0.1786	0.3821	0.6044

In the case of structural measures, specifically ρ_{deg} ρ_{lcc}, due to graph heterogeneity, their values for individual datasets are generally close to the random baseline, showing how our empirical datasets don't tend to follow structurally assortative patterns in the first place, therefore the correlation with the gain is weak. Furthermore, as discussed in the synthetic pipeline, whether nodes have high degree or local clustering coefficient assortativity generally does not align with whether message passing will provide an advantage or not, so neither of these attributes is a direct proxy of the potential advantage of GCN.

In synthetic datasets, h_{NS} was the strongest predictor since it modeled node distinguishability before aggregation, the dominant mechanism that determined GCN performance, which is done by comparing the neighborhood distributions of intra-class nodes with those of inter-class nodes. Since degree distributions, feature informativeness and separability, and class balance were controlled in the synthetic setting, we were able attribute the gain's behavior to such neighborhood distributions, which were directly determined by the class compatibility matrix $\mathbf{H}$ and were increasingly distinctive with homophily and heterophily. Since the aforementioned factors are not controlled in empirical datasets, relying purely on a structural description of node distributions is not enough to model the true gain behavior, as it is now influenced by each of those variables.

In fact, heterophilic datasets like Texas and Cornell have rather distinctive class distributions:

$$\mathbf{H}_{\text{Tx}} = \begin{array}{c|ccccc} & c_1 & c_2 & c_3 & c_4 & c_5 \\ \hline c_1 & 0.02 & 0.00 & 0.07 & 0.60 & 0.30 \\ c_2 & 0.00 & 0.00 & 1.00 & 0.00 & 0.00 \\ c_3 & 0.13 & 0.02 & 0.09 & 0.53 & 0.23 \\ c_4 & 0.51 & 0.00 & 0.25 & 0.11 & 0.13 \\ c_5 & 0.52 & 0.00 & 0.23 & 0.26 & 0.00 \end{array} \qquad \mathbf{H}_{\text{Co}} = \begin{array}{c|ccccc} & c_1 & c_2 & c_3 & c_4 & c_5 \\ \hline c_1 & 0.11 & 0.06 & 0.19 & 0.63 & 0.02 \\ c_2 & 0.31 & 0.11 & 0.08 & 0.19 & 0.31 \\ c_3 & 0.50 & 0.04 & 0.00 & 0.32 & 0.14 \\ c_4 & 0.61 & 0.04 & 0.12 & 0.15 & 0.09 \\ c_5 & 0.06 & 0.20 & 0.19 & 0.33 & 0.22 \end{array}$$

and h_{NS} is able to capture the potential benefit of aggregation (Texas: $h_{\text{NS}} = 1.6555$, Cornell: $h_{\text{NS}} = 1.5149$, where the baseline is 1), yet realized gains are negative since they aren't exclusively determined by these distributions (for instance, these datasets

have very low average degrees). Therefore, it doesn't align with the gain as well as most label-based metrics.

Furthermore, h_{NS} operates at the level of nodes ($\mathbf{D}^{\mathcal{N}}$), not at the level of classes. The empirical class compatibility matrix represents the expected neighborhood distribution of nodes in each class, which can be heterogeneous at the individual level, and h_{NS} collapses all pairwise node relations to a single scalar. This worked well in synthetic datasets because individual node distributions are directly sampled from $\mathbf{H}$, so neighborhood distributions were homogeneous for nodes of the same class, but it's not guaranteed in empirical datasets, meaning h_{NS} can be obscured by noisy node distributions. To counter this limitation, works like Wang et al. 2024 propose "separability gains" at the level of classes, a multi-dimensional object (per class pair) which enables a more fine-grained representation.

These results show how the gain prediction task in empirical datasets can be simplified to being able to estimate how (label-)homophilic a graph is (after accounting for class and degree imbalances by using normalized metrics). Even under label scarcity, the available labels provide the most useful signal, with $\hat{h}_{\text{unb}}$ being the best proxy (provided $lr \geq 0.10$, else $\hat{h}_{\text{class}}$) to indicate whether to use graph-aware or graph-agnostic models, especially in the homophilic and heterophilic regimes. In the neutral regime, gain predictions are less accurate and a deeper analysis of the graph's properties at the local level is generally required. Also, in the homophilic and heterophilic regimes, the label rate used for training is secondary in determining whether gains will be positive or negative, as shown in Figure 5.16, and the supervision-variance trade-off indicates that the more available labels, the more predictable the actual value of the gain will be, yet its sign remains generally unaffected.

Finally, we observe how measures that compress the information in the graph into a single scalar are inherently limited, and this effect is particularly visible when graphs are structurally heterogeneous. This suggests on the one hand the need for multi-dimensional gain proxies that don't summarize complex patterns to one unique quantity, which clearly comes at the expense of interpretability, and on the other hand, in a label scarce setting, the need to leverage the information in the available labels but also, selectively, the potentially useful signals coming from the graph's features and topology.

6. Discussion

This chapter maps the results to the research questions, compares the results with previous literature, and discusses the limitations the work.

RQ1: Descriptive task

The first research question asks what factors influence the graph-aware/graph-agnostic performance gap. Specifically, it aims to stress-test each model under label scarcity, and to understand what mechanisms influence whether message passing will provide an advantage in SSNC by making nodes more distinguishable, building from the common observations that homophily tends to be a beneficial condition for GNNs (Kipf and Welling 2017; Y. Ma et al. 2023; Luan, Hua, M. Xu, et al. 2024; Platonov, Kuznedelev, Diskin, et al. 2024), and that GNNs are generally robust under label scarcity because they leverage graph structure to compensate for low supervision (Subedi et al. 2025; Q. Li, Han, and X.-M. Wu 2018).

Through the synthetic pipeline, we show that the more accurate picture of the relationship between homophily and GNN performance is that homophily is a description of a graph's global connectivity tendency, and locally it defines how neighbor aggregation will work, but ultimately the main structural mechanism influencing whether message passing will be advantageous are the class-level neighborhood distributions. This is supported by works like Wang et al. 2024, whose HSBM model allows to design different heterophily patterns to show that class separability after aggregation is modulated by the distance between class neighborhood distributions and the graph's average node degree. Our instantiations of the CSBM-H model for the 2-class and multiclass settings support this claim, we were able to manufacture neighborhood patterns where, despite graphs being heterophilic, GNN accuracy is actually superior and showed this advantage occurs independently of the number of labeled nodes. Hence, label rate plays primarily the role of modulating gain variability, but not its behavior (nor its sign).

Furthermore, we observed that, depending on the number of classes and the corresponding class-compatibility matrix $\mathbf{H}$, there is a homophily level for which node distinguishability is the least after applying GCN feature aggregation. This point represents the greatest disadvantage of GCN w.r.t. MLP and is approximately $h = \frac{1}{C}$. Finally, through statistical testing, we concluded that the effect of label rate is comparatively much smaller than that of homophily in determining the advantage of graph-aware models, and that they jointly and not independently influence the gain. Therefore, the main role of the label rate is to regulate the effect of homophily.

Through the empirical pipeline, we tested the influence of homophily and supervision regime on the performance gap between the models in complex graphs. The perfor-

mance degradation analysis, especially on the neutral regime showed how the advantage of GCN is related to several structural properties beyond homophily, as the latter is approximately the same for datasets in this regime. Furthermore, we observed how, even in the presence of distinctive class distributions, factors like low degrees and feature informativeness play a role in the expected advantage of graph-aware models. Finally, we visualized the variance-supervision trade-off, and concluded that in homophily and heterophily, the gain tends to rise as the number of labels shrinks as a result of GCN accuracy degrading slower due to its ability to leverage graph structure when class distributions are informative. In this sense, our work is closest to Subedi et al. 2025, which associates GCN performance on the Amazon Computers dataset with the interaction between homophily and feature quality. Our work complements the study by analyzing datasets beyond the homophilic regime, which is known to be beneficial for GNNs and by studying the graph-aware model using its graph-agnostic counterpart as a baseline.

RQ2: Predictive task

The second research question asks how to reliably predict gain under label scarcity. Much effort has been put on estimating GNN performance (Wang et al. 2024; Luan 2023; Luan, Hua, M. Xu, et al. 2024), especially from the perspective of post-aggregation node distinguishability. We focus on estimating GNN performance before aggregation considering that in real-world datasets, we normally don't have access to the entire set of labels, so we test the robustness of commonly used metrics in predicting GNN performance using its graph-agnostic counterpart as reference.

Under the synthetic pipeline, results suggest that h^2 is the most robust label-scarce predictor (highest correlation with gain, lowest NRMSE), which is explained by the very consistent neighborhood patterns that the 2-class CSBM-H graphs models, on which 2-hop neighbors tend to share the same label as the ego node. From the feature- and structure-based measures h_{NS} is the strongest predictor, as it identifies the underlying mechanism that leads to high GCN performance, distinctive neighborhood distributions, and since confounders are controlled in the synthetic setting, we are able to pinpoint performance specifically to this property.

The empirical pipeline showed how the previous predictors, despite strongly aligning with the gain behavior in synthetic datasets, work conditionally and not universally. Specifically, when studying gain estimation in empirical datasets spanning the whole homophily spectrum, we observe that, since there are multiple factors that jointly influence node distinguishability, metrics that track a single graph property are inherently limited in their ability to estimate the gain. Nonetheless, results showed that label homophily (particularly measures that correct for class imbalances) is generally a good proxy for the performance of advantage of GCN in real-world datasets, despite the fact that it not directly the factor that governs it, as discussed previously. Therefore, in datasets with irregular neighborhood distributions, multiple classes and limited neighborhood information, the most useful signal still comes from the visible nodes' labels.

Finally, we posit that a good predictor should not only rely on a single property of the graph, but selectively incorporate information from structure, labels and features, as

these are informative about the potential performance of GNNs. This approach is similar to Tri-Hom (Zheng, Luan, and Lihui Chen 2024), a composite metric that incorporates feature and structure information into homophily estimation, but we highlight the need of such a measure to also work under limited label visibility. Such a gap, performance estimation under scarce labeling, was addressed in Luan, Hua, M. Xu, et al. 2024, whose hypothesis testing-based CPM metric suits the label sparse setting, but requires training additional classifiers to determine a threshold indicating whether aggregation is beneficial. Also, multi-dimensional gain proxies like CCNS (Y. Ma et al. 2023), which operates at the level of classes instead of collapsing complex patterns into a scalar, are inherently more expressive at the expense of interpretability, but have the potential to more accurately describe the mechanisms that influence node distinguishability.

Limitations

We acknowledge some limitations of the present work, as follows. First, the analysis is restricted to the two canonical neural architectures, the Graph Convolutional Network and the Multi-Layer Perceptron, which are widely studied in graph ML, yet a number of alternatives exist and remain to be studied from the label scarcity perspective. Depending on the application, this specialized architectures might be preferred, like H_2GCN (Y. Ma et al. 2023), which was proven to significantly boost performance in heterophilic datasets while remaining competitive in homophilic datasets. Secondly, we select specific architectural conditions to test the models and evaluate performance estimators, so further analysis needs to be performed to assess the extent to which the results generalize to e.g. deeper architectures, other message-passing mechanisms, or different graph ML tasks. Finally, the work identified the core mechanism that governs node distinguishability after aggregation while acknowledging it does not independently determine GNN performance, as proposed by works like Subedi et al. 2025 and Wang et al. 2024, a more fine-grained analysis of each of the additional factors (namely through ablation tests) in isolation is still required and is left open for future work.

7. Conclusion and Future Work

This work is a systematic study of the performance of GNNs under different dataset-specific and supervision-regime specific conditions. We provide a deeper understanding of the underlying mechanism behind GNN performance, characterize the advantage of graph-aware models, and present an approach to inform model selection under label scarcity conditions, which aligns to real-world scenarios where labeling is expensive. Ultimately, as mentioned in Chapter 1, the main objective goes beyond boosting predictive performance in SSNC, but rather about getting a more comprehensive understanding of why and when Graph Neural Networks work, especially in scenarios with imperfect or incomplete information. This has implications both on from the theoretical and practical points of view. From the theoretical perspective, studying the properties of a graph dataset that determine whether aggregation will be beneficial lets us design architectures that directly exploit those specific properties, bridging the gap between data, model and performance and making models more interpretable. From the practical point of view, understanding the conditions that make GNNs suitable helps practitioners during model selection and evaluation, potentially avoiding unnecessary complexity and increasing trust and reliability in real-world application.

The directions for future work are diverse. A natural continuation of this work is the design of estimators that are both robust under label scarcity and that incorporate signals from multiple sources beyond node labels. Here, we highlight the need for such performance proxies to be readily computable and ideally to work before model training to align with real-world scenarios. Second, an analysis of the generalization power of the derived conclusions should be made, on the one hand to other GNN architectures, particularly those designed to leverage specific properties of datasets like heterophily, and on the other hand to more diverse empirical networks (e.g. directed graphs, heterogeneous graphs). Third, to study GNN-style aggregation on more diverse datasets while preserving the ability to isolate and control for a graph's attributes (e.g. complex neighborhood patters, irregular degree distributions and different feature-generation paradigms), more expressive synthetic graph generative frameworks are a promising direction. Finally, this work naturally aligns with Explainable AI (XAI) in GNNs, which historically has been mainly concerned with post-hoc explanations of model predictions. Our approach for model selection precedes this strategy as it examines graph properties before predictions are made, so it is better characterized as model-/dataset-level explainability, but both approaches can be unified into a system that jointly recommends architectures based on dataset properties and links predictions to specific graph attributes for informed, human-interpretable model selection.

Bibliography

[1] Abu-El-Haija, Sami et al. (June 19, 2019). *MixHop: Higher-Order Graph Convolutional Architectures via Sparsified Neighborhood Mixing*. DOI: 10.48550/arXiv.1905.00067. arXiv: 1905.00067 [cs.LG]. URL: http://arxiv.org/abs/1905.00067 (visited on 08/24/2026). Pre-published.

[2] Ahmed, Amr et al. (May 13, 2013). "Distributed Large-Scale Natural Graph Factorization". In: *Proceedings of the 22nd International Conference on World Wide Web*. WWW '13: 22nd International World Wide Web Conference. Rio de Janeiro Brazil: ACM, pp. 37–48. ISBN: 978-1-4503-2035-1. DOI: 10.1145/2488388.2488393. URL: https://dl.acm.org/doi/10.1145/2488388.2488393 (visited on 08/08/2026).

[3] Arvind, V. et al. (May 4, 2015). *Graph Isomorphism, Color Refinement, and Compactness*. DOI: 10.48550/arXiv.1502.01255. arXiv: 1502.01255 [cs.CC]. URL: http://arxiv.org/abs/1502.01255 (visited on 08/15/2026). Pre-published.

[4] Barabasi, Albert-Laszlo and Reka Albert (Oct. 15, 1999). "Emergence of Scaling in Random Networks". In: *Science* 286.5439, pp. 509–512. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.286.5439.509. arXiv: cond-mat/9910332. URL: http://arxiv.org/abs/cond-mat/9910332 (visited on 08/05/2026).

[5] Barabási, Albert-László (Mar. 28, 2013). "Network Science". In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 371.1987, p. 20120375. ISSN: 1364-503X, 1471-2962. DOI: 10.1098/rsta.2012.0375. URL: https://royalsocietypublishing.org/doi/10.1098/rsta.2012.0375 (visited on 08/18/2026).

[6] Belkin, Mikhail, Partha Niyogi, and Vikas Sindhwani (2006). "Manifold Regularization: A Geometric Framework for Learning from Labeled and Unlabeled Examples". In.

[7] Blondel, Vincent D et al. (Oct. 1, 2008). "Fast Unfolding of Communities in Large Networks". In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10, P10008. ISSN: 1742-5468. DOI: 10.1088/1742-5468/2008/10/P10008. URL: https://iopscience.iop.org/article/10.1088/1742-5468/2008/10/P10008 (visited on 08/19/2026).

[8] Blum, Avrim and Tom Mitchell (July 24, 1998). "Combining Labeled and Unlabeled Data with Co-Training". In: *Proceedings of the Eleventh Annual Conference on Computational Learning Theory*. COLT98: The 11th Annual Conference on Computational Learning Theroy. Madison Wisconsin USA: ACM, pp. 92–100. ISBN: 978-1-58113-057-7. DOI: 10.1145/279943.279962. URL: https://dl.acm.org/doi/10.1145/279943.279962 (visited on 08/17/2026).

[9] Bo, Deyu et al. (2021). *Beyond Low-frequency Information in Graph Convolutional Networks*. Version 1. DOI: 10.48550/ARXIV.2101.00797. URL: https://arxiv.org/abs/2101.00797 (visited on 08/24/2026). Pre-published.

[10] Bonacich, Phillip (Mar. 1987). "Power and Centrality: A Family of Measures". In: *American Journal of Sociology* 92.5, pp. 1170–1182. ISSN: 0002-9602, 1537-5390. DOI: 10.1086/228631. URL: https://www.journals.uchicago.edu/doi/10.1086/228631 (visited on 08/18/2026).

[11] Bronstein, Michael M. et al. (May 2, 2021). *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. DOI: 10.48550/arXiv.2104.13478. arXiv: 2104.13478 [cs.LG]. URL: http://arxiv.org/abs/2104.13478 (visited on 08/11/2026). Pre-published.

[12] Bruna, Joan et al. (2013). *Spectral Networks and Locally Connected Networks on Graphs*. Version 3. DOI: 10.48550/ARXIV.1312.6203. URL: https://arxiv.org/abs/1312.6203 (visited on 08/24/2026). Pre-published.

[13] Cao, Bokai et al. (n.d.). "Collective Fraud Detection Capturing Inter-Transaction Dependency". In: ().

[14] Cao, Qi et al. (Nov. 27, 2019). *Popularity Prediction on Social Platforms with Coupled Graph Neural Networks*. DOI: 10.48550/arXiv.1906.09032. arXiv: 1906.09032 [cs.SI]. URL: http://arxiv.org/abs/1906.09032 (visited on 08/06/2026). Pre-published.

[15] Cao, Shaosheng, Wei Lu, and Qiongkai Xu (Oct. 17, 2015). "GraRep: Learning Graph Representations with Global Structural Information". In: *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*. CIKM'15: 24th ACM International Conference on Information and Knowledge Management. Melbourne Australia: ACM, pp. 891–900. ISBN: 978-1-4503-3794-6. DOI: 10.1145/2806416.2806512. URL: https://dl.acm.org/doi/10.1145/2806416.2806512 (visited on 08/08/2026).

[16] Cavallo, Andrea et al. (Dec. 26, 2022). *2-Hop Neighbor Class Similarity (2NCS): A Graph Structural Metric Indicative of Graph Neural Network Performance*. DOI: 10.48550/arXiv.2212.13202. arXiv: 2212.13202 [cs.LG]. URL: http://arxiv.org/abs/2212.13202 (visited on 07/30/2026). Pre-published.

[17] Chen, Yuhan et al. (June 20, 2023). *LSGNN: Towards General Graph Neural Network in Node Classification by Local Similarity*. DOI: 10.48550/arXiv.2305.04225. arXiv: 2305.04225 [cs.LG]. URL: http://arxiv.org/abs/2305.04225 (visited on 08/21/2026). Pre-published.

[18] Defferrard, Michaël, Xavier Bresson, and Pierre Vandergheynst (2016). "Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering". Version 3. In: DOI: 10.48550/ARXIV.1606.09375. URL: https://arxiv.org/abs/1606.09375 (visited on 08/24/2026).

[19] — (Feb. 5, 2017). *Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering*. DOI: 10.48550/arXiv.1606.09375. arXiv: 1606.09375 [cs.LG]. URL: http://arxiv.org/abs/1606.09375 (visited on 08/12/2026). Pre-published.

[20] Deshpande, Yash et al. (2018). *Contextual Stochastic Block Models*. Version 1. DOI: 10.48550/ARXIV.1807.09596. URL: https://arxiv.org/abs/1807.09596 (visited on 08/22/2026). Pre-published.

[21] Erdős, P. and A. Rényi (July 1, 2022). "On Random Graphs. I." In: *Publicationes Mathematicae Debrecen* 6.3–4, pp. 290–297. ISSN: 00333883. DOI: 10.5486/PMD.1959.6.3-4.12. URL: https://publi.math.unideb.hu/load_doi.php?pdoi=10_5486_PMD_1959_6_3_4_12 (visited on 08/22/2026).

[22] Gilbert, E. N. (1959). "Random Graphs". In: *The Annals of Mathematical Statistics* 30.4, pp. 1141–1144. JSTOR: 2237458. URL: http://www.jstor.org/stable/2237458.

[23] Gilmer, Justin et al. (June 12, 2017). *Neural Message Passing for Quantum Chemistry*. DOI: 10.48550/arXiv.1704.01212. arXiv: 1704.01212 [cs.LG]. URL: http://arxiv.org/abs/1704.01212 (visited on 08/06/2026). Pre-published.

[24] Gong, Chenghua et al. (2024). "A Survey on Learning from Graphs with Heterophily: Recent Advances and Future Directions". Version 4. In: DOI: 10.48550/ARXIV.2401.09769. URL: https://arxiv.org/abs/2401.09769 (visited on 08/24/2026).

[25] Grover, Aditya and Jure Leskovec (July 3, 2016). *Node2vec: Scalable Feature Learning for Networks*. DOI: 10.48550/arXiv.1607.00653. arXiv: 1607.00653 [cs.SI]. URL: http://arxiv.org/abs/1607.00653 (visited on 08/08/2026). Pre-published.

[26] Grover, Aditya, Aaron Zweig, and Stefano Ermon (May 15, 2019). *Graphite: Iterative Generative Modeling of Graphs*. DOI: 10.48550/arXiv.1803.10459. arXiv: 1803.10459 [stat.ML]. URL: http://arxiv.org/abs/1803.10459 (visited on 08/06/2026). Pre-published.

[27] Hamilton, William L (2020). "Graph Representation Learning". In.

[28] Hamilton, William L., Rex Ying, and Jure Leskovec (Sept. 10, 2018a). *Inductive Representation Learning on Large Graphs*. DOI: 10.48550/arXiv.1706.02216. arXiv: 1706.02216 [cs.SI]. URL: http://arxiv.org/abs/1706.02216 (visited on 07/30/2026). Pre-published.

[29] — (Apr. 10, 2018b). *Representation Learning on Graphs: Methods and Applications*. DOI: 10.48550/arXiv.1709.05584. arXiv: 1709.05584 [cs.SI]. URL: http://arxiv.org/abs/1709.05584 (visited on 08/06/2026). Pre-published.

[30] Hammond, David K., Pierre Vandergheynst, and Rémi Gribonval (Dec. 19, 2009). *Wavelets on Graphs via Spectral Graph Theory*. DOI: 10.48550/arXiv.0912.3848. arXiv: 0912.3848 [math.FA]. URL: http://arxiv.org/abs/0912.3848 (visited on 08/12/2026). Pre-published.

[31] He, Xiangnan et al. (July 7, 2020). *LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation*. DOI: 10.48550/arXiv.2002.02126. arXiv: 2002.02126 [cs.IR]. URL: http://arxiv.org/abs/2002.02126 (visited on 08/06/2026). Pre-published.

[32] Holland, Paul W., Kathryn Blackmond Laskey, and Samuel Leinhardt (June 1983). "Stochastic Blockmodels: First Steps". In: *Social Networks* 5.2, pp. 109–137. ISSN: 03788733. DOI: 10.1016/0378-8733(83)90021-7. URL: https://linkinghub.elsevier.com/retrieve/pii/0378873383900217 (visited on 08/06/2026).

[33] Hornik, Kurt, Maxwell Stinchcombe, and Halbert White (1990). "Universal Approximation of an Unknown Mapping and Its Derivatives Using Multilayer Feedforward Networks". In: *Neural Networks* 3.5, pp. 551–560. ISSN: 0893-6080. DOI: 10.1016/0893-6080(90)90005-6. URL: https://www.sciencedirect.com/science/article/pii/0893608090900056.

[34] Hu, Yang et al. (June 8, 2021). *Graph-MLP: Node Classification without Message Passing in Graph*. DOI: 10.48550/arXiv.2106.04051. arXiv: 2106.04051 [cs.LG]. URL: http://arxiv.org/abs/2106.04051 (visited on 08/10/2026). Pre-published.

[35] Huang, Qian et al. (Nov. 2, 2020). *Combining Label Propagation and Simple Models Out-performs Graph Neural Networks*. DOI: 10.48550/arXiv.2010.13993. arXiv: 2010.13993 [cs.LG]. URL: http://arxiv.org/abs/2010.13993 (visited on 07/30/2026). Pre-published.

[36] Hussain, Hussain et al. (Dec. 2021). "The Interplay between Communities and Homophily in Semi-Supervised Classification Using Graph Neural Networks". In: *Applied Network Science* 6.1, p. 80. ISSN: 2364-8228. DOI: 10.1007/s41109-021-00423-1. URL: https://appliednetsci.springeropen.com/articles/10.1007/s41109-021-00423-1 (visited on 07/30/2026).

[37] Jin, Di et al. (2022). *RAW-GNN: RAndom Walk Aggregation Based Graph Neural Network*. Version 1. DOI: 10.48550/ARXIV.2206.13953. URL: https://arxiv.org/abs/2206.13953 (visited on 08/21/2026). Pre-published.

[38] K. Rizi, Abbas et al. (Dec. 13, 2025). "Homophily within and across Groups". In: *Nature Communications* 16.1, p. 11351. ISSN: 2041-1723. DOI: 10.1038/s41467-025-66415-2. URL: https://www.nature.com/articles/s41467-025-66415-2 (visited on 07/30/2026).

[39] Katz, Leo (Mar. 1953). "A New Status Index Derived from Sociometric Analysis". In: *Psychometrika* 18.1, pp. 39–43. ISSN: 0033-3123, 1860-0980. DOI: 10.1007/BF02289026. URL: https://www.cambridge.org/core/product/identifier/S0033312300049000/type/journal_article (visited on 08/18/2026).

[40] Kernighan, B. W. and S. Lin (Feb. 1970). "An Efficient Heuristic Procedure for Partitioning Graphs". In: *Bell System Technical Journal* 49.2, pp. 291–307. ISSN: 00058580. DOI: 10.1002/j.1538-7305.1970.tb01770.x. URL: https://ieeexplore.ieee.org/document/6771089 (visited on 08/19/2026).

[41] Kingma, Diederik P. and Jimmy Ba (2014). *Adam: A Method for Stochastic Optimization*. Version 9. DOI: 10.48550/ARXIV.1412.6980. URL: https://arxiv.org/abs/1412.6980 (visited on 09/01/2026). Pre-published.

[42] Kipf, Thomas N and Max Welling (2017). "SEMI-SUPERVISED CLASSIFICATION WITH GRAPH CONVOLUTIONAL NETWORKS". In.

[43] Kohn, Matthias, Marcel Hoffmann, and Ansgar Scherp (Dec. 11, 2024). *Edge-Splitting MLP: Node Classification on Homophilic and Heterophilic Graphs without Message Passing*. DOI: 10.48550/arXiv.2412.08310. arXiv: 2412.08310 [cs.LG]. URL: http://arxiv.org/abs/2412.08310 (visited on 08/10/2026). Pre-published.

[44] Lee, Junseok et al. (2022). "GraFN: Semi-Supervised Node Classification on Graph with Few Labels via Non-Parametric Distribution Assignment". Version 3. In: DOI: 10.48550/ARXIV.2204.01303. URL: https://arxiv.org/abs/2204.01303 (visited on 08/26/2026).

[45] Li, Qimai, Zhichao Han, and Xiao-Ming Wu (Jan. 22, 2018). *Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning*. DOI: 10.48550/arXiv.1801.07606. arXiv: 1801.07606 [cs.LG]. URL: http://arxiv.org/abs/1801.07606 (visited on 07/30/2026). Pre-published.

[46] Li, Yayong, Jie Yin, and Ling Chen (2022). *Informative Pseudo-Labeling for Graph Neural Networks with Few Labels*. Version 1. DOI: 10.48550/ARXIV.2201.07951. URL: https://arxiv.org/abs/2201.07951 (visited on 08/26/2026). Pre-published.

[47] Lim, Derek, Felix Hohne, et al. (2021). *Large Scale Learning on Non-Homophilous Graphs: New Benchmarks and Strong Simple Methods*. Version 1. DOI: 10.48550/ARXIV.2110.14446. URL: https://arxiv.org/abs/2110.14446 (visited on 08/24/2026). Pre-published.

[48] Lim, Derek, Xiuyu Li, et al. (2021). *New Benchmarks for Learning on Non-Homophilous Graphs*. Version 2. DOI: 10.48550/ARXIV.2104.01404. URL: https://arxiv.org/abs/2104.01404 (visited on 08/31/2026). Pre-published.

[49] Luan, Sitao (2023). *On Addressing the Limitations of Graph Neural Networks*. Version 2. DOI: 10.48550/ARXIV.2306.12640. URL: https://arxiv.org/abs/2306.12640 (visited on 08/24/2026). Pre-published.

[50] Luan, Sitao, Chenqing Hua, Qincheng Lu, et al. (Nov. 3, 2023). *When Do We Need Graph Neural Networks for Node Classification?* DOI: 10.48550/arXiv.2210.16979. arXiv: 2210.16979 [cs.LG]. URL: http://arxiv.org/abs/2210.16979 (visited on 07/30/2026). Pre-published.

[51] Luan, Sitao, Chenqing Hua, Minkai Xu, et al. (Jan. 1, 2024). *When Do Graph Neural Networks Help with Node Classification? Investigating the Impact of Homophily Principle on Node Distinguishability*. DOI: 10.48550/arXiv.2304.14274. arXiv: 2304.14274 [cs.SI]. URL: http://arxiv.org/abs/2304.14274 (visited on 07/30/2026). Pre-published.

[52] Luan, Sitao, Mingde Zhao, Xiao-Wen Chang, et al. (2019). *Break the Ceiling: Stronger Multi-scale Deep Graph Convolutional Networks*. Version 3. DOI: 10.48550/ARXIV.1906.02174. URL: https://arxiv.org/abs/1906.02174 (visited on 08/24/2026). Pre-published.

[53] Luan, Sitao, Mingde Zhao, Chenqing Hua, et al. (2020). *Complete the Missing Half: Augmenting Aggregation Filtering with Diversification for Graph Convolutional Networks*. Version 4. DOI: 10.48550/ARXIV.2008.08844. URL: https://arxiv.org/abs/2008.08844 (visited on 08/24/2026). Pre-published.

[54] Ma, Yao et al. (July 21, 2023). *Is Homophily a Necessity for Graph Neural Networks?* DOI: 10.48550/arXiv.2106.06134. arXiv: 2106.06134 [cs.LG]. URL: http://arxiv.org/abs/2106.06134 (visited on 07/30/2026). Pre-published.

[55] Mernyei, Péter and Cătălina Cangea (Jan. 9, 2022). *Wiki-CS: A Wikipedia-Based Benchmark for Graph Neural Networks*. DOI: 10.48550/arXiv.2007.02901. arXiv: 2007.02901 [cs.LG]. URL: http://arxiv.org/abs/2007.02901 (visited on 07/30/2026). Pre-published.

[56] Mironov, Mikhail and Liudmila Prokhorenkova (2024). "Revisiting Graph Homophily Measures". In.

[57] Neville, Jennifer and David Jensen (n.d.). "Iterative Classification in Relational Data". In: ().

[58] Newman, M. E. J. (Oct. 28, 2002). "Assortative Mixing in Networks". In: *Physical Review Letters* 89.20, p. 208701. ISSN: 0031-9007, 1079-7114. DOI: 10.1103/PhysRevLett.89.208701. URL: https://link.aps.org/doi/10.1103/PhysRevLett.89.208701 (visited on 08/20/2026).

[59] Newman, Mark E. J. (2016). *Networks: An Introduction*. Reprinted. Oxford: Oxford University Press. 772 pp. ISBN: 978-0-19-920665-0.

[60] Ou, Mingdong et al. (Aug. 13, 2016). "Asymmetric Transitivity Preserving Graph Embedding". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16: The 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco California USA: ACM, pp. 1105–1114. ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939751. URL: https://dl.acm.org/doi/10.1145/2939672.2939751 (visited on 08/08/2026).

[61] Pearl, Judea (Feb. 28, 2022). "Reverend Bayes on Inference Engines: A Distributed Hierarchical Approach". In: *Probabilistic and Causal Inference*. Ed. by Hector Geffner, Rina Dechter, and Joseph Y. Halpern. 1st ed. New York, NY, USA: ACM, pp. 129–138. ISBN: 978-1-4503-9586-1. DOI: 10.1145/3501714.3501727. URL: https://dl.acm.org/doi/10.1145/3501714.3501727 (visited on 08/26/2026).

[62] Peel, Leto, Jean-Charles Delvenne, and Renaud Lambiotte (Apr. 17, 2018). "Multiscale Mixing Patterns in Networks". In: *Proceedings of the National Academy of Sciences* 115.16, pp. 4057–4062. ISSN: 0027-8424, 1091-6490. DOI: 10.1073/pnas.1713019115. URL: https://pnas.org/doi/full/10.1073/pnas.1713019115 (visited on 08/20/2026).

[63] Pei, Hongbin et al. (Feb. 14, 2020). *Geom-GCN: Geometric Graph Convolutional Networks*. DOI: 10.48550/arXiv.2002.05287. arXiv: 2002.05287 [cs.LG]. URL: http://arxiv.org/abs/2002.05287 (visited on 07/30/2026). Pre-published.

[64] Perozzi, Bryan, Rami Al-Rfou, and Steven Skiena (Aug. 24, 2014). "DeepWalk: Online Learning of Social Representations". In: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 701–710. DOI: 10.1145/2623330.2623732. arXiv: 1403.6652 [cs.SI]. URL: http://arxiv.org/abs/1403.6652 (visited on 08/08/2026).

[65] Platonov, Oleg, Denis Kuznedelev, Artem Babenko, et al. (Apr. 15, 2024). *Characterizing Graph Datasets for Node Classification: Homophily-Heterophily Dichotomy and Beyond*. DOI: 10.48550/arXiv.2209.06177. arXiv: 2209.06177 [cs.SI]. URL: http://arxiv.org/abs/2209.06177 (visited on 07/30/2026). Pre-published.

[66] Platonov, Oleg, Denis Kuznedelev, Michael Diskin, et al. (Mar. 2, 2024). *A Critical Look at the Evaluation of GNNs under Heterophily: Are We Really Making Progress?* DOI: 10.48550/arXiv.2302.11640. arXiv: 2302.11640 [cs.LG]. URL: http://arxiv.org/abs/2302.11640 (visited on 07/30/2026). Pre-published.

[67] Rozemberczki, Benedek, Carl Allen, and Rik Sarkar (Mar. 21, 2021). *Multi-Scale Attributed Node Embedding*. DOI: 10.48550/arXiv.1909.13021. arXiv: 1909.13021 [cs.LG]. URL: http://arxiv.org/abs/1909.13021 (visited on 07/30/2026). Pre-published.

[68] Rozemberczki, Benedek and Rik Sarkar (Feb. 16, 2021). *Twitch Gamers: A Dataset for Evaluating Proximity Preserving and Structural Role-based Node Embeddings*. DOI: 10.48550/arXiv.2101.03091. arXiv: 2101.03091 [cs.SI]. URL: http://arxiv.org/abs/2101.03091 (visited on 08/06/2026). Pre-published.

[69] Rubin, Jonathan, Sahil Loomba, and Nick S. Jones (Aug. 25, 2025). *Limits of Message Passing for Node Classification: How Class-Bottlenecks Restrict Signal-to-Noise Ratio*. DOI: 10.48550/arXiv.2508.17822. arXiv: 2508.17822 [cs.LG]. URL: http://arxiv.org/abs/2508.17822 (visited on 07/30/2026). Pre-published.

[70] Rumelhart, David E, Geoffrey E Hintont, and Ronald J Williams (1986). "Learning Representations by Back-Propagating Errors". In.

[71] Sandryhaila, Aliaksei and Jose M. F. Moura (Apr. 2013). "Discrete Signal Processing on Graphs". In: *IEEE Transactions on Signal Processing* 61.7, pp. 1644–1656. ISSN: 1053-587X, 1941-0476. DOI: 10.1109/TSP.2013.2238935. arXiv: 1210.4752 [cs.SI]. URL: http://arxiv.org/abs/1210.4752 (visited on 08/12/2026).

[72] Shchur, Oleksandr et al. (June 18, 2019). *Pitfalls of Graph Neural Network Evaluation*. DOI: 10.48550/arXiv.1811.05868. arXiv: 1811.05868 [cs.LG]. URL: http://arxiv.org/abs/1811.05868 (visited on 07/30/2026). Pre-published.

[73] Subedi, Nischal et al. (Dec. 15, 2025). *Understanding When Graph Convolutional Networks Help: A Diagnostic Study on Label Scarcity and Structural Properties*. DOI: 10.48550/arXiv.2512.12947. arXiv: 2512.12947 [cs.LG]. URL: http://arxiv.org/abs/2512.12947 (visited on 07/30/2026). Pre-published.

[74] Suresh, Susheel et al. (2021). "Breaking the Limit of Graph Neural Networks by Improving the Assortativity of Graphs with Local Mixing Patterns". Version 1. In: DOI: 10.48550/ARXIV.2106.06586. URL: https://arxiv.org/abs/2106.06586 (visited on 08/25/2026).

[75] Thost, Veronika and Jie Chen (2021). *Directed Acyclic Graph Neural Networks*. Version 3. DOI: 10.48550/ARXIV.2101.07965. URL: https://arxiv.org/abs/2101.07965 (visited on 08/26/2026). Pre-published.

[76] Traag, V. A., L. Waltman, and N. J. Van Eck (Mar. 26, 2019). "From Louvain to Leiden: Guaranteeing Well-Connected Communities". In: *Scientific Reports* 9.1, p. 5233. ISSN: 2045-2322. DOI: 10.1038/s41598-019-41695-z. URL: https://www.nature.com/articles/s41598-019-41695-z (visited on 08/19/2026).

[77] Vaswani, Ashish et al. (Aug. 2, 2023). *Attention Is All You Need*. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs.CL]. URL: http://arxiv.org/abs/1706.03762 (visited on 08/14/2026). Pre-published.

[78] Veličković, Petar et al. (Feb. 4, 2018). *Graph Attention Networks*. DOI: 10.48550/arXiv.1710.10903. arXiv: 1710.10903 [stat.ML]. URL: http://arxiv.org/abs/1710.10903 (visited on 07/30/2026). Pre-published.

[79] Vignac, Clement et al. (May 23, 2023). *DiGress: Discrete Denoising Diffusion for Graph Generation*. DOI: 10.48550/arXiv.2209.14734. arXiv: 2209.14734 [cs.LG]. URL: http://arxiv.org/abs/2209.14734 (visited on 08/06/2026). Pre-published.

[80] Wang, Junfu et al. (June 4, 2024). *Understanding Heterophily for Graph Neural Networks*. DOI: 10.48550/arXiv.2401.09125. arXiv: 2401.09125 [cs.LG]. URL: http://arxiv.org/abs/2401.09125 (visited on 07/30/2026). Pre-published.

[81] Wu, Felix et al. (June 20, 2019). *Simplifying Graph Convolutional Networks*. DOI: 10.48550/arXiv.1902.07153. arXiv: 1902.07153 [cs.LG]. URL: http://arxiv.org/abs/1902.07153 (visited on 08/11/2026). Pre-published.

[82] Wu, Qitian et al. (2023). *NodeFormer: A Scalable Graph Structure Learning Transformer for Node Classification*. Version 1. DOI: 10.48550/ARXIV.2306.08385. URL: https://arxiv.org/abs/2306.08385 (visited on 08/24/2026). Pre-published.

[83] Wu, Yuexin et al. (Nov. 19, 2021). *Active Learning for Graph Neural Networks via Node Feature Propagation*. DOI: 10.48550/arXiv.1910.07567. arXiv: 1910.07567 [cs.LG]. URL: http://arxiv.org/abs/1910.07567 (visited on 07/30/2026). Pre-published.

[84] Xu, Keyulu et al. (Feb. 22, 2019). *How Powerful Are Graph Neural Networks?* DOI: 10.48550/arXiv.1810.00826. arXiv: 1810.00826 [cs.LG]. URL: http://arxiv.org/abs/1810.00826 (visited on 08/06/2026). Pre-published.

[85] Yang, Han, Kaili Ma, and James Cheng (Dec. 20, 2020). *Rethinking Graph Regularization for Graph Neural Networks*. DOI: 10.48550/arXiv.2009.02027. arXiv: 2009.02027 [cs.LG]. URL: http://arxiv.org/abs/2009.02027 (visited on 08/24/2026). Pre-published.

[86] Yang, Zhilin, William W. Cohen, and Ruslan Salakhutdinov (May 26, 2016). *Revisiting Semi-Supervised Learning with Graph Embeddings*. DOI: 10.48550/arXiv.1603.08861. arXiv: 1603.08861 [cs.LG]. URL: http://arxiv.org/abs/1603.08861 (visited on 07/30/2026). Pre-published.

[87] You, Jiaxuan, Rex Ying, and Jure Leskovec (July 23, 2021). *Design Space for Graph Neural Networks*. DOI: 10.48550/arXiv.2011.08843. arXiv: 2011.08843 [cs.LG]. URL: http://arxiv.org/abs/2011.08843 (visited on 07/30/2026). Pre-published.

[88] You, Jiaxuan, Rex Ying, Xiang Ren, et al. (June 23, 2018). *GraphRNN: Generating Realistic Graphs with Deep Auto-regressive Models*. DOI: 10.48550/arXiv.1802.08773. arXiv: 1802.08773 [cs.LG]. URL: http://arxiv.org/abs/1802.08773 (visited on 08/06/2026). Pre-published.

[89] Zhang, Baoming et al. (2025). "Normalize Then Propagate: Efficient Homophilous Regularization for Few-shot Semi-Supervised Node Classification". Version 1. In: DOI: 10.48550/ARXIV.2501.08581. URL: https://arxiv.org/abs/2501.08581 (visited on 08/26/2026).

[90] Zheng, Yilun, Sitao Luan, and Lihui Chen (June 27, 2024). *What Is Missing In Homophily? Disentangling Graph Homophily For Graph Neural Networks*. DOI: 10.48550/arXiv.2406.18854. arXiv: 2406.18854 [cs.LG]. URL: http://arxiv.org/abs/2406.18854 (visited on 07/30/2026). Pre-published.

[91] Zhou, Dengyong et al. (2003). "Learning with Local and Global Consistency". In: *Advances in Neural Information Processing Systems*. Ed. by S. Thrun, L. Saul, and B. Schölkopf. Vol. 16. MIT Press. URL: https://proceedings.neurips.cc/paper_files/paper/2003/file/87682805257e619d49b8e0dfdc14affa-Paper.pdf.

[92] Zhou, Ziang et al. (June 2023). "Effective Stabilized Self-Training on Few-Labeled Graph Data". In: *Information Sciences* 631, pp. 369–384. ISSN: 00200255. DOI: 10.1016/j.ins.2023.02.032. arXiv: 1910.02684 [cs.LG]. URL: http://arxiv.org/abs/1910.02684 (visited on 07/30/2026).

[93] Zhu, Jiong, Ryan A. Rossi, et al. (June 14, 2021). *Graph Neural Networks with Heterophily*. DOI: 10.48550/arXiv.2009.13566. arXiv: 2009.13566 [cs.LG]. URL: http://arxiv.org/abs/2009.13566 (visited on 07/30/2026). Pre-published.

[94] Zhu, Jiong, Yujun Yan, et al. (Oct. 23, 2020). *Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs*. DOI: 10.48550/arXiv.2006.11468. arXiv: 2006.11468 [cs.LG]. URL: http://arxiv.org/abs/2006.11468 (visited on 07/30/2026). Pre-published.

[95] Zhu, Xiaojin (2008). "Semi-Supervised Learning Literature Survey". In.

[96] Zhu, Xiaojin and Zoubin Ghahramani (2002). *Learning From Labeled and Unlabeled Data with Label Propagation*. Technical Report CMU-CALD-02-107. Carnegie Mellon University. URL: http://pages.cs.wisc.edu/~jerryzhu/pub/CMU-CALD-02-107.pdf.

[97] Zhu, Xiaojin and Andrew B. Goldberg (2009). *Introduction to Semi-Supervised Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Cham: Springer International Publishing. ISBN: 978-3-031-00420-9 978-3-031-01548-9. DOI: 10.1007/978-3-031-01548-9. URL: https://link.springer.com/10.1007/978-3-031-01548-9 (visited on 08/17/2026).

A. ANOVA tables for the multiclass CSBM-H

Table A.1.: One-Way Repeated Measures ANOVA results for $C = 3$.

h	DF_n	DF_d	F	p	η_G^2
0.0	3.02	299.23	7.27	<0.001***	0.050
0.1	2.75	272.05	0.47	0.687	0.003
0.2	2.67	264.59	0.13	0.925	0.001
0.3	2.73	270.53	1.03	0.374	0.008
0.4	2.47	244.30	1.64	0.189	0.013
0.5	2.57	254.52	0.63	0.574	0.005
0.6	3.08	304.57	8.77	<0.001***	0.064
0.7	3.09	305.43	22.30	<0.001***	0.145
0.8	2.94	290.94	42.81	<0.001***	0.247
0.9	3.08	304.56	54.44	<0.001***	0.293
1.0	3.07	303.59	55.78	<0.001***	0.298

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

Table A.2.: One-Way Repeated Measures ANOVA results for $C = 4$.

h	DF_n	DF_d	F	p	η_G^2
0.0	2.75	272.24	1.06	0.363	0.008
0.1	2.54	251.38	1.99	0.126	0.014
0.2	2.73	269.93	3.91	0.012*	0.028
0.3	2.36	233.51	0.45	0.673	0.003
0.4	2.11	208.43	1.83	0.161	0.013
0.5	2.58	255.05	7.32	<0.001***	0.050
0.6	2.56	253.41	27.62	<0.001***	0.175
0.7	2.55	252.60	85.46	<0.001***	0.394
0.8	2.66	263.07	130.90	<0.001***	0.497
0.9	2.66	263.32	147.52	<0.001***	0.524
1.0	2.62	259.17	148.65	<0.001***	0.525

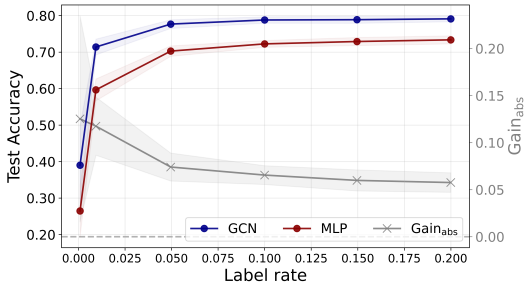
*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

Table A.3.: One-Way Repeated Measures ANOVA results for $C = 5$.

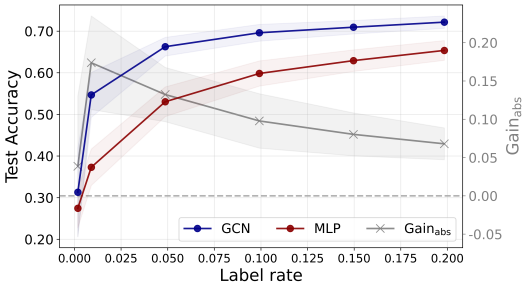
h	DF_n	DF_d	F	p	η^2_G
0.0	3.11	308.26	2.86	0.035*	0.020
0.1	3.13	309.54	16.73	<0.001***	0.113
0.2	3.18	314.63	7.97	<0.001***	0.055
0.3	3.19	316.02	4.90	0.002**	0.034
0.4	2.69	266.29	3.48	0.020*	0.024
0.5	2.74	271.35	16.63	<0.001***	0.102
0.6	2.91	288.07	76.22	<0.001***	0.358
0.7	2.97	293.87	173.61	<0.001***	0.555
0.8	2.76	272.81	294.19	<0.001***	0.681
0.9	2.75	271.91	345.53	<0.001***	0.709
1.0	2.71	268.21	353.71	<0.001***	0.714

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

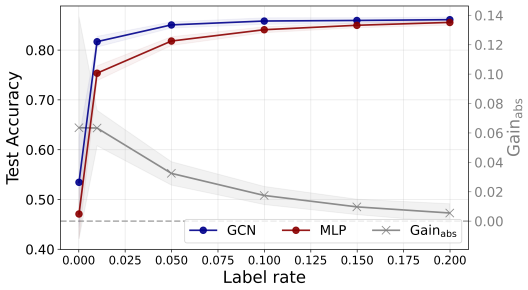
B. Accuracy and gain plots of empirical datasets



(a) WikiCS ($h_{\text{unb}} = 0.8569$).

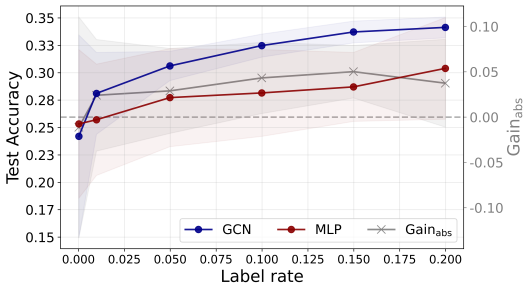


(b) CiteSeer ($h_{\text{unb}} = 0.8520$).

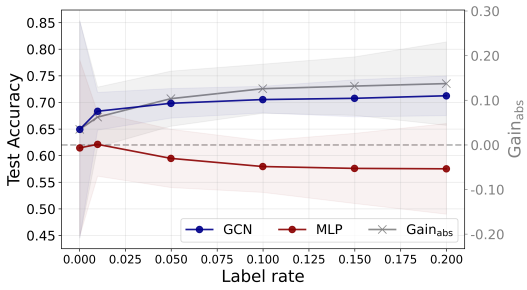


(c) PubMed ($h_{\text{unb}} = 0.7653$).

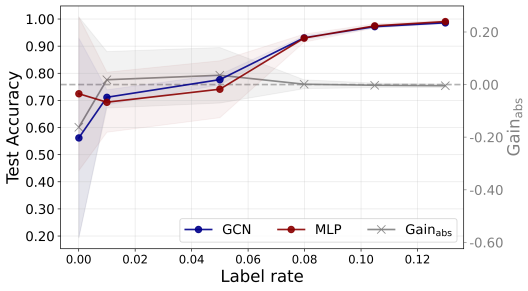
Figure B.1.: Homophilic datasets.



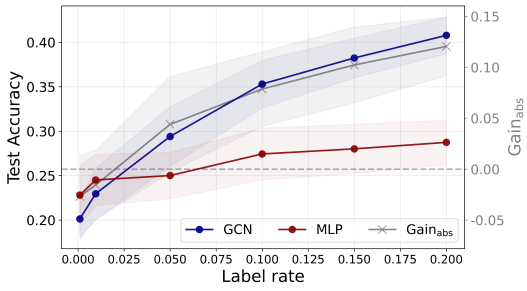
(a) Amazon-ratings ($h_{\text{unb}} = 0.2769$).



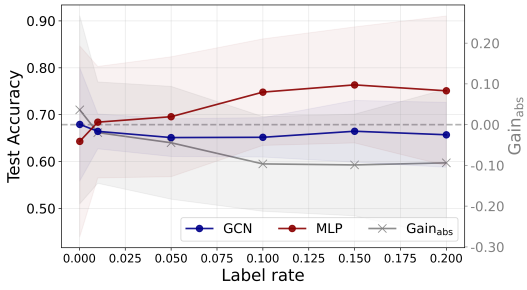
(b) Tolokers ($h_{\text{unb}} = 0.1016$).



(c) Questions ($h_{\text{unb}} = 0.0583$).

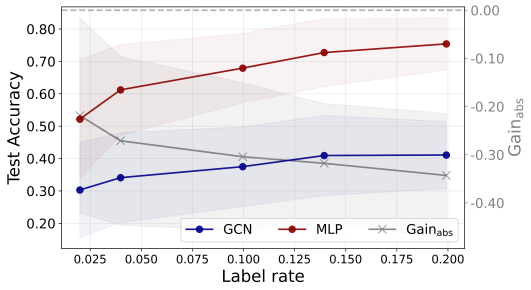


(d) Squirrel ($h_{\text{unb}} = 0.0270$).

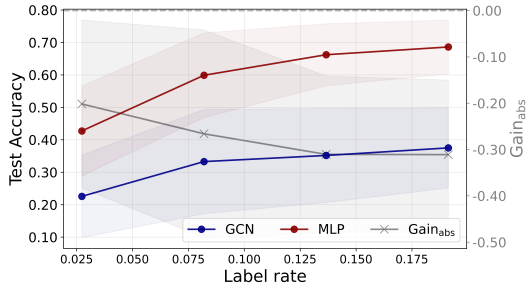


(e) Minesweeper ($h_{\text{unb}} = 0.0145$).

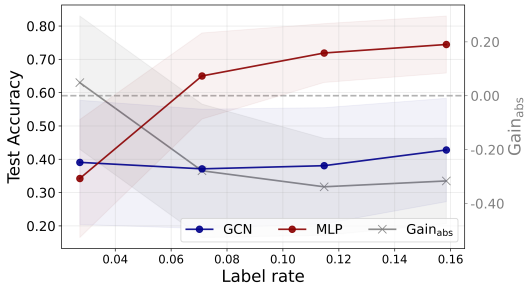
Figure B.2.: Neutral datasets.



(a) Wisconsin ($h_{\text{unb}} = -0.2851$).



(b) Cornell ($h_{\text{unb}} = -0.4675$).



(c) Texas ($h_{\text{unb}} = -0.8050$).

Figure B.3.: Heterophilic datasets.